Digital Image Watermarking

Project source code

A hybrid DWT, DCT and SVD approach

Krishang Vikram Singh

Roll No. B106 Batch C1

B.Tech Computer Engineering, NMIMS MPSTME

Subject: Image and Video Processing

Language: Python 3.13 Main libraries: NumPy, SciPy, scikit-image, PyWavelets, Pillow, pandas, matplotlib

Optional: OpenCV and FFmpeg (video test), blind-watermark (comparison)

Contents

Part 1: Core algorithm **4**
1.1 watermark_core.py 4
1.2 metrics.py 5
1.3 attacks.py 6
1.4 logo.py 7
1.5 image_io.py 8
Part 2: Dataset **10**
2.1 dataset_manifest.py 10
2.2 build_dataset.py 12
Part 3: Experiments **16**
3.1 run_experiment.py 16
3.2 run_batch_experiment.py 18
3.3 video_demo.py 21
3.4 exp_jpeg_truncation.py 23
3.5 exp_principal_component_variant.py 24
Part 4: Analysis and benchmarks **25**
4.1 analysis.py 25
4.2 analysis_tech.py 30
4.3 benchmark_tech.py 31
4.4 make_diagram.py 38
4.5 make_paper_figures.py 39
Part 5: Document generators **41**
5.1 docx_helpers.py 41
5.2 build_report.py 45
5.3 build_deck.py 59
5.4 build_paper.py 70
5.5 validate_submission.py 78
Part 6: Tests **80**
6.1 conftest.py 80
6.2 test_watermark_core.py 80
6.3 test_metrics.py 81
6.4 test_attacks.py 82
6.5 test_image_io.py 82
6.6 test_dataset_and_results.py 83
Part 7: Configuration **85**
7.1 config.json 85
7.2 requirements.txt 85

How the project is organised

The code is grouped into seven parts. The core algorithm is small and has no dependency on the other parts. The dataset builder prepares the images, the experiment scripts run the algorithm on them and record results, and the analysis scripts turn the recorded results into tables and figures. The document generators assemble the report, presentation and paper from those results. Tests check the algorithm, the loaders and the completeness of the results.

File	Part	Purpose
watermark_core.py	1. Core algorithm	Hybrid DWT, DCT and SVD embedding and extraction
metrics.py	1. Core algorithm	PSNR, SSIM, MSE, NC, ZNCC and BER
attacks.py	1. Core algorithm	Fourteen attacks plus the unattacked reference
logo.py	1. Core algorithm	Procedural 64x64 binary watermark logo
image_io.py	1. Core algorithm	Safe image loading and standardisation
dataset_manifest.py	2. Dataset	List of source images, licences and categories
build_dataset.py	2. Dataset	Downloads, standardises and records the 100 images
run_experiment.py	3. Experiments	Original two image experiment
run_batch_experiment.py	3. Experiments	Batch experiment on 100 images
video_demo.py	3. Experiments	Per frame video test
exp_jpeg_truncation.py	3. Experiments	Truncation versus rounding before JPEG
exp_principal_component_variant.py	3. Experiments	Principal component embedding variant
analysis.py	4. Analysis and benchmarks	Tables and figures for the main results
analysis_tech.py	4. Analysis and benchmarks	Figures for the technology comparison
benchmark_tech.py	4. Analysis and benchmarks	Library and scheme benchmarks
make_diagram.py	4. Analysis and benchmarks	Pipeline diagrams
make_paper_figures.py	4. Analysis and benchmarks	Column width figures for the paper
docx_helpers.py	5. Document generators	Helper functions for Word documents
build_report.py	5. Document generators	Builds the project report
build_deck.py	5. Document generators	Builds the presentation
build_paper.py	5. Document generators	Builds the research paper
validate_submission.py	5. Document generators	Checks the finished package
conftest.py	6. Tests	Test configuration
test_watermark_core.py	6. Tests	Tests for embedding and extraction
test_metrics.py	6. Tests	Tests for the metrics
test_attacks.py	6. Tests	Tests for the attacks
test_image_io.py	6. Tests	Tests for image loading
test_dataset_and_results.py	6. Tests	Checks on the dataset and results
config.json	7. Configuration	Experiment settings
requirements.txt	7. Configuration	Python packages used

Order in which the main scripts are run

1. build_dataset.py: downloads and standardises the 100 images and writes the metadata table.
2. run_batch_experiment.py: embeds the watermark, applies the attacks, extracts and records every result.
3. analysis.py: statistics, tables and figures from the recorded results.
4. benchmark_tech.py and analysis_tech.py: library and scheme comparisons.
5. video_demo.py: optional per frame video test.
6. pytest tests: runs the automated tests.

Part 1: Core algorithm

The watermarking method itself: embedding, extraction, metrics, attacks, logo and image loading.

1.1 watermark_core.py

Hybrid DWT, DCT and SVD embedding and extraction. 93 lines.

```
from __future__ import annotations

import numpy as np
import pywt
from scipy.fftpack import dct, idct
from skimage.transform import resize

_SUBBANDS = ("LL", "LH", "HL", "HH")


def dct2(block: np.ndarray) -> np.ndarray:
    return dct(dct(block.T, norm="ortho").T, norm="ortho")


def idct2(block: np.ndarray) -> np.ndarray:
    return idct(idct(block.T, norm="ortho").T, norm="ortho")


def _check_2d(name: str, a: np.ndarray, min_side: int = 8) -> np.ndarray:
    a = np.asarray(a)
    if a.ndim != 2:
        raise ValueError(f"{name} must be a 2-D greyscale array, got shape {a.shape}")
    if min(a.shape) < min_side:
        raise ValueError(f"{name} too small: {a.shape}")
    if not np.issubdtype(a.dtype, np.number):
        raise TypeError(f"{name} must be numeric, got {a.dtype}")
    if not np.isfinite(a).all():
        raise ValueError(f"{name} contains NaN/Inf")
    return a.astype(np.float64)


def embed_watermark(host: np.ndarray, watermark: np.ndarray, alpha: float = 0.10,
                    wavelet: str = "haar", subband: str = "LL"):
    host = _check_2d("host", host)
    watermark = _check_2d("watermark", watermark)
    if not alpha > 0:
        raise ValueError(f"alpha must be > 0, got {alpha}")
    if subband not in _SUBBANDS:
        raise ValueError(f"subband must be one of {_SUBBANDS}, got {subband!r}")

    LL, (LH, HL, HH) = pywt.dwt2(host, wavelet)
    subbands = {"LL": LL, "LH": LH, "HL": HL, "HH": HH}
    target = subbands[subband]

    wm_resized = resize(watermark, target.shape, anti_aliasing=True, preserve_range=True)

    target_dct = dct2(target)

    U, S, Vt = np.linalg.svd(target_dct, full_matrices=False)
    Uw, Sw, Vtw = np.linalg.svd(wm_resized, full_matrices=False)

    S_mod = S + alpha * Sw

    target_dct_mod = (U * S_mod) @ Vt
    target_mod = idct2(target_dct_mod)

    subbands[subband] = target_mod
    watermarked = pywt.idwt2((subbands["LL"], (subbands["LH"], subbands["HL"], subbands["HH"])),
wavelet)

    watermarked = watermarked[:host.shape[0], :host.shape[1]]
    watermarked = np.clip(watermarked, 0, 255)

    key = {
        "U": U, "Vt": Vt, "S_original": S,
        "Uw": Uw, "Vtw": Vtw,
        "alpha": alpha, "wavelet": wavelet, "subband": subband,
        "wm_shape": watermark.shape, "target_shape": target.shape,
```

```
    }
    return watermarked, key


def extract_watermark(possibly_attacked: np.ndarray, key: dict) -> np.ndarray:
    img = _check_2d("image", possibly_attacked)
    wavelet = key["wavelet"]
    subband = key["subband"]

    LL, (LH, HL, HH) = pywt.dwt2(img, wavelet)
    subbands = {"LL": LL, "LH": LH, "HL": HL, "HH": HH}
    target = subbands[subband]

    target_shape = key["target_shape"]
    if target.shape != target_shape:
        target = resize(target, target_shape, anti_aliasing=True, preserve_range=True)

    target_dct = dct2(target)
    _, S_att, _ = np.linalg.svd(target_dct, full_matrices=False)

    Sw_extracted = (S_att - key["S_original"]) / key["alpha"]
    watermark_extracted = (key["Uw"] * Sw_extracted) @ key["Vtw"]

    watermark_extracted = resize(watermark_extracted, key["wm_shape"],
                                 anti_aliasing=True, preserve_range=True)
    return np.clip(watermark_extracted, 0, 255)
```

1.2 metrics.py

PSNR, SSIM, MSE, NC, ZNCC and BER. 63 lines.

```
import numpy as np
from skimage.metrics import peak_signal_noise_ratio as _psnr
from skimage.metrics import structural_similarity as _ssim

PSNR_CAP_DB = 100.0


def _prep(a: np.ndarray, b: np.ndarray):
    a = np.clip(np.asarray(a, dtype=np.float64), 0, 255)
    b = np.clip(np.asarray(b, dtype=np.float64), 0, 255)
    if a.shape != b.shape:
        raise ValueError(f"shape mismatch: {a.shape} vs {b.shape}")
    return a, b


def mse(original: np.ndarray, test: np.ndarray) -> float:
    a, b = _prep(original, test)
    return float(np.mean((a - b) ** 2))


def psnr(original: np.ndarray, test: np.ndarray) -> float:
    a, b = _prep(original, test)
    if np.array_equal(a, b):
        return PSNR_CAP_DB
    return float(min(_psnr(a, b, data_range=255), PSNR_CAP_DB))


def ssim(original: np.ndarray, test: np.ndarray) -> float:
    a, b = _prep(original, test)
    return float(_ssim(a, b, data_range=255))


def normalized_correlation(original_wm: np.ndarray, extracted_wm: np.ndarray) -> float:
    a = np.asarray(original_wm, dtype=np.float64).flatten()
    b = np.asarray(extracted_wm, dtype=np.float64).flatten()
    if a.shape != b.shape:
        raise ValueError(f"shape mismatch: {original_wm.shape} vs {extracted_wm.shape}")
    num = np.sum(a * b)
    den = np.sqrt(np.sum(a ** 2)) * np.sqrt(np.sum(b ** 2))
    if den == 0:
        return 0.0
    return float(num / den)


def zero_normalized_correlation(original_wm: np.ndarray, extracted_wm: np.ndarray) -> float:
    a = np.asarray(original_wm, dtype=np.float64).flatten()
    b = np.asarray(extracted_wm, dtype=np.float64).flatten()
```

```
    if a.shape != b.shape:
        raise ValueError(f"shape mismatch: {original_wm.shape} vs {extracted_wm.shape}")
    a = a - a.mean()
    b = b - b.mean()
    den = np.sqrt(np.sum(a ** 2)) * np.sqrt(np.sum(b ** 2))
    if den == 0:
        return 0.0
    return float(np.sum(a * b) / den)


def bit_error_rate(original_wm: np.ndarray, extracted_wm: np.ndarray, thresh: float = 127.5) ->
float:
    a = (np.asarray(original_wm, dtype=np.float64) > thresh).astype(np.uint8).flatten()
    b = (np.asarray(extracted_wm, dtype=np.float64) > thresh).astype(np.uint8).flatten()
    if a.shape != b.shape:
        raise ValueError(f"shape mismatch: {original_wm.shape} vs {extracted_wm.shape}")
    return float(np.mean(a != b))
```

1.3 attacks.py

Fourteen attacks plus the unattacked reference. 112 lines.

```
from __future__ import annotations

import io

import numpy as np
from PIL import Image
from scipy import ndimage
from skimage.exposure import equalize_hist
from skimage.transform import rotate, resize
from skimage.util import random_noise


def jpeg_compression(img: np.ndarray, quality: int = 50) -> np.ndarray:
    if not 1 <= quality <= 95:
        raise ValueError("quality must be in 1..95")
    im = Image.fromarray(np.clip(np.rint(img), 0, 255).astype(np.uint8))
    buf = io.BytesIO()
    im.save(buf, format="JPEG", quality=quality)
    buf.seek(0)
    return np.array(Image.open(buf).convert("L")).astype(np.float64)


def gaussian_noise(img: np.ndarray, var: float = 0.01, rng=None) -> np.ndarray:
    noisy = random_noise(img / 255.0, mode="gaussian", var=var, rng=rng)
    return noisy * 255.0


def salt_pepper_noise(img: np.ndarray, amount: float = 0.02, rng=None) -> np.ndarray:
    noisy = random_noise(img / 255.0, mode="s&p", amount=amount, rng=rng)
    return noisy * 255.0


def median_filter(img: np.ndarray, size: int = 3) -> np.ndarray:
    return ndimage.median_filter(img, size=size)


def gaussian_blur(img: np.ndarray, sigma: float = 1.5) -> np.ndarray:
    return ndimage.gaussian_filter(img, sigma=sigma)


def rotation_attack(img: np.ndarray, angle: float = 5.0) -> np.ndarray:
    rotated = rotate(img / 255.0, angle, resize=False, mode="edge")
    return rotated * 255.0


def cropping_attack(img: np.ndarray, frac: float = 0.25) -> np.ndarray:
    h, w = img.shape
    ch, cw = int(h * frac / 2), int(w * frac / 2)
    out = img.copy()
    out[:ch, :] = 0
    out[h - ch:, :] = 0
    out[:, :cw] = 0
    out[:, w - cw:] = 0
    return out
```

```
def scaling_attack(img: np.ndarray, factor: float = 0.5) -> np.ndarray:
    h, w = img.shape
    small = resize(img, (int(h * factor), int(w * factor)), anti_aliasing=True, preserve_range=True)
    back = resize(small, (h, w), anti_aliasing=True, preserve_range=True)
    return back


def histogram_equalization(img: np.ndarray) -> np.ndarray:
    eq = equalize_hist(img / 255.0)
    return eq * 255.0


def sharpening_attack(img: np.ndarray, amount: float = 1.0) -> np.ndarray:
    blurred = ndimage.gaussian_filter(img, sigma=1.0)
    sharpened = img + amount * (img - blurred)
    return np.clip(sharpened, 0, 255)


ATTACK_NAMES = [
    "No Attack", "JPEG Q=90", "JPEG Q=50", "JPEG Q=20",
    "Gaussian Noise (var=0.005)", "Gaussian Noise (var=0.02)", "Salt & Pepper (2%)",
    "Median Filter 3x3", "Gaussian Blur (sigma=1.5)", "Rotation 5 deg", "Rotation 15 deg",
    "Cropping 25%", "Scaling 0.5x", "Histogram Equalization", "Sharpening",
]

ATTACK_FAMILY = {
    "No Attack": "none", "JPEG Q=90": "compression", "JPEG Q=50": "compression", "JPEG Q=20":
"compression",
    "Gaussian Noise (var=0.005)": "noise", "Gaussian Noise (var=0.02)": "noise", "Salt & Pepper
(2%)": "noise",
    "Median Filter 3x3": "filtering", "Gaussian Blur (sigma=1.5)": "filtering", "Sharpening":
"filtering",
    "Rotation 5 deg": "geometric", "Rotation 15 deg": "geometric", "Cropping 25%": "geometric",
    "Scaling 0.5x": "geometric", "Histogram Equalization": "global intensity",
}


def make_attack_suite(seed: int = 0) -> dict:
    return {
        "No Attack":                  lambda x: x.copy(),
        "JPEG Q=90":                  lambda x: jpeg_compression(x, 90),
        "JPEG Q=50":                  lambda x: jpeg_compression(x, 50),
        "JPEG Q=20":                  lambda x: jpeg_compression(x, 20),
        "Gaussian Noise (var=0.005)": lambda x: gaussian_noise(x, 0.005, rng=seed),
        "Gaussian Noise (var=0.02)":  lambda x: gaussian_noise(x, 0.02, rng=seed + 1),
        "Salt & Pepper (2%)":         lambda x: salt_pepper_noise(x, 0.02, rng=seed + 2),
        "Median Filter 3x3":          lambda x: median_filter(x, 3),
        "Gaussian Blur (sigma=1.5)":  lambda x: gaussian_blur(x, 1.5),
        "Rotation 5 deg":             lambda x: rotation_attack(x, 5),
        "Rotation 15 deg":            lambda x: rotation_attack(x, 15),
        "Cropping 25%":               lambda x: cropping_attack(x, 0.25),
        "Scaling 0.5x":               lambda x: scaling_attack(x, 0.5),
        "Histogram Equalization":     lambda x: histogram_equalization(x),
        "Sharpening":                 lambda x: sharpening_attack(x, 1.0),
    }


ATTACK_SUITE = make_attack_suite(0)
assert list(ATTACK_SUITE) == ATTACK_NAMES
```

1.4 logo.py

Procedural 64x64 binary watermark logo. 17 lines.

```
import numpy as np
from PIL import Image, ImageDraw, ImageFont


def make_watermark_logo(size=64, text="CYBERION"):
    img = Image.new("L", (size * 4, size), color=0)
    draw = ImageDraw.Draw(img)
    try:
        font = ImageFont.load_default()
    except Exception:
        font = None
    draw.rectangle([0, 0, size * 4 - 1, size - 1], outline=255, width=3)
    draw.text((10, size // 2 - 6), text, fill=255, font=font)
    img = img.resize((size, size))
```

```
    arr = np.array(img).astype(np.float64)
    arr = (arr > 100).astype(np.float64) * 255.0
    return arr
```

1.5 image_io.py

Safe image loading and standardisation. 86 lines.

```
from __future__ import annotations

import os
from dataclasses import dataclass

import numpy as np
from PIL import Image, UnidentifiedImageError

Image.MAX_IMAGE_PIXELS = 200_000_000


class ImageLoadError(RuntimeError):
    pass


@dataclass
class SourceInfo:
    width: int
    height: int
    mode: str
    fmt: str
    file_size: int


def _to_uint8_rgbish(im: Image.Image) -> Image.Image:
    mode = im.mode
    if mode in ("RGBA", "LA", "PA") or (mode == "P" and "transparency" in im.info):
        im = im.convert("RGBA")
        bg = Image.new("RGBA", im.size, (255, 255, 255, 255))
        return Image.alpha_composite(bg, im).convert("RGB")
    if mode in ("I;16", "I;16B", "I;16L", "I", "F"):
        arr = np.asarray(im, dtype=np.float64)
        lo, hi = float(arr.min()), float(arr.max())
        if hi <= lo:
            raise ImageLoadError("constant-valued high-bit-depth image")
        return Image.fromarray(((arr - lo) / (hi - lo) * 255.0).round().astype(np.uint8), "L")
    if mode == "1":
        return im.convert("L")
    if mode in ("L", "RGB"):
        return im
    return im.convert("RGB")


def load_gray_square(path: str, size: int = 512) -> tuple[np.ndarray, SourceInfo]:
    if not os.path.isfile(path):
        raise ImageLoadError(f"file not found: {path}")
    fsize = os.path.getsize(path)
    if fsize == 0:
        raise ImageLoadError(f"empty file: {path}")
    try:
        with Image.open(path) as im:
            fmt = im.format or ""
            w, h = im.size
            mode = im.mode
            im.load()
            rgb = _to_uint8_rgbish(im)
    except (UnidentifiedImageError, OSError, ValueError, SyntaxError) as exc:
        raise ImageLoadError(f"cannot decode {os.path.basename(path)}: {exc}") from exc
    if min(w, h) < 16:
        raise ImageLoadError(f"image too small ({w}x{h})")

    gray = rgb.convert("L")
    side = min(gray.size)
    left, top = (gray.width - side) // 2, (gray.height - side) // 2
    gray = gray.crop((left, top, left + side, top + side)).resize((size, size), Image.LANCZOS)
    arr = np.asarray(gray, dtype=np.float64)
    return arr, SourceInfo(w, h, mode, fmt, fsize)


def save_gray_png(arr: np.ndarray, path: str) -> None:
```

```
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    Image.fromarray(np.clip(np.rint(arr), 0, 255).astype(np.uint8), "L").save(path)


def verify_png(path: str, expected_shape: tuple[int, int] | None = None) -> bool:
    try:
        with Image.open(path) as im:
            im.load()
            a = np.asarray(im)
    except Exception:
        return False
    if a.ndim != 2 or a.dtype != np.uint8:
        return False
    if expected_shape is not None and a.shape != tuple(expected_shape):
        return False
    return bool(np.isfinite(a).all())
```

Part 2: Dataset

Everything needed to rebuild the 100 image evaluation set.

2.1 dataset_manifest.py

List of source images, licences and categories. 118 lines.

```
OCV = "opencv/opencv/4.x/samples/data/"
OCVX = "opencv/opencv_extra/4.x/testdata/"
OXF = OCVX + "cv/detectors_descriptors_evaluation/images_datasets/"
SKI = "scikit-image/scikit-image/v0.26.0/src/skimage/data/"
THR = "mrdoob/three.js/dev/examples/textures/"
PIL = "python-pillow/Pillow/main/Tests/images/"

_APACHE = "Apache-2.0"
_NOT_DOC = "not individually documented upstream"

REAL_IMAGES = [
    ("ocv_apple",        OCV + "apple.jpg",        "OpenCV samples", _APACHE, _NOT_DOC, "object"),
    ("ocv_baboon",       OCV + "baboon.jpg",       "OpenCV samples", _APACHE, "classic USC-SIPI
'mandrill' test image; " + _NOT_DOC, "animal"),
    ("ocv_board",        OCV + "board.jpg",        "OpenCV samples", _APACHE, _NOT_DOC,
"object_fine_detail"),
    ("ocv_building",     OCV + "building.jpg",     "OpenCV samples", _APACHE, _NOT_DOC,
"urban_architecture"),
    ("ocv_butterfly",    OCV + "butterfly.jpg",    "OpenCV samples", _APACHE, _NOT_DOC, "animal"),
    ("ocv_chicky",       OCV + "chicky_512.png",   "OpenCV samples", _APACHE, _NOT_DOC, "animal"),
    ("ocv_fruits",       OCV + "fruits.jpg",       "OpenCV samples", _APACHE, _NOT_DOC, "object"),
    ("ocv_graf1",        OCV + "graf1.png",        "OpenCV samples", _APACHE, "Oxford affine-regions
'graf' image; research use", "urban_architecture"),
    ("ocv_home",         OCV + "home.jpg",         "OpenCV samples", _APACHE, _NOT_DOC,
"urban_architecture"),
    ("ocv_left01",       OCV + "left01.jpg",       "OpenCV samples", _APACHE, "calibration-pattern
scene (one of 12 near-identical frames; only one kept)", "indoor_person"),
    ("ocv_messi",        OCV + "messi5.jpg",       "OpenCV samples", _APACHE, _NOT_DOC,
"person_sports"),
    ("ocv_orange",       OCV + "orange.jpg",       "OpenCV samples", _APACHE, _NOT_DOC, "object"),
    ("ocv_smarties",     OCV + "smarties.png",     "OpenCV samples", _APACHE, _NOT_DOC, "object"),
    ("ocv_starry_night", OCV + "starry_night.jpg", "OpenCV samples", _APACHE, "reproduction of a
public-domain painting (van Gogh, 1889)", "painting_art"),
    ("ocv_stuff",        OCV + "stuff.jpg",        "OpenCV samples", _APACHE, _NOT_DOC, "object"),
    ("ocv_rubberwhale",  OCV + "rubberwhale1.png", "OpenCV samples", _APACHE, "Middlebury optical-
flow dataset frame; research use", "object"),
    ("ocv_text_motion",  OCV + "text_motion.jpg",  "OpenCV samples", _APACHE, _NOT_DOC,
"document_text"),
    ("ocv_imagetext",    OCV + "imageTextN.png",   "OpenCV samples", _APACHE, _NOT_DOC,
"document_text"),
    ("ocv_basketball",   OCV + "basketball1.png",  "OpenCV samples", _APACHE, "video frame; " +
_NOT_DOC, "indoor_person"),
    ("ocv_blox",         OCV + "blox.jpg",         "OpenCV samples", _APACHE, _NOT_DOC, "object"),
    ("ocv_books",        OCV + "left.jpg",         "OpenCV samples", _APACHE, "left view of a stereo
pair (right view omitted); " + _NOT_DOC, "indoor_object"),
    ("ocv_aloe",         OCV + "aloeL.jpg",        "OpenCV samples", _APACHE, "Middlebury stereo
dataset (left view only); research use", "plant_object"),
    ("ocvx_airplane",    OCVX + "cv/shared/airplane.png",       "OpenCV extra", _APACHE, "classic
USC-SIPI 'airplane (F-16)' test image; " + _NOT_DOC, "aerial_vehicle"),
    ("ocvx_monalisa",    OCVX + "cv/cascadeandhog/images/mona-lisa.png", "OpenCV extra", _APACHE,
"reproduction of a public-domain painting", "painting_art"),
    ("ocvx_shuttle",     OCVX + "dnn/space_shuttle.jpg",        "OpenCV extra", _APACHE, _NOT_DOC,
"aerial_vehicle"),
    ("ocvx_street",      OCVX + "dnn/street.png",               "OpenCV extra", _APACHE, _NOT_DOC,
"urban_architecture"),
    ("ocvx_cones",       OCVX + "cv/stereomatching/datasets/cones/im2.png", "OpenCV extra", _APACHE,
"Middlebury stereo 'cones' (view 2); research use", "indoor_object"),
    ("ocvx_david",       OCVX + "cv/face/david1.jpg",           "OpenCV extra", _APACHE, "face video
frame; " + _NOT_DOC, "person_portrait"),
    ("oxf_bark",         OXF + "bark/img1.png",   "Oxford affine-regions (via OpenCV extra)",
_APACHE, "K. Mikolajczyk et al., Oxford VGG; research use", "texture_natural"),
    ("oxf_bikes",        OXF + "bikes/img1.png",  "Oxford affine-regions (via OpenCV extra)",
_APACHE, "K. Mikolajczyk et al., Oxford VGG; research use", "urban_architecture"),
    ("oxf_boat",         OXF + "boat/img1.png",   "Oxford affine-regions (via OpenCV extra)",
_APACHE, "K. Mikolajczyk et al., Oxford VGG; research use", "urban_architecture"),
    ("oxf_leuven",       OXF + "leuven/img1.png", "Oxford affine-regions (via OpenCV extra)",
_APACHE, "K. Mikolajczyk et al., Oxford VGG; research use", "urban_architecture"),
```

```
    ("oxf_trees",        OXF + "trees/img1.png",  "Oxford affine-regions (via OpenCV extra)",
_APACHE, "K. Mikolajczyk et al., Oxford VGG; research use", "natural_scene"),
    ("oxf_ubc",          OXF + "ubc/img1.png",    "Oxford affine-regions (via OpenCV extra)",
_APACHE, "K. Mikolajczyk et al., Oxford VGG; research use", "urban_architecture"),
    ("oxf_wall",         OXF + "wall/img1.png",   "Oxford affine-regions (via OpenCV extra)",
_APACHE, "K. Mikolajczyk et al., Oxford VGG; research use", "texture_natural"),
    ("dkn_dog",          "pjreddie/darknet/master/data/dog.jpg",     "darknet", "YOLO licence (public
domain)", _NOT_DOC, "animal"),
    ("dkn_eagle",        "pjreddie/darknet/master/data/eagle.jpg",   "darknet", "YOLO licence (public
domain)", _NOT_DOC, "animal"),
    ("dkn_giraffe",      "pjreddie/darknet/master/data/giraffe.jpg", "darknet", "YOLO licence (public
domain)", _NOT_DOC, "animal"),
    ("dkn_horses",       "pjreddie/darknet/master/data/horses.jpg",  "darknet", "YOLO licence (public
domain)", _NOT_DOC, "animal"),
    ("dkn_kite",         "pjreddie/darknet/master/data/kite.jpg",    "darknet", "YOLO licence (public
domain)", _NOT_DOC, "natural_scene"),
    ("dkn_person",       "pjreddie/darknet/master/data/person.jpg",  "darknet", "YOLO licence (public
domain)", _NOT_DOC, "natural_scene"),
    ("dkn_scream",       "pjreddie/darknet/master/data/scream.jpg",  "darknet", "YOLO licence (public
domain)", "reproduction of a painting (Munch, 1893)", "painting_art"),
    ("tv_dog1",          "pytorch/vision/main/gallery/assets/dog1.jpg", "torchvision gallery", "BSD-
3-Clause", _NOT_DOC, "animal"),
    ("tv_dog2",          "pytorch/vision/main/gallery/assets/dog2.jpg", "torchvision gallery", "BSD-
3-Clause", _NOT_DOC, "animal"),
    ("tfm_dogs",         "tensorflow/models/master/research/object_detection/test_images/image1.jpg",
"TensorFlow models", _APACHE, _NOT_DOC, "animal"),
    ("tfm_beach",        "tensorflow/models/master/research/object_detection/test_images/image2.jpg",
"TensorFlow models", _APACHE, _NOT_DOC, "natural_scene"),
    ("sam_dog",          "facebookresearch/segment-anything/main/notebooks/images/dog.jpg",
"segment-anything", _APACHE, _NOT_DOC, "indoor_person"),
    ("sam_groceries",    "facebookresearch/segment-anything/main/notebooks/images/groceries.jpg",
"segment-anything", _APACHE, _NOT_DOC, "object"),
    ("sam_truck",        "facebookresearch/segment-anything/main/notebooks/images/truck.jpg",
"segment-anything", _APACHE, _NOT_DOC, "urban_architecture"),
    ("mmd_demo",         "open-mmlab/mmdetection/main/demo/demo.jpg", "mmdetection", _APACHE,
_NOT_DOC, "natural_scene"),
    ("hf_cats",
"huggingface/transformers/main/tests/fixtures/tests_samples/COCO/000000039769.png", "transformers
test fixtures", _APACHE, "COCO val image (Flickr, CC BY 2.0 per COCO terms; attribution per COCO)",
"animal"),
    ("thr_park",         THR + "cube/Park2/posx.jpg",                "three.js examples", "MIT",
"cube-map face; texture rights " + _NOT_DOC, "natural_scene"),
    ("thr_bridge",       THR + "cube/Bridge2/posx.jpg",              "three.js examples", "MIT",
"cube-map face; texture rights " + _NOT_DOC, "natural_scene"),
    ("thr_castle",       THR + "cube/SwedishRoyalCastle/px.jpg",     "three.js examples", "MIT",
"cube-map face (night scene); texture rights " + _NOT_DOC, "urban_architecture"),
    ("thr_pisa",         THR + "cube/pisa/px.png",                   "three.js examples", "MIT",
"cube-map face; texture rights " + _NOT_DOC, "urban_architecture"),
    ("thr_milkyway",     THR + "cube/MilkyWay/dark-s_px.jpg",        "three.js examples", "MIT",
"cube-map face (near-black sky); texture rights " + _NOT_DOC, "astronomy_space"),
    ("thr_caravaggio",   THR + "758px-Canestra_di_frutta_%28Caravaggio%29.jpg", "three.js examples",
"MIT", "reproduction of a public-domain painting (Caravaggio)", "painting_art"),
    ("thr_interior",     THR + "2294472375_24a3b8ef46_o.jpg",        "three.js examples", "MIT",
"equirectangular interior panorama (Flickr origin); " + _NOT_DOC, "indoor_object"),
    ("thr_earth",        THR + "planets/earth_atmos_2048.jpg",       "three.js examples", "MIT",
"NASA-derived Earth map; " + _NOT_DOC, "scientific_map"),
    ("thr_moonmap",      THR + "planets/moon_1024.jpg",              "three.js examples", "MIT",
"lunar surface map; " + _NOT_DOC, "scientific_map"),
    ("thr_hardwood",     THR + "hardwood2_diffuse.jpg",              "three.js examples", "MIT",
"wood-floor texture; " + _NOT_DOC, "texture_natural"),
    ("thr_brick",        THR + "brick_diffuse.jpg",                  "three.js examples", "MIT",
"brick-wall texture; " + _NOT_DOC, "texture_natural"),
    ("thr_grass",        THR + "terrain/grasslight-big.jpg",         "three.js examples", "MIT",
"grass texture; " + _NOT_DOC, "texture_natural"),
    ("thr_lava",         THR + "lava/lavatile.jpg",                  "three.js examples", "MIT",
"lava texture; " + _NOT_DOC, "texture_natural"),
    ("ski_astronaut",    SKI + "astronaut.png", "scikit-image data", "BSD-3-Clause", "NASA photograph
of Eileen Collins (public domain)", "person_portrait"),
    ("ski_brick",        SKI + "brick.png",     "scikit-image data", "BSD-3-Clause", "CC0
(CC0Textures)", "texture_natural"),
    ("ski_camera",       SKI + "camera.png",    "scikit-image data", "BSD-3-Clause", "CC0 by the
photographer (Lav Varshney)", "person_portrait"),
    ("ski_cell",         SKI + "cell.png",      "scikit-image data", "BSD-3-Clause", "quantitative
phase image of a cell (qpformat)", "scientific_medical"),
    ("ski_chelsea",      SKI + "chelsea.png",   "scikit-image data", "BSD-3-Clause", "CC0 by the
photographer (Stefan van der Walt)", "animal"),
    ("ski_clock",        SKI + "clock_motion.png", "scikit-image data", "BSD-3-Clause", "public
domain (Stefan van der Walt); motion-blurred", "object"),
```

```
    ("ski_coffee",       SKI + "coffee.png",    "scikit-image data", "BSD-3-Clause", "CC0 by the
photographer (Rachel Michetti)", "object"),
    ("ski_coins",        SKI + "coins.png",     "scikit-image data", "BSD-3-Clause", "Brooklyn Museum
Collection (open collection)", "object"),
    ("ski_grass",        SKI + "grass.png",     "scikit-image data", "BSD-3-Clause", "CC0
(DeviantArt, linolafett)", "texture_natural"),
    ("ski_gravel",       SKI + "gravel.png",    "scikit-image data", "BSD-3-Clause", "CC0
(CC0Textures)", "texture_natural"),
    ("ski_hubble",       SKI + "hubble_deep_field.jpg", "scikit-image data", "BSD-3-Clause",
"HubbleSite / NASA image", "astronomy_space"),
    ("ski_ihc",          SKI + "ihc.png",       "scikit-image data", "BSD-3-Clause",
"immunohistochemistry micrograph (Center for Microscopy and Image Analysis, Zurich)",
"scientific_medical"),
    ("ski_moon",         SKI + "moon.png",      "scikit-image data", "BSD-3-Clause", "low-contrast
lunar surface image", "astronomy_space"),
    ("ski_motorcycle",   SKI + "motorcycle_left.png", "scikit-image data", "BSD-3-Clause",
"Middlebury stereo dataset (left view only); research use", "indoor_object"),
    ("ski_retina",       SKI + "retina.jpg",    "scikit-image data", "BSD-3-Clause", "CC0 (Wikimedia
Commons fundus photograph)", "scientific_medical"),
    ("ski_rocket",       SKI + "rocket.jpg",    "scikit-image data", "BSD-3-Clause", "SpaceX Photos
(Flickr); see upstream licence", "aerial_vehicle"),
    ("skl_china",        "scikit-learn/scikit-learn/main/sklearn/datasets/images/china.jpg",
"scikit-learn", "BSD-3-Clause", "CC BY 2.0, danielbuechele (Flickr)", "urban_architecture"),
    ("skl_flower",       "scikit-learn/scikit-learn/main/sklearn/datasets/images/flower.jpg",
"scikit-learn", "BSD-3-Clause", "CC BY 2.0, vultilion (Flickr)", "plant_object"),
    ("mpl_hopper",
"matplotlib/matplotlib/main/lib/matplotlib/mpl-data/sample_data/grace_hopper.jpg", "matplotlib",
"PSF-based (matplotlib licence)", "official US Navy photograph (public domain)", "person_portrait"),
    ("scp_face",         "scipy:face",   "scipy.datasets", "BSD-3-Clause", "raccoon image derived
from pixnio.com", "animal"),
    ("scp_ascent",       "scipy:ascent", "scipy.datasets", "BSD-3-Clause", "derived from pixnio.com
('accent-to-the-top')", "person_sports"),
    ("iio_bricks",       "imageio/imageio-binaries/master/images/bricks.jpg", "imageio-binaries",
"BSD-2-Clause", _NOT_DOC, "texture_natural"),
    ("iio_wikkie",       "imageio/imageio-binaries/master/images/wikkie.png", "imageio-binaries",
"BSD-2-Clause", _NOT_DOC, "animal"),
    ("pil_plumeria",     PIL + "flower.jpg",      "Pillow tests", "HPND", _NOT_DOC, "plant_object"),
    ("pil_heather",      PIL + "flower2.jpg",     "Pillow tests", "HPND", _NOT_DOC, "plant_object"),
    ("pil_field",        PIL + "junk_jpeg_header.jpg", "Pillow tests", "HPND", _NOT_DOC,
"natural_scene"),
    ("pil_sign",         PIL + "iptc.jpg",        "Pillow tests", "HPND", _NOT_DOC,
"urban_architecture"),
    ("pil_cabin",        PIL + "sugarshack_bad_mpo_header.jpg", "Pillow tests", "HPND", _NOT_DOC,
"natural_scene"),
    ("pil_coins",        PIL + "xmp_test.jpg",    "Pillow tests", "HPND", _NOT_DOC, "object"),
    ("pil_aircraft",     PIL + "exif_gps_typeerror.jpg", "Pillow tests", "HPND", _NOT_DOC,
"aerial_vehicle"),
    ("pil_beach",        PIL + "icc_profile_big.jpg", "Pillow tests", "HPND", _NOT_DOC,
"natural_scene"),
    ("pil_mountain",     PIL + "exif-72dpi-int.jpg",  "Pillow tests", "HPND", _NOT_DOC,
"natural_scene"),
]

EXCLUDED = {
    "lena.jpg": "contested provenance/consent; many institutions advise against its use",
    "left02..left14 (OpenCV calibration frames)": "near-identical frames of one calibration scene",
    "aloeR / right.jpg / rubberwhale2 / motorcycle_right": "second view of an already-included stereo
pair or frame",
    "meadow_cube.jpg": "2048x12288 strip, not a natural image of a scene",
    "images < 240 px on the short side": "up-sampling to 512 px would make them artificially smooth",
    "Pillow format-edge-case images (hopper_* variants, EXIF/ICC tests)": "same picture in many
encodings; not independent content",
    "ultralytics bus.jpg / zidane.jpg": "AGPL-3.0 repository; photographs of identifiable persons
with unclear rights",
}
```

2.2 build_dataset.py

Downloads, standardises and records the 100 images. 209 lines.

```
from __future__ import annotations

import argparse
import csv
import hashlib
import json
import logging
import os
```

```
import sys
import time
import urllib.parse
import urllib.request

import numpy as np
from PIL import Image
from scipy import ndimage

from dataset_manifest import REAL_IMAGES
from image_io import ImageLoadError, load_gray_square, save_gray_png

LOG = logging.getLogger("build_dataset")
RAW = "https://raw.githubusercontent.com/"
SIZE = 512


def sha256_file(path: str) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()


def dhash64(arr: np.ndarray) -> int:
    im = Image.fromarray(np.clip(arr, 0, 255).astype(np.uint8)).resize((9, 8), Image.LANCZOS)
    a = np.asarray(im, dtype=np.int16)
    bits = (a[:, 1:] > a[:, :-1]).flatten()
    return int("".join("1" if b else "0" for b in bits), 2)


def hamming(a: int, b: int) -> int:
    return bin(a ^ b).count("1")


def download(rel: str, dest: str, retries: int = 3) -> bool:
    url = RAW + rel
    last = None
    for k in range(retries):
        try:
            with urllib.request.urlopen(urllib.request.Request(url, headers={"User-Agent": "ivp-
watermark-build/1.0"}), timeout=60) as r:
                data = r.read()
            if not data:
                raise IOError("empty response")
            with open(dest, "wb") as f:
                f.write(data)
            return True
        except Exception as exc:
            last = exc
            time.sleep(1.5 * (k + 1))
    LOG.warning("download failed %s (%s)", url, last)
    return False


def fetch_scipy(name: str, dest: str) -> bool:
    try:
        import scipy.datasets as sd
        arr = getattr(sd, name)()
        Image.fromarray(arr).save(dest)
        return True
    except Exception as exc:
        LOG.warning("scipy.datasets.%s unavailable: %s", name, exc)
        return False


def _smooth_noise(rng, sigma, mean, std):
    n = ndimage.gaussian_filter(rng.standard_normal((SIZE, SIZE)), sigma)
    n = (n - n.mean()) / n.std()
    return np.clip(mean + std * n, 0, 255)


def synth_images():
    rng = np.random.default_rng(20260713)
    yy, xx = np.mgrid[0:SIZE, 0:SIZE].astype(np.float64)
    out = []
```

```
    out.append(("syn_hgradient", "horizontal linear gradient 0-255", np.tile(np.linspace(0, 255,
SIZE), (SIZE, 1))))
    r = np.hypot(xx - SIZE / 2, yy - SIZE / 2)
    out.append(("syn_radial", "radial gradient (smooth, low detail)", np.clip(255 - r * 0.7, 0,
255)))
    out.append(("syn_grating", "2-D sinusoidal grating (periodic texture)", 128 + 100 * np.sin(2 *
np.pi * (xx * 9 + yy * 5) / SIZE)))
    out.append(("syn_white_noise", "white Gaussian noise, mean 128 / std 40", np.clip(128 + 40 *
rng.standard_normal((SIZE, SIZE)), 0, 255)))
    out.append(("syn_lowcontrast", "smooth noise, mean 128 / std 8 (very low contrast)",
_smooth_noise(rng, 6, 128, 8)))
    out.append(("syn_checker", "32-px checkerboard (hard edges)", 255.0 * (((xx // 32) + (yy // 32))
% 2)))
    out.append(("syn_dark", "smooth noise, mean 25 / std 12 (dark)", _smooth_noise(rng, 5, 25, 12)))
    out.append(("syn_bright", "smooth noise, mean 230 / std 12 (bright)", _smooth_noise(rng, 5, 230,
12)))
    f = np.fft.fftfreq(SIZE)
    fx, fy = np.meshgrid(f, f)
    amp = 1.0 / np.maximum(np.hypot(fx, fy), 1.0 / SIZE)
    spec = amp * np.exp(2j * np.pi * rng.random((SIZE, SIZE)))
    frac = np.real(np.fft.ifft2(spec))
    frac = (frac - frac.mean()) / frac.std()
    out.append(("syn_fractal", "1/f fractal noise texture", np.clip(128 + 45 * frac, 0, 255)))
    scene = np.full((SIZE, SIZE), 110.0)
    for _ in range(14):
        x0, y0 = rng.integers(0, SIZE - 80, 2)
        w, h = rng.integers(40, 160, 2)
        scene[y0:y0 + h, x0:x0 + w] = rng.integers(20, 240)
    out.append(("syn_blocks", "random grey rectangles on grey background", scene))
    return out


def main():
    ap = argparse.ArgumentParser()
    here = os.path.dirname(os.path.abspath(__file__))
    ap.add_argument("--out", default=os.path.join(here, "..", "..", "data"))
    ap.add_argument("--target", type=int, default=100)
    ap.add_argument("--dup-threshold", type=int, default=6)
    args = ap.parse_args()
    out = os.path.abspath(args.out)
    src_dir = os.path.join(out, "source_images")
    orig_dir = os.path.join(out, "original")
    os.makedirs(src_dir, exist_ok=True)
    os.makedirs(orig_dir, exist_ok=True)
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")

    rows, hashes, skipped = [], [], []
    for key, rel, group, lic, note, category in REAL_IMAGES:
        ext = os.path.splitext(urllib.parse.unquote(rel))[1] or ".png"
        if rel.startswith("scipy:"):
            dest = os.path.join(src_dir, f"{key}.png")
            ok = os.path.isfile(dest) or fetch_scipy(rel.split(":", 1)[1], dest)
            url = f"scipy.datasets.{rel.split(':', 1)[1]}()"
        else:
            dest = os.path.join(src_dir, f"{key}{ext}")
            ok = os.path.isfile(dest) and os.path.getsize(dest) > 0 or download(rel, dest)
            url = RAW + rel
        if not ok:
            skipped.append((key, "download failed"))
            continue
        try:
            arr, info = load_gray_square(dest, SIZE)
        except ImageLoadError as exc:
            skipped.append((key, str(exc)))
            LOG.warning("skip %s: %s", key, exc)
            continue
        if min(info.width, info.height) < 240:
            skipped.append((key, f"too small ({info.width}x{info.height})"))
            continue
        hsh = dhash64(arr)
        dup = next((k for k, h in hashes if hamming(h, hsh) <= args.dup_threshold), None)
        if dup:
            skipped.append((key, f"near-duplicate of {dup}"))
            LOG.info("duplicate %s ~ %s", key, dup)
            continue
        hashes.append((key, hsh))
        rows.append(dict(key=key, arr=arr, url=url, group=group, lic=lic, note=note,
                         category=category, info=info, sha=sha256_file(dest),
```

```
                        fname=os.path.basename(dest), synthetic=False))

    need = args.target - len(rows)
    if need < 0:
        rows = rows[:args.target]
        need = 0
    synth = synth_images()
    if need > len(synth):
        LOG.warning("only %d synthetic generators available; dataset will have %d images",
                    len(synth), len(rows) + len(synth))
    for key, desc, arr in synth[:need]:
        rows.append(dict(key=key, arr=arr, url="generated by build_dataset.py (seeded)",
group="synthetic control",
                        lic="n/a (generated)", note=desc, category="synthetic_control",
                        info=None, sha="", fname="", synthetic=True))

    meta_path = os.path.join(out, "metadata.csv")
    cols = ["image_id", "key", "original_filename", "source_group", "source_url",
"repository_license",
            "rights_note", "original_width", "original_height", "original_mode", "original_format",
            "original_file_size_bytes", "source_sha256", "standardized_shape",
"standardized_channels",
            "category", "is_synthetic", "mean_intensity", "contrast_std", "brightness_class",
            "contrast_class", "applicable_operations"]
    with open(meta_path, "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=cols)
        w.writeheader()
        for i, r in enumerate(rows, 1):
            iid = f"image_{i:03d}"
            save_gray_png(r["arr"], os.path.join(orig_dir, iid + ".png"))
            a = np.rint(r["arr"])
            mean, std = float(a.mean()), float(a.std())
            info = r["info"]
            w.writerow(dict(
                image_id=iid, key=r["key"], original_filename=r["fname"], source_group=r["group"],
                source_url=r["url"], repository_license=r["lic"], rights_note=r["note"],
                original_width=info.width if info else SIZE, original_height=info.height if info else
SIZE,
                original_mode=info.mode if info else "generated", original_format=info.fmt if info
else "PNG",
                original_file_size_bytes=info.file_size if info else "", source_sha256=r["sha"],
                standardized_shape=f"{SIZE}x{SIZE}", standardized_channels=1, category=r["category"],
                is_synthetic=int(r["synthetic"]), mean_intensity=round(mean, 2),
contrast_std=round(std, 2),
                brightness_class="dark" if mean < 80 else ("bright" if mean > 170 else "mid"),
                contrast_class="low" if std < 35 else ("high" if std > 65 else "medium"),
                applicable_operations="embed;extract;14 attacks + reference;PSNR;SSIM;NC;BER;alpha-
sweep;controls"))

    with open(os.path.join(out, "build_log.json"), "w") as f:
        json.dump({"images_written": len(rows), "real": sum(not r["synthetic"] for r in rows),
                   "synthetic": sum(r["synthetic"] for r in rows), "skipped": skipped}, f, indent=2)
    LOG.info("wrote %d images (%d real, %d synthetic); skipped %d",
             len(rows), sum(not r["synthetic"] for r in rows), sum(r["synthetic"] for r in rows),
len(skipped))
    for k, why in skipped:
        LOG.info("  skipped %-16s %s", k, why)
    return 0 if len(rows) == args.target else 1


if __name__ == "__main__":
    sys.exit(main())
```

Part 3: Experiments

Scripts that run the experiments and record results.

3.1 run_experiment.py

Original two image experiment. 152 lines.

```
import os
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from PIL import Image
from skimage import data as skdata
from skimage.transform import resize

from watermark_core import embed_watermark, extract_watermark
from metrics import psnr, ssim, normalized_correlation, bit_error_rate
from attacks import ATTACK_SUITE
from logo import make_watermark_logo

RESULTS_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "..", "results",
"legacy_two_image")
os.makedirs(RESULTS_DIR, exist_ok=True)


def load_host_images():
    hosts = {
        "cameraman": resize(skdata.camera().astype(np.float64), (512, 512), preserve_range=True),
        "astronaut_gray": resize(skdata.astronaut().mean(axis=2).astype(np.float64), (512, 512),
preserve_range=True),
    }
    return hosts


def save_image(arr, path):
    Image.fromarray(np.clip(arr, 0, 255).astype(np.uint8)).save(path)


def main():
    alpha = 0.10
    hosts = load_host_images()
    watermark = make_watermark_logo(size=64, text="CYBERION")
    save_image(watermark, os.path.join(RESULTS_DIR, "watermark_logo.png"))

    all_rows = []
    per_host_extracted = {}

    for host_name, host in hosts.items():
        watermarked, key = embed_watermark(host, watermark, alpha=alpha)
        save_image(host, os.path.join(RESULTS_DIR, f"{host_name}_original.png"))
        save_image(watermarked, os.path.join(RESULTS_DIR, f"{host_name}_watermarked.png"))

        imperceptibility_psnr = psnr(host, watermarked)
        imperceptibility_ssim = ssim(host, watermarked)

        extracted_examples = {}
        for attack_name, attack_fn in ATTACK_SUITE.items():
            attacked = attack_fn(watermarked)
            extracted = extract_watermark(attacked, key)
            nc = normalized_correlation(watermark, extracted)
            ber = bit_error_rate(watermark, extracted)

            all_rows.append({
                "Host Image": host_name,
                "Attack": attack_name,
                "Watermarked PSNR (dB)": round(imperceptibility_psnr, 3),
                "Watermarked SSIM": round(imperceptibility_ssim, 4),
                "Extracted NC": round(nc, 4),
                "Extracted BER": round(ber, 4),
            })
            extracted_examples[attack_name] = extracted
```

```
        per_host_extracted[host_name] = extracted_examples

    df = pd.DataFrame(all_rows)
    df.to_csv(os.path.join(RESULTS_DIR, "results_table.csv"), index=False)
    print(df.to_string(index=False))

    for host_name, host in hosts.items():
        watermarked = np.array(Image.open(os.path.join(RESULTS_DIR,
f"{host_name}_watermarked.png"))).astype(np.float64)
        diff = np.abs(host - watermarked)
        fig, axes = plt.subplots(1, 3, figsize=(12, 4.2))
        axes[0].imshow(host, cmap="gray"); axes[0].set_title("Original Host"); axes[0].axis("off")
        axes[1].imshow(watermarked, cmap="gray"); axes[1].set_title("Watermarked");
axes[1].axis("off")
        im = axes[2].imshow(diff, cmap="hot"); axes[2].set_title("Absolute Difference (x1)");
axes[2].axis("off")
        fig.colorbar(im, ax=axes[2], fraction=0.046)
        plt.tight_layout()
        plt.savefig(os.path.join(RESULTS_DIR, f"{host_name}_comparison.png"), dpi=150)
        plt.close(fig)

    gallery_attacks = ["No Attack", "JPEG Q=50", "JPEG Q=20", "Gaussian Noise (var=0.02)",
                       "Median Filter 3x3", "Rotation 15 deg", "Cropping 25%", "Scaling 0.5x"]
    fig, axes = plt.subplots(2, 4, figsize=(14, 7))
    for ax, aname in zip(axes.flat, gallery_attacks):
        ex = per_host_extracted["cameraman"][aname]
        ax.imshow(ex, cmap="gray")
        row = df[(df["Host Image"] == "cameraman") & (df["Attack"] == aname)].iloc[0]
        ax.set_title(f"{aname}\nNC={row['Extracted NC']:.3f}", fontsize=9)
        ax.axis("off")
    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "extraction_gallery.png"), dpi=150)
    plt.close(fig)

    sub = df[df["Host Image"] == "cameraman"].sort_values("Extracted NC")
    fig, ax = plt.subplots(figsize=(9, 6))
    colors = ["#c0392b" if v < 0.75 else "#27ae60" for v in sub["Extracted NC"]]
    ax.barh(sub["Attack"], sub["Extracted NC"], color=colors)
    ax.axvline(0.75, color="black", linestyle="--", linewidth=1, label="Recognizability threshold
(NC=0.75)")
    ax.set_xlabel("Normalized Correlation (NC)")
    ax.set_title("Watermark Robustness Across Attack Types (host: cameraman)")
    ax.legend(loc="lower right")
    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "nc_bar_chart.png"), dpi=150)
    plt.close(fig)

    host = hosts["cameraman"]
    alphas = [0.02, 0.05, 0.08, 0.10, 0.15, 0.20, 0.30, 0.40]
    sweep_rows = []
    for a in alphas:
        wm_img, k = embed_watermark(host, watermark, alpha=a)
        p = psnr(host, wm_img)
        attacked = ATTACK_SUITE["JPEG Q=50"](wm_img)
        ext = extract_watermark(attacked, k)
        nc = normalized_correlation(watermark, ext)
        sweep_rows.append({"alpha": a, "PSNR": p, "NC_after_JPEG50": nc})
    sweep_df = pd.DataFrame(sweep_rows)
    sweep_df.to_csv(os.path.join(RESULTS_DIR, "alpha_sweep.csv"), index=False)

    fig, ax1 = plt.subplots(figsize=(8, 5.5))
    ax2 = ax1.twinx()
    ax1.plot(sweep_df["alpha"], sweep_df["PSNR"], "o-", color="#2980b9", label="PSNR
(imperceptibility)")
    ax2.plot(sweep_df["alpha"], sweep_df["NC_after_JPEG50"], "s-", color="#e67e22", label="NC after
JPEG Q=50 (robustness)")
    ax1.set_xlabel("Embedding strength (alpha)")
    ax1.set_ylabel("PSNR (dB)", color="#2980b9")
    ax2.set_ylabel("NC after JPEG-50 attack", color="#e67e22")
    ax1.set_title("Imperceptibility vs. Robustness Trade-off")
    fig.legend(loc="upper center", bbox_to_anchor=(0.5, 0.02), ncol=2)
    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "alpha_tradeoff.png"), dpi=150, bbox_inches="tight")
    plt.close(fig)

    summary = {
        "alpha_used_main_experiment": alpha,
        "hosts": list(hosts.keys()),
```

```
        "num_attacks_tested": len(ATTACK_SUITE),
        "mean_watermarked_psnr_db": round(df["Watermarked PSNR (dB)"].mean(), 3),
        "mean_watermarked_ssim": round(df["Watermarked SSIM"].mean(), 4),
        "mean_nc_all_attacks": round(df["Extracted NC"].mean(), 4),
        "attacks_with_NC_above_0.75": int((df["Extracted NC"] > 0.75).sum()),
        "attacks_total_rows": len(df),
    }
    with open(os.path.join(RESULTS_DIR, "summary.json"), "w") as f:
        json.dump(summary, f, indent=2)
    print("\nSUMMARY:", json.dumps(summary, indent=2))


if __name__ == "__main__":
    main()
```

3.2 run_batch_experiment.py

Batch experiment on 100 images. 258 lines.

```
from __future__ import annotations

import os

for _v in ("OPENBLAS_NUM_THREADS", "OMP_NUM_THREADS", "MKL_NUM_THREADS"):
    os.environ.setdefault(_v, "1")

import argparse
import concurrent.futures as cf
import json
import logging
import platform
import resource
import sys
import time

import numpy as np
import pandas as pd

from attacks import ATTACK_FAMILY, ATTACK_NAMES, make_attack_suite
from image_io import save_gray_png, verify_png
from logo import make_watermark_logo
from metrics import bit_error_rate, normalized_correlation, psnr, ssim, zero_normalized_correlation
from watermark_core import embed_watermark, extract_watermark

HERE = os.path.dirname(os.path.abspath(__file__))
LOG = logging.getLogger("batch")

DEFAULT_CONFIG = {
    "alpha": 0.10,
    "image_size": 512,
    "logo_size": 64,
    "logo_text": "CYBERION",
    "other_logos": ["OTHERLOGO", "OWNER-B"],
    "protocols": ["uint8", "float"],
    "primary_protocol": "uint8",
    "base_seed": 20260713,
    "alphas": [0.02, 0.05, 0.08, 0.10, 0.15, 0.20, 0.30, 0.40],
    "sweep_attacks": ["JPEG Q=50", "Gaussian Noise (var=0.02)", "Gaussian Blur (sigma=1.5)",
"Rotation 5 deg"],
    "n_controls_per_image": 5,
    "nc_threshold": 0.75,
    "data_dir": "../../data",
    "results_dir": "../../results",
    "outputs_dir": "../../outputs",
}


def _t():
    return time.perf_counter()


def _load_host(data_dir, image_id):
    from PIL import Image
    a = np.asarray(Image.open(os.path.join(data_dir, "original", image_id + ".png")),
dtype=np.float64)
    if a.ndim != 2:
        raise ValueError(f"{image_id}: expected greyscale, got shape {a.shape}")
    return a
```

```
def _quantize(a):
    return np.clip(np.rint(a), 0, 255)


def process_image(task):
    idx, image_id, cfg, others = task
    data_dir = cfg["data_dir"]
    rows_att, rows_ctl, rows_sw = [], [], []
    rec = {"image_id": image_id, "status": "ok", "error": ""}
    try:
        host = _load_host(data_dir, image_id)
        logo = make_watermark_logo(cfg["logo_size"], cfg["logo_text"])
        alpha = cfg["alpha"]

        t0 = _t()
        wm_float, key = embed_watermark(host, logo, alpha=alpha)
        rec["t_embed_ms"] = (_t() - t0) * 1e3
        wm_u8 = _quantize(wm_float)
        out_path = os.path.join(cfg["outputs_dir"], "watermarked", f"{image_id}_wm.png")
        save_gray_png(wm_u8, out_path)
        rec["output_valid"] = bool(verify_png(out_path, host.shape))
        rec["psnr_float_db"] = psnr(host, wm_float)
        rec["ssim_float"] = ssim(host, wm_float)
        rec["psnr_db"] = psnr(host, wm_u8)
        rec["ssim"] = ssim(host, wm_u8)
        rec["max_abs_diff"] = float(np.abs(host - wm_u8).max())
        rec["mean_abs_diff"] = float(np.abs(host - wm_u8).mean())
        rec["key_bytes"] = int(sum(v.nbytes for v in key.values() if isinstance(v, np.ndarray)))

        suite = make_attack_suite(cfg["base_seed"] + 100 * idx)
        for protocol in cfg["protocols"]:
            base = wm_u8 if protocol == "uint8" else wm_float
            for name in ATTACK_NAMES:
                row = {"image_id": image_id, "protocol": protocol, "attack": name,
                       "family": ATTACK_FAMILY[name], "status": "ok", "error": ""}
                try:
                    t0 = _t()
                    attacked = suite[name](base)
                    row["t_attack_ms"] = (_t() - t0) * 1e3
                    t0 = _t()
                    ext = extract_watermark(attacked, key)
                    row["t_extract_ms"] = (_t() - t0) * 1e3
                    row["nc"] = normalized_correlation(logo, ext)
                    row["zncc"] = zero_normalized_correlation(logo, ext)
                    row["ber"] = bit_error_rate(logo, ext)
                except Exception as exc:
                    row.update(status="error", error=f"{type(exc).__name__}: {exc}")
                rows_att.append(row)

        sweep_suite = make_attack_suite(cfg["base_seed"] + 100 * idx + 50)
        for a in cfg["alphas"]:
            try:
                w, k = embed_watermark(host, logo, alpha=a)
                wq = _quantize(w)
                p_a, s_a = psnr(host, wq), ssim(host, wq)
            except Exception as exc:
                rows_sw.append({"image_id": image_id, "alpha": a, "attack": "(embed)", "status":
"error",
                                "error": f"{type(exc).__name__}: {exc}"})
                continue
            for an in ["No Attack"] + list(cfg["sweep_attacks"]):
                row = {"image_id": image_id, "alpha": a, "attack": an, "psnr_db": p_a, "ssim": s_a,
                       "status": "ok", "error": ""}
                try:
                    ext = extract_watermark(sweep_suite[an](wq), k)
                    row["nc"] = normalized_correlation(logo, ext)
                    row["zncc"] = zero_normalized_correlation(logo, ext)
                    row["ber"] = bit_error_rate(logo, ext)
                except Exception as exc:
                    row.update(status="error", error=f"{type(exc).__name__}: {exc}")
                rows_sw.append(row)

        def ctl(kind, other, ncv, berv, zz, extra=""):
            rows_ctl.append({"image_id": image_id, "control": kind, "other": other,
                             "nc": ncv, "ber": berv, "zncc": zz, "note": extra})
```

```
        ext = extract_watermark(wm_u8, key)
        ctl("genuine (own key, no attack)", image_id, normalized_correlation(logo, ext),
bit_error_rate(logo, ext), zero_normalized_correlation(logo, ext))
        ext = extract_watermark(host, key)
        ctl("d: own host unwatermarked, own key", image_id, normalized_correlation(logo, ext),
bit_error_rate(logo, ext), zero_normalized_correlation(logo, ext),
            extra="degenerate: S_att == S_original so Sw' = 0")
        for oid in others:
            oh = _load_host(data_dir, oid)
            ext = extract_watermark(oh, key)
            ctl("a: other host unwatermarked, key_i", oid, normalized_correlation(logo, ext),
bit_error_rate(logo, ext), zero_normalized_correlation(logo, ext))
            ow, _ = embed_watermark(oh, logo, alpha=alpha)
            ext = extract_watermark(_quantize(ow), key)
            ctl("b: other host watermarked, key_i", oid, normalized_correlation(logo, ext),
bit_error_rate(logo, ext), zero_normalized_correlation(logo, ext))
        for text in cfg["other_logos"]:
            logo_b = make_watermark_logo(cfg["logo_size"], text)
            wb, kb = embed_watermark(host, logo_b, alpha=alpha)
            wbq = _quantize(wb)
            ext_a = extract_watermark(wbq, key)
            ext_b = extract_watermark(wbq, kb)
            ctl("c: same host, other logo B, decoded with key A (NC vs A)", text,
                normalized_correlation(logo, ext_a), bit_error_rate(logo, ext_a),
zero_normalized_correlation(logo, ext_a))
            ctl("c-ref: same host, logo B, decoded with key B (NC vs B)", text,
                normalized_correlation(logo_b, ext_b), bit_error_rate(logo_b, ext_b),
zero_normalized_correlation(logo_b, ext_b))
    except Exception as exc:
        rec.update(status="error", error=f"{type(exc).__name__}: {exc}")
        LOG.exception("image %s failed", image_id)
    rec["peak_rss_mb"] = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss / 1024.0
    return rec, rows_att, rows_sw, rows_ctl


def env_info():
    import PIL, pywt, scipy, skimage
    cpu = ""
    try:
        with open("/proc/cpuinfo") as f:
            cpu = next(l.split(":", 1)[1].strip() for l in f if l.startswith("model name"))
    except Exception:
        pass
    return {"python": sys.version.split()[0], "platform": platform.platform(), "cpu": cpu,
            "cpu_count": os.cpu_count(), "numpy": np.__version__, "scipy": scipy.__version__,
            "scikit-image": skimage.__version__, "PyWavelets": pywt.__version__, "Pillow":
PIL.__version__,
            "pandas": pd.__version__}


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--config", default=None)
    ap.add_argument("--workers", type=int, default=os.cpu_count() or 1)
    ap.add_argument("--limit", type=int, default=None, help="process only the first N images
(debug)")
    args = ap.parse_args()
    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")

    cfg = dict(DEFAULT_CONFIG)
    if args.config:
        with open(args.config) as f:
            cfg.update(json.load(f))
    for k in ("data_dir", "results_dir", "outputs_dir"):
        cfg[k] = os.path.abspath(os.path.join(HERE, cfg[k]))
    os.makedirs(cfg["results_dir"], exist_ok=True)
    os.makedirs(os.path.join(cfg["outputs_dir"], "watermarked"), exist_ok=True)

    meta = pd.read_csv(os.path.join(cfg["data_dir"], "metadata.csv"))
    if args.limit:
        meta = meta.head(args.limit)
    ids = list(meta["image_id"])
    n = len(ids)
    rng = np.random.default_rng(cfg["base_seed"])
    tasks = []
    for i, iid in enumerate(ids):
        pool = [x for x in ids if x != iid]
```

```
        others = list(rng.choice(pool, size=min(cfg["n_controls_per_image"], len(pool)),
replace=False))
        tasks.append((i, iid, cfg, others))

    LOG.info("processing %d images with %d worker(s)", n, args.workers)
    t_start = _t()
    results = []
    with cf.ProcessPoolExecutor(max_workers=args.workers) as ex:
        for res in ex.map(process_image, tasks):
            results.append(res)
            if len(results) % 10 == 0:
                LOG.info("  %d/%d images done", len(results), n)
    wall = _t() - t_start

    img_df = pd.DataFrame([r[0] for r in results]).sort_values("image_id")
    att_df = pd.DataFrame([row for r in results for row in r[1]])
    sw_df = pd.DataFrame([row for r in results for row in r[2]])
    ctl_df = pd.DataFrame([row for r in results for row in r[3]])
    rd = cfg["results_dir"]
    img_df.to_csv(os.path.join(rd, "results_images.csv"), index=False)
    att_df.to_csv(os.path.join(rd, "results_attacks.csv"), index=False)
    sw_df.to_csv(os.path.join(rd, "results_alpha_sweep.csv"), index=False)
    ctl_df.to_csv(os.path.join(rd, "results_controls.csv"), index=False)

    full = pd.read_csv(os.path.join(cfg["data_dir"], "metadata.csv"))
    st = img_df.set_index("image_id")["status"]
    att_ok = att_df.groupby("image_id")["status"].apply(lambda s: int((s == "ok").sum()))
    full["processing_status"] = full["image_id"].map(st).fillna(full.get("processing_status", "not
processed"))
    full["attack_trials_ok"] = full["image_id"].map(att_ok).fillna(full.get("attack_trials_ok"))
    full["attack_trials_expected"] = len(ATTACK_NAMES) * len(cfg["protocols"])
    full.to_csv(os.path.join(cfg["data_dir"], "metadata.csv"), index=False)

    summary = {
        "config": {k: v for k, v in cfg.items() if not k.endswith("_dir")},
        "environment": env_info(),
        "images_requested": n,
        "images_ok": int((img_df["status"] == "ok").sum()),
        "watermarked_files_valid": int(img_df["output_valid"].fillna(False).sum()),
        "attack_trials_total": int(len(att_df)),
        "attack_trials_ok": int((att_df["status"] == "ok").sum()),
        "attack_trials_failed": int((att_df["status"] != "ok").sum()),
        "alpha_sweep_rows_ok": int((sw_df["status"] == "ok").sum()),
        "alpha_sweep_rows_total": int(len(sw_df)),
        "control_rows": int(len(ctl_df)),
        "wall_time_s": round(wall, 1),
        "workers": args.workers,
        "peak_rss_mb_max_over_workers": round(float(img_df["peak_rss_mb"].max()), 1),
    }
    with open(os.path.join(rd, "run_summary.json"), "w") as f:
        json.dump(summary, f, indent=2)
    LOG.info("summary: %s", json.dumps({k: v for k, v in summary.items() if k not in ("config",
"environment")}))
    return 0 if summary["attack_trials_failed"] == 0 and summary["images_ok"] == n else 1


if __name__ == "__main__":
    sys.exit(main())
```

3.3 video_demo.py

Per frame video test. 149 lines.

```
from __future__ import annotations

import os
for _v in ("OPENBLAS_NUM_THREADS", "OMP_NUM_THREADS", "MKL_NUM_THREADS"):
    os.environ.setdefault(_v, "1")

import argparse
import json
import subprocess
import tempfile
import time
import urllib.request

import cv2
import numpy as np
```

```
import pandas as pd
from PIL import Image

from logo import make_watermark_logo
from metrics import bit_error_rate, normalized_correlation, psnr
from watermark_core import embed_watermark, extract_watermark

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
RES = os.path.join(ROOT, "results")
CACHE = os.path.join(ROOT, "data", "source_videos")
CLIPS = {
    "vtest.avi": "opencv/opencv/4.x/samples/data/vtest.avi",
    "Megamind.avi": "opencv/opencv/4.x/samples/data/Megamind.avi",
}
SIZE = 512
CODECS = {
    "lossless (FFV1)":  ["-c:v", "ffv1"],
    "H.264 CRF 18":     ["-c:v", "libx264", "-crf", "18", "-pix_fmt", "yuv420p"],
    "H.264 CRF 23":     ["-c:v", "libx264", "-crf", "23", "-pix_fmt", "yuv420p"],
    "H.264 CRF 28":     ["-c:v", "libx264", "-crf", "28", "-pix_fmt", "yuv420p"],
    "MJPEG q=5":        ["-c:v", "mjpeg", "-q:v", "5", "-pix_fmt", "yuvj420p"],
    "MJPEG q=15":       ["-c:v", "mjpeg", "-q:v", "15", "-pix_fmt", "yuvj420p"],
}


def fetch(name):
    os.makedirs(CACHE, exist_ok=True)
    p = os.path.join(CACHE, name)
    if not os.path.exists(p):
        urllib.request.urlretrieve("https://raw.githubusercontent.com/" + CLIPS[name], p)
    return p


def gray_square(frame_bgr):
    g = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2GRAY)
    s = min(g.shape)
    y0, x0 = (g.shape[0] - s) // 2, (g.shape[1] - s) // 2
    g = g[y0:y0 + s, x0:x0 + s]
    return np.asarray(Image.fromarray(g).resize((SIZE, SIZE), Image.LANCZOS), dtype=np.float64)


def read_frames(path, n):
    cap = cv2.VideoCapture(path)
    if not cap.isOpened():
        raise IOError(f"cannot open {path}")
    fps = cap.get(cv2.CAP_PROP_FPS)
    total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    w, h = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)), int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    frames = []
    try:
        while len(frames) < n:
            ok, f = cap.read()
            if not ok:
                break
            frames.append(gray_square(f))
    finally:
        cap.release()
    return frames, dict(fps=fps, total_frames=total, width=w, height=h)


def encode_decode(frames, codec_args, fps, tmp):
    out = os.path.join(tmp, "clip." + ("mkv" if "ffv1" in codec_args else ("avi" if "mjpeg" in
codec_args else "mp4")))
    raw = b"".join(np.clip(np.rint(f), 0, 255).astype(np.uint8).tobytes() for f in frames)
    enc = subprocess.run(["ffmpeg", "-v", "error", "-y", "-f", "rawvideo", "-pix_fmt", "gray", "-s",
f"{SIZE}x{SIZE}",
                          "-r", str(fps), "-i", "-", *codec_args, out], input=raw,
capture_output=True)
    if enc.returncode != 0:
        raise RuntimeError(enc.stderr.decode()[:300])
    dec = subprocess.run(["ffmpeg", "-v", "error", "-i", out, "-f", "rawvideo", "-pix_fmt", "gray",
"-"], capture_output=True)
    if dec.returncode != 0:
        raise RuntimeError(dec.stderr.decode()[:300])
    buf = np.frombuffer(dec.stdout, dtype=np.uint8)
    n = buf.size // (SIZE * SIZE)
```

```
    decoded = [buf[i * SIZE * SIZE:(i + 1) * SIZE * SIZE].reshape(SIZE, SIZE).astype(np.float64) for
i in range(n)]
    return decoded, os.path.getsize(out)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--frames", type=int, default=60)
    args = ap.parse_args()
    logo = make_watermark_logo()
    rows, meta = [], {}
    for clip in CLIPS:
        frames, info = read_frames(fetch(clip), args.frames)
        meta[clip] = dict(info, frames_read=len(frames))
        t0 = time.perf_counter()
        marked, keys = [], []
        for f in frames:
            w, k = embed_watermark(f, logo, alpha=0.10)
            k = {kk: vv for kk, vv in k.items() if kk not in ("U", "Vt")}
            marked.append(np.clip(np.rint(w), 0, 255)); keys.append(k)
        t_embed = (time.perf_counter() - t0) / len(frames)
        fps = info["fps"] if info["fps"] and info["fps"] > 0 else 25
        with tempfile.TemporaryDirectory() as tmp:
            for codec, args_ in CODECS.items():
                row = dict(clip=clip, codec=codec, frames_in=len(frames), status="ok")
                try:
                    dec_wm, size_wm = encode_decode(marked, args_, fps, tmp)
                    dec_ref, size_ref = encode_decode(frames, args_, fps, tmp)
                    n = min(len(dec_wm), len(frames))
                    row["frames_decoded"] = len(dec_wm)
                    t0 = time.perf_counter()
                    ncs, bers = [], []
                    for i in range(n):
                        ex = extract_watermark(dec_wm[i], keys[i])
                        ncs.append(normalized_correlation(logo, ex));
bers.append(bit_error_rate(logo, ex))
                    row["t_extract_ms_per_frame"] = (time.perf_counter() - t0) / n * 1e3
                    row["psnr_wm_vs_orig_db"] = float(np.mean([psnr(frames[i], dec_wm[i]) for i in
range(n)]))
                    row["psnr_codec_only_db"] = float(np.mean([psnr(frames[i], dec_ref[i]) for i in
range(n)]))
                    row["nc_mean"] = float(np.mean(ncs)); row["nc_min"] = float(np.min(ncs))
                    row["frames_nc_ge_0_75"] = int(np.sum(np.array(ncs) >= 0.75)); row["ber_mean"] =
float(np.mean(bers))
                    row["size_wm_kb"] = size_wm / 1024; row["size_ref_kb"] = size_ref / 1024
                    if codec == "lossless (FFV1)":
                        keep = [i for i in range(len(dec_wm)) if i % 10 != 9]
                        naive = [normalized_correlation(logo, extract_watermark(dec_wm[j], keys[i]))
for i, j in enumerate(keep)]
                        aware = [normalized_correlation(logo, extract_watermark(dec_wm[j], keys[j]))
for j in keep]
                        row["framedrop_nc_naive_index"] = float(np.mean(naive))
                        row["framedrop_nc_with_frame_ids"] = float(np.mean(aware))
                except Exception as exc:
                    row.update(status=f"error: {type(exc).__name__}: {exc}")
                rows.append(row)
        meta[clip]["embed_ms_per_frame"] = t_embed * 1e3
        meta[clip]["embed_fps_single_thread"] = 1 / t_embed
        print(clip, meta[clip], flush=True)
    df = pd.DataFrame(rows)
    df.to_csv(os.path.join(RES, "video_demo_results.csv"), index=False)
    json.dump(meta, open(os.path.join(RES, "video_demo_meta.json"), "w"), indent=2)
    pd.set_option("display.width", 250)
    print(df.round(3).to_string())


if __name__ == "__main__":
    main()
```

3.4 exp_jpeg_truncation.py

Truncation versus rounding before JPEG. 28 lines.

```
import io

import numpy as np
from PIL import Image
from skimage import data as skdata
```

```
from skimage.transform import resize

from logo import make_watermark_logo
from metrics import normalized_correlation as nc, psnr
from watermark_core import embed_watermark, extract_watermark


def jpeg(img, q, mode):
    a = np.clip(img, 0, 255)
    a = a.astype(np.uint8) if mode == "truncate" else np.rint(a).astype(np.uint8)
    b = io.BytesIO(); Image.fromarray(a).save(b, "JPEG", quality=q); b.seek(0)
    return np.array(Image.open(b).convert("L")).astype(np.float64)


if __name__ == "__main__":
    wm = make_watermark_logo()
    host = resize(skdata.camera().astype(np.float64), (512, 512), preserve_range=True)
    print(f"{'alpha':>6} {'PSNR':>7} {'NC trunc.':>10} {'NC round':>9} {'mean(trunc.-float)':>19}")
    for a in (0.02, 0.05, 0.08, 0.10, 0.15, 0.20, 0.30, 0.40):
        w, k = embed_watermark(host, wm, alpha=a)
        t, r = jpeg(w, 50, "truncate"), jpeg(w, 50, "round")
        print(f"{a:6.2f} {psnr(host, w):7.2f} {nc(wm, extract_watermark(t, k)):10.3f} {nc(wm,
extract_watermark(r, k)):9.3f} "
              f"{float((jpeg(w, 100, 'truncate') - w).mean()):19.3f}")
```

3.5 exp_principal_component_variant.py

Principal component embedding variant. 44 lines.

```
import os
import numpy as np
import pywt
from PIL import Image
from skimage.transform import resize

from logo import make_watermark_logo
from metrics import normalized_correlation as nc, psnr
from watermark_core import dct2, idct2

ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", ".."))


def emb_pc(host, wm, alpha):
    LL, (LH, HL, HH) = pywt.dwt2(host, "haar")
    wr = resize(wm, LL.shape, anti_aliasing=True, preserve_range=True) / 255.0
    U, S, Vt = np.linalg.svd(dct2(LL))
    Uw, Sw, Vtw = np.linalg.svd(wr)
    D = np.diag(S) + alpha * (Uw * Sw)
    U1, S1, V1t = np.linalg.svd(D)
    out = pywt.idwt2((idct2((U * S1) @ Vt), (LH, HL, HH)), "haar")[:host.shape[0], :host.shape[1]]
    return np.clip(out, 0, 255), dict(U1=U1, V1t=V1t, S=S, Vtw=Vtw, alpha=alpha, ts=LL.shape,
ws=wm.shape)


def ext_pc(img, key):
    LL, _ = pywt.dwt2(img, "haar")
    if LL.shape != key["ts"]:
        LL = resize(LL, key["ts"], anti_aliasing=True, preserve_range=True)
    S1s = np.linalg.svd(dct2(LL), compute_uv=False)
    D = (key["U1"] * S1s) @ key["V1t"]
    W = ((D - np.diag(key["S"])) / key["alpha"]) @ key["Vtw"]
    return np.clip(resize(W, key["ws"], anti_aliasing=True, preserve_range=True) * 255, 0, 255)


if __name__ == "__main__":
    wm, wmB = make_watermark_logo(), make_watermark_logo(text="OTHERLOGO")
    host = np.asarray(Image.open(os.path.join(ROOT, "data", "original", "image_066.png")),
dtype=np.float64)
    other = np.asarray(Image.open(os.path.join(ROOT, "data", "original", "image_064.png")),
dtype=np.float64)
    for a in (0.1, 1.0, 5.0):
        w, k = emb_pc(host, wm, a)
        wB, kB = emb_pc(host, wmB, a)
        print(f"alpha={a:<4} PSNR={psnr(host, w):6.2f} dB | genuine NC={nc(wm, ext_pc(w, k)):.3f} | "
              f"UNwatermarked host NC={nc(wm, ext_pc(host, k)):.3f} | other image NC={nc(wm,
ext_pc(other, k)):.3f} | "
              f"logo B decoded with key A NC_A={nc(wm, ext_pc(wB, k)):.3f}")
```

Part 4: Analysis and benchmarks

Statistics, figures and comparisons built from the recorded results.

4.1 analysis.py

Tables and figures for the main results. 379 lines.

```
from __future__ import annotations

import os
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")

import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from PIL import Image
from scipy import stats

from attacks import ATTACK_FAMILY, ATTACK_NAMES, make_attack_suite
from logo import make_watermark_logo
from metrics import normalized_correlation, psnr, ssim, zero_normalized_correlation
from watermark_core import embed_watermark, extract_watermark

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
RES = os.path.join(ROOT, "results")
TAB = os.path.join(RES, "tables")
VIS = os.path.join(ROOT, "visualizations")
DATA = os.path.join(ROOT, "data")
os.makedirs(TAB, exist_ok=True)
os.makedirs(VIS, exist_ok=True)

THR = 0.75
BLUE, ORANGE, AQUA, GREY, INK = "#2a78d6", "#eb6834", "#1baf7a", "#8a8a85", "#222222"
plt.rcParams.update({
    "font.size": 10, "axes.titlesize": 11, "axes.labelsize": 10, "axes.edgecolor": "#888888",
    "axes.spines.top": False, "axes.spines.right": False, "axes.grid": True, "grid.color": "#e4e4e0",
    "grid.linewidth": 0.7, "axes.axisbelow": True, "figure.facecolor": "white", "axes.facecolor":
"white",
    "savefig.dpi": 200, "savefig.bbox": "tight", "legend.frameon": False,
})
rng = np.random.default_rng(20260713)


def wilson(k, n, z=1.96):
    if n == 0:
        return (np.nan, np.nan)
    p = k / n
    d = 1 + z * z / n
    c = (p + z * z / (2 * n)) / d
    h = z * np.sqrt(p * (1 - p) / n + z * z / (4 * n * n)) / d
    return c - h, c + h


def boot_ci(x, n=2000):
    x = np.asarray(x)
    means = rng.choice(x, size=(n, len(x)), replace=True).mean(axis=1)
    return np.percentile(means, [2.5, 97.5])


def save(fig, name):
    fig.savefig(os.path.join(VIS, name))
    plt.close(fig)


def main():
    meta = pd.read_csv(os.path.join(DATA, "metadata.csv"))
    img = pd.read_csv(os.path.join(RES, "results_images.csv"))
    att = pd.read_csv(os.path.join(RES, "results_attacks.csv"))
    sw = pd.read_csv(os.path.join(RES, "results_alpha_sweep.csv"))
    ctl = pd.read_csv(os.path.join(RES, "results_controls.csv"))
    summ = json.load(open(os.path.join(RES, "run_summary.json")))
```

```
    prim = summ["config"]["primary_protocol"]
    cat = meta.set_index("image_id")[["category", "is_synthetic", "mean_intensity", "contrast_std",
"key"]]
    out = {}

    im = img.merge(meta[["image_id", "is_synthetic", "category"]], on="image_id")
    def desc(s):
        return dict(mean=s.mean(), std=s.std(), median=s.median(), min=s.min(), max=s.max())
    out["E1"] = {
        "n_images": int(len(im)),
        "psnr_all": desc(im.psnr_db), "ssim_all": desc(im.ssim),
        "psnr_real": desc(im[im.is_synthetic == 0].psnr_db), "ssim_real": desc(im[im.is_synthetic ==
0].ssim),
        "psnr_synthetic": desc(im[im.is_synthetic == 1].psnr_db), "ssim_synthetic":
desc(im[im.is_synthetic == 1].ssim),
        "n_psnr_below_30": int((im.psnr_db < 30).sum()),
        "n_ssim_below_0.98": int((im.ssim < 0.98).sum()),
        "ssim_below_0.98_ids": list(im[im.ssim < 0.98].image_id),
        "psnr_float_mean": float(im.psnr_float_db.mean()),
        "max_abs_diff_max": float(im.max_abs_diff.max()), "mean_abs_diff_mean":
float(im.mean_abs_diff.mean()),
        "t_embed_ms_median": float(im.t_embed_ms.median()), "key_mb": float(im.key_bytes.median() /
1e6),
    }
    t1 = pd.DataFrame({"PSNR (dB)": im.psnr_db.describe(), "SSIM": im.ssim.describe()}).round(4)
    t1.to_csv(os.path.join(TAB, "imperceptibility_summary.csv"))

    a = att[att.protocol == prim].merge(cat, left_on="image_id", right_index=True)
    rows = []
    for name in ATTACK_NAMES:
        x = a[a.attack == name]
        lo, hi = boot_ci(x.nc)
        k = int((x.nc >= THR).sum())
        wl, wh = wilson(k, len(x))
        real = x[x.is_synthetic == 0]
        rows.append(dict(attack=name, family=ATTACK_FAMILY[name], n=len(x), mean_nc=x.nc.mean(),
ci_lo=lo, ci_hi=hi,
                         median_nc=x.nc.median(), std_nc=x.nc.std(), min_nc=x.nc.min(), pass_rate=k /
len(x),
                         mean_zncc=x.zncc.mean(), median_zncc=x.zncc.median(), zncc_pass_rate=(x.zncc
>= THR).mean(),
                         pass_lo=wl, pass_hi=wh, mean_ber=x.ber.mean(),
mean_nc_real_only=real.nc.mean(),
                         pass_rate_real_only=(real.nc >= THR).mean(),
t_attack_ms=x.t_attack_ms.median(),
                         t_extract_ms=x.t_extract_ms.median()))
    rob = pd.DataFrame(rows)
    rob.to_csv(os.path.join(TAB, "robustness_by_attack.csv"), index=False)
    fam = a.groupby("family").agg(mean_nc=("nc", "mean"), pass_rate=("nc", lambda s: (s >=
THR).mean()), n=("nc", "size")).reset_index()
    fam.to_csv(os.path.join(TAB, "robustness_by_family.csv"), index=False)

    other = att[att.protocol != prim].merge(cat, left_on="image_id", right_index=True)
    cmp_rows = []
    for name in ATTACK_NAMES:
        u = a[a.attack == name].set_index("image_id").nc
        f = other[other.attack == name].set_index("image_id").nc
        cmp_rows.append(dict(attack=name, mean_nc_uint8=u.mean(), mean_nc_float=f.mean(),
                             max_abs_diff_per_image=(u - f.reindex(u.index)).abs().max()))
    pd.DataFrame(cmp_rows).to_csv(os.path.join(TAB, "protocol_comparison.csv"), index=False)

    out["E2"] = {
        "protocol": prim, "n_trials": int(len(a)), "mean_nc": float(a.nc.mean()),
        "n_trials_attacks_only": int((a.attack != "No Attack").sum()),
        "n_pass_attacks_only": int(((a.attack != "No Attack") & (a.nc >= THR)).sum()),
        "pass_rate_attacks_only": float((a[a.attack != "No Attack"].nc >= THR).mean()),
        "mean_nc_attacks_only": float(a[a.attack != "No Attack"].nc.mean()),
        "pass_rate": float((a.nc >= THR).mean()), "n_pass": int((a.nc >= THR).sum()),
        "pass_rate_real_only": float((a[a.is_synthetic == 0].nc >= THR).mean()),
        "mean_nc_real_only": float(a[a.is_synthetic == 0].nc.mean()),
        "mean_nc_float_protocol": float(other.nc.mean()), "pass_rate_float_protocol": float((other.nc
>= THR).mean()),
        "mean_ber": float(a.ber.mean()), "mean_zncc": float(a.zncc.mean()), "zncc_pass_rate":
float((a.zncc >= THR).mean()),
        "zncc_clean_mean": float(a[a.attack == "No Attack"].zncc.mean()),
        "clean_nc_mean": float(a[a.attack == "No Attack"].nc.mean()), "clean_nc_min":
float(a[a.attack == "No Attack"].nc.min()),
        "attacks_with_pass_rate_ge_0.95": list(rob[rob.pass_rate >= 0.95].attack),
```

```
        "weakest": rob.sort_values("pass_rate").head(3)[["attack", "pass_rate",
"mean_nc"]].to_dict("records"),
    }

    piv = a.pivot(index="image_id", columns="attack", values="nc").join(cat)
    cor = []
    for name in ATTACK_NAMES:
        if name == "No Attack":
            continue
        r1 = stats.spearmanr(piv["contrast_std"], piv[name])
        r2 = stats.spearmanr(piv["mean_intensity"], piv[name])
        cor.append(dict(attack=name, rho_contrast=r1[0], p_contrast=r1[1], rho_mean_intensity=r2[0],
p_mean_intensity=r2[1]))
    pd.DataFrame(cor).round(4).to_csv(os.path.join(TAB, "nc_vs_image_statistics.csv"), index=False)

    ct = a.groupby(["category", "family"]).nc.mean().unstack().round(3)
    ct["n_images"] = meta.groupby("category").size()
    ct.to_csv(os.path.join(TAB, "nc_by_category_family.csv"))
    ca = a.groupby(["category", "attack"]).nc.mean().unstack().round(3)
    ca.to_csv(os.path.join(TAB, "nc_by_category_attack.csv"))

    swo = sw[sw.status == "ok"]
    s = swo.groupby(["attack", "alpha"]).agg(psnr=("psnr_db", "mean"), psnr_sd=("psnr_db", "std"),
ssim=("ssim", "mean"),
                                             nc_mean=("nc", "mean"), nc_median=("nc", "median"),
                                             nc_q25=("nc", lambda x: x.quantile(.25)), nc_q75=("nc",
lambda x: x.quantile(.75)),
                                             zncc_mean=("zncc", "mean"), ber_mean=("ber", "mean"),
                                             pass_rate=("nc", lambda x: (x >=
THR).mean())).reset_index()
    s.round(4).to_csv(os.path.join(TAB, "alpha_sweep_summary.csv"), index=False)
    out["E3"] = {"table": s.round(4).to_dict("records")}
    logo0 = make_watermark_logo(summ["config"]["logo_size"], summ["config"]["logo_text"])
    U_, S_, Vt_ = np.linalg.svd(logo0)
    rank1 = np.clip((U_[:, :1] * S_[:1]) @ Vt_[:1], 0, 255)
    out["metric_baselines"] = {
        "logo_fill_ratio": float((logo0 > 0).mean()),
        "nc_constant_white_image": normalized_correlation(logo0, np.full_like(logo0, 255.0)),
        "nc_rank1_approximation": normalized_correlation(logo0, rank1),
        "zncc_rank1_approximation": zero_normalized_correlation(logo0, rank1),
        "sigma1_energy_fraction": float(S_[0] ** 2 / (S_ ** 2).sum()),
    }

    def sel(prefix):
        return ctl[ctl.control.str.startswith(prefix)].nc
    g = sel("genuine")
    crow = []
    for lbl, pre in [("genuine: own key, clean watermarked image", "genuine"),
                     ("a: other host, UNwatermarked, key of image i", "a:"),
                     ("b: other host, watermarked, key of image i", "b:"),
                     ("c: same host, different logo embedded, decoded with key A (NC vs logo A)", "c:
"),
                     ("c-ref: same host, logo B decoded with its own key (NC vs logo B)", "c-ref"),
                     ("d: own host unwatermarked, own key (degenerate)", "d:")]:
        x = sel(pre)
        z = ctl[ctl.control.str.startswith(pre)].zncc
        crow.append(dict(control=lbl, n=len(x), mean_nc=x.mean(), median=x.median(),
p95=x.quantile(.95), max=x.max(),
                         frac_ge_0_75=(x >= THR).mean(), frac_ge_0_90=(x >= .9).mean(),
                         mean_zncc=z.mean(), p95_zncc=z.quantile(.95), max_zncc=z.max(),
frac_zncc_ge_0_75=(z >= THR).mean()))
    pd.DataFrame(crow).round(4).to_csv(os.path.join(TAB, "controls_summary.csv"), index=False)
    auc = {}
    for k, pre in [("a", "a:"), ("b", "b:"), ("c", "c: ")]:
        x = sel(pre)
        u = stats.mannwhitneyu(g, x, alternative="greater")
        auc[k] = float(u.statistic / (len(g) * len(x)))
    out["E4"] = {"summary": pd.DataFrame(crow).round(4).to_dict("records"), "auc_genuine_vs_control":
auc}

    wrong_p95 = float(pd.concat([sel("a:"), sel("b:")]).quantile(.95))
    out["E4"]["wrong_image_p95"] = wrong_p95
    out["E4"]["attacked_genuine_below_wrong_p95"] = float((a.nc < wrong_p95).mean())

    tim = a.groupby("attack").agg(t_attack_ms=("t_attack_ms", "median"),
t_extract_ms=("t_extract_ms", "median")).reset_index()
    tim.round(3).to_csv(os.path.join(TAB, "timing_by_attack.csv"), index=False)
    out["timing"] = {"embed_ms_median": float(img.t_embed_ms.median()),
```

```
                     "extract_ms_median": float(a.t_extract_ms.median()),
                     "throughput_embed_per_s_single_core": float(1000 / img.t_embed_ms.median()),
                     "peak_rss_mb": float(img.peak_rss_mb.max())}

    json.dump(out, open(os.path.join(RES, "analysis_summary.json"), "w"), indent=2, default=float)

    fig, ax = plt.subplots(1, 2, figsize=(9, 3.3))
    real = im[im.is_synthetic == 0]; syn = im[im.is_synthetic == 1]
    ax[0].hist(real.psnr_db, bins=np.arange(35.4, 44.2, 0.2), color=BLUE, label=f"real
(n={len(real)})")
    ax[0].hist(syn.psnr_db, bins=np.arange(35.4, 44.2, 0.2), color=ORANGE, label=f"synthetic
(n={len(syn)})")
    ax[0].axvline(30, color=INK, lw=1, ls="--"); ax[0].text(30.1, ax[0].get_ylim()[1] * .9, "30 dB",
fontsize=8)
    ax[0].set_xlabel("PSNR of watermarked image (dB)"); ax[0].set_ylabel("images"); ax[0].legend()
    ax[1].hist(real.ssim, bins=np.linspace(0.55, 1.0, 46), color=BLUE)
    ax[1].hist(syn.ssim, bins=np.linspace(0.55, 1.0, 46), color=ORANGE)
    ax[1].axvline(0.98, color=INK, lw=1, ls="--"); ax[1].text(0.9805, ax[1].get_ylim()[1] * .9,
"0.98", fontsize=8)
    ax[1].set_xlabel("SSIM of watermarked image"); ax[1].set_ylabel("images")
    fig.suptitle(f"Imperceptibility at alpha = {summ['config']['alpha']} across {len(im)} images",
y=1.02, fontsize=11)
    fig.tight_layout(); save(fig, "fig_imperceptibility_hist.png")

    order = rob.sort_values("median_nc")["attack"].tolist()
    fig, ax = plt.subplots(figsize=(8.6, 6.2))
    data = [a[a.attack == n].nc.values for n in order]
    bp = ax.boxplot(data, orientation="horizontal", patch_artist=True, widths=0.6, showfliers=True,
                    flierprops=dict(marker="o", markersize=2.5, markerfacecolor=GREY,
markeredgecolor="none", alpha=.6),
                    medianprops=dict(color=INK, lw=1.4), whiskerprops=dict(color=GREY),
capprops=dict(color=GREY),
                    boxprops=dict(edgecolor=GREY))
    for patch, n in zip(bp["boxes"], order):
        patch.set_facecolor(BLUE if rob.set_index("attack").loc[n, "median_nc"] >= THR else ORANGE)
        patch.set_alpha(.85)
    ax.set_yticklabels(order)
    ax.axvline(THR, color=INK, ls="--", lw=1)
    ax.text(THR - .005, len(order) + 0.55, "threshold 0.75", fontsize=8, ha="right", va="center")
    for i, n in enumerate(order, 1):
        pr = rob.set_index("attack").loc[n, "pass_rate"]
        ax.text(1.03, i, f"{pr*100:.0f}%", va="center", fontsize=8.5, color=INK)
    ax.text(1.03, len(order) + 0.55, "NC >= 0.75", fontsize=8, color=INK, va="center")
    ax.set_xlim(-0.02, 1.12); ax.set_xlabel("Normalized correlation (NC) of extracted watermark, 100
images")
    ax.set_title("Robustness per attack: NC over 100 images (box = IQR, line = median)", loc="left")
    ax.grid(axis="y", visible=False)
    fig.tight_layout(); save(fig, "fig_nc_boxplot.png")

    r = rob.sort_values("pass_rate")
    fig, ax = plt.subplots(figsize=(8, 5.4))
    ax.barh(r.attack, r.pass_rate * 100, color=[BLUE if v >= .9 else ORANGE for v in r.pass_rate],
height=.62)
    ax.errorbar(r.pass_rate * 100, np.arange(len(r)), xerr=[np.clip((r.pass_rate - r.pass_lo) * 100,
0, None), np.clip((r.pass_hi - r.pass_rate) * 100, 0, None)],
                fmt="none", ecolor=INK, elinewidth=1, capsize=2)
    for i, v in enumerate(r.pass_rate):
        ax.text(min(v * 100 + 3.5, 104), i, f"{v*100:.0f}%", va="center", fontsize=8.5)
    ax.set_xlim(0, 112); ax.set_xlabel("Images with NC >= 0.75 (%)  - error bars: 95% Wilson
interval")
    ax.set_title("Detection pass rate per attack (n = 100 images each)")
    ax.grid(axis="y", visible=False)
    fig.tight_layout(); save(fig, "fig_pass_rate.png")

    fig, ax = plt.subplots(1, 3, figsize=(11.5, 3.5))
    p0 = s[s.attack == "No Attack"]
    ax[0].plot(p0.alpha, p0.psnr, color=BLUE, marker="o", ms=4, lw=1.6)
    ax[0].fill_between(p0.alpha, p0.psnr - p0.psnr_sd, p0.psnr + p0.psnr_sd, color=BLUE, alpha=.15,
lw=0)
    ax[0].axhline(30, color=INK, lw=1, ls="--"); ax[0].set_ylabel("mean PSNR (dB)")
    ax[0].set_title("Imperceptibility (band = +-1 sd)")
    cols = {"JPEG Q=50": BLUE, "Gaussian Noise (var=0.02)": ORANGE, "Gaussian Blur (sigma=1.5)":
AQUA, "Rotation 5 deg": GREY}
    for an, c in cols.items():
        q = s[s.attack == an]
        ax[1].plot(q.alpha, q.nc_mean, color=c, marker="o", ms=3.5, lw=1.5, label=an)
        ax[2].plot(q.alpha, q.pass_rate * 100, color=c, marker="o", ms=3.5, lw=1.5, label=an)
```

```
    ax[1].axhline(THR, color=INK, lw=1, ls="--"); ax[1].set_ylim(0.4, 1.02); ax[1].set_ylabel("mean
NC (100 images)")
    ax[1].set_title("Robustness: mean NC after attack"); ax[1].legend(fontsize=7.5, loc="lower
right")
    ax[2].set_ylim(0, 102); ax[2].set_ylabel("images with NC >= 0.75 (%)");
ax[2].set_title("Robustness: detection pass rate")
    for x_ in ax:
        x_.set_xlabel("embedding strength alpha")
    fig.tight_layout(); save(fig, "fig_alpha_tradeoff.png")

    fig, ax = plt.subplots(figsize=(8, 5.2))
    rr = rob.sort_values("mean_nc")
    yy = np.arange(len(rr))
    ax.barh(yy + .18, rr.mean_nc, height=.34, color=BLUE, label="NC (original metric)")
    ax.barh(yy - .18, rr.mean_zncc, height=.34, color=ORANGE, label="ZNCC (mean-centred)")
    ax.set_yticks(yy); ax.set_yticklabels(rr.attack); ax.axvline(THR, color=INK, ls="--", lw=1)
    ax.set_xlabel("mean over 100 images"); ax.legend(loc="lower right"); ax.grid(axis="y",
visible=False)
    ax.set_title("Plain NC is optimistic: ZNCC removes the brightness offset")
    fig.tight_layout(); save(fig, "fig_nc_vs_zncc.png")

    fig, ax = plt.subplots(figsize=(8, 4))
    bins = np.linspace(0, 1, 41)
    ax.hist(sel("a:"), bins=bins, alpha=.85, color=ORANGE, label="a: other host, unwatermarked",
density=True)
    ax.hist(sel("b:"), bins=bins, alpha=.55, color=AQUA, label="b: other host, watermarked (wrong
key)", density=True)
    ax.hist(g, bins=bins, alpha=.75, color=BLUE, label="genuine: own key", density=True)
    ax.axvline(THR, color=INK, ls="--", lw=1)
    ax.set_xlabel("NC against the owner's logo"); ax.set_ylabel("density")
    ax.set_title("Control experiments: wrong-image/wrong-key NC overlaps the 0.75 threshold")
    ax.legend(loc="upper center", fontsize=8)
    fig.tight_layout(); save(fig, "fig_controls_hist.png")

    cc = sel("c: "); cr = sel("c-ref")
    fig, ax = plt.subplots(figsize=(6.4, 3.4))
    vals = [g.mean(), cc.mean(), cr.mean(), sel("a:").mean(), sel("b:").mean()]
    labs = ["genuine\n(logo A, key A)", "logo B embedded,\ndecoded with key A", "logo B embedded,\
ndecoded with key B", "other host,\nunwatermarked", "other host,\nwrong key"]
    ax.bar(labs, vals, color=[BLUE, ORANGE, BLUE, GREY, GREY], width=.6)
    for i, v in enumerate(vals):
        ax.text(i, v + .015, f"{v:.3f}", ha="center", fontsize=9)
    ax.set_ylim(0, 1.12); ax.set_ylabel("mean NC vs the logo tested")
    ax.set_title("False-positive problem: the wrong logo is reported as the owner's logo")
    ax.tick_params(axis="x", labelsize=8); ax.grid(axis="x", visible=False)
    fig.tight_layout(); save(fig, "fig_false_positive.png")

    hm = ct.drop(columns="n_images")
    hm = hm.loc[hm.index != "synthetic_control"]
    fig, ax = plt.subplots(figsize=(8.4, 6.2))
    im_ = ax.imshow(hm.values, cmap="Blues", vmin=0, vmax=1, aspect="auto")
    ax.set_xticks(range(hm.shape[1])); ax.set_xticklabels(hm.columns, rotation=30, ha="right")
    ax.set_yticks(range(hm.shape[0])); ax.set_yticklabels([f"{c} ({int(ct.loc[c,'n_images'])})" for c
in hm.index])
    for i in range(hm.shape[0]):
        for j in range(hm.shape[1]):
            v = hm.values[i, j]
            ax.text(j, i, f"{v:.2f}", ha="center", va="center", fontsize=8, color="white" if v > .62
else INK)
    ax.grid(False); fig.colorbar(im_, ax=ax, fraction=.04, label="mean NC")
    ax.set_title("Mean NC by image category and attack family (real images)")
    fig.tight_layout(); save(fig, "fig_category_heatmap.png")

    fig, ax = plt.subplots(1, 3, figsize=(10.5, 3.3), sharey=True)
    for k, name in enumerate(["Gaussian Noise (var=0.02)", "Rotation 15 deg", "Histogram
Equalization"]):
        d = piv
        ax[k].scatter(d["contrast_std"], d[name], s=14, c=np.where(d["is_synthetic"] == 1, ORANGE,
BLUE), alpha=.8, lw=0)
        rho = stats.spearmanr(d["contrast_std"], d[name])[0]
        ax[k].set_title(f"{name}\nSpearman rho = {rho:.2f}", fontsize=9.5)
        ax[k].set_xlabel("image contrast (std of grey levels)"); ax[k].axhline(THR, color=INK,
ls="--", lw=1)
    ax[0].set_ylabel("NC after attack")
    fig.tight_layout(); save(fig, "fig_nc_vs_contrast.png")

    fig, ax = plt.subplots(figsize=(6.5, 3.2))
```

```
    ax.hist(img.t_embed_ms, bins=20, color=BLUE, label=f"embed (median {img.t_embed_ms.median():.0f}
ms)")
    ax.hist(a.t_extract_ms, bins=40, color=ORANGE, alpha=.8, label=f"extract (median
{a.t_extract_ms.median():.0f} ms)")
    ax.set_xlabel("time per 512x512 image (ms, single thread)"); ax.set_ylabel("operations");
ax.legend()
    fig.tight_layout(); save(fig, "fig_timing.png")

    cfg = summ["config"]; alpha = cfg["alpha"]
    logo = make_watermark_logo(cfg["logo_size"], cfg["logo_text"])
    Image.fromarray(logo.astype(np.uint8)).save(os.path.join(VIS, "watermark_logo.png"))
    reps = ["image_066", "image_064", "image_001", "image_051", "image_055", "image_100"]
    fig, axes = plt.subplots(len(reps), 3, figsize=(8.4, 2.8 * len(reps)))
    for r_i, iid in enumerate(reps):
        host = np.asarray(Image.open(os.path.join(DATA, "original", iid + ".png")), dtype=np.float64)
        w = np.asarray(Image.open(os.path.join(ROOT, "outputs", "watermarked", iid + "_wm.png")),
dtype=np.float64)
        d = np.abs(host - w)
        row = img.set_index("image_id").loc[iid]
        key = meta.set_index("image_id").loc[iid, "key"]
        axes[r_i, 0].imshow(host, cmap="gray", vmin=0, vmax=255); axes[r_i, 0].set_title(f"{iid}
({key}) original", fontsize=8.5)
        axes[r_i, 1].imshow(w, cmap="gray", vmin=0, vmax=255); axes[r_i, 1].set_title(f"watermarked
PSNR {row.psnr_db:.1f} dB, SSIM {row.ssim:.3f}", fontsize=8.5)
        im2 = axes[r_i, 2].imshow(np.clip(d * 10, 0, 255), cmap="magma", vmin=0, vmax=255); axes[r_i,
2].set_title(f"|difference| x10 (max {d.max():.0f} grey levels)", fontsize=8.5)
        for x in axes[r_i]:
            x.axis("off")
    fig.tight_layout(); save(fig, "fig_representative_comparison.png")

    iid = "image_066"
    host = np.asarray(Image.open(os.path.join(DATA, "original", iid + ".png")), dtype=np.float64)
    w, key = embed_watermark(host, logo, alpha=alpha)
    wq = np.clip(np.rint(w), 0, 255)
    suite = make_attack_suite(cfg["base_seed"] + 100 * (int(iid[-3:]) - 1))
    fig, axes = plt.subplots(3, 6, figsize=(12, 6.6))
    axes[0, 0].imshow(logo, cmap="gray"); axes[0, 0].set_title("original logo", fontsize=8.5)
    for ax_, name in zip(axes.flat[1:], ATTACK_NAMES):
        ext = extract_watermark(suite[name](wq), key)
        ax_.imshow(ext, cmap="gray", vmin=0, vmax=255)
        ax_.set_title(f"{name}\nNC={normalized_correlation(logo, ext):.3f}", fontsize=7.8)
    for ax_ in axes.flat[16:]:
        ax_.axis("off")
    for ax_ in axes.flat:
        ax_.set_xticks([]); ax_.set_yticks([]); ax_.grid(False)
    fig.suptitle("Extracted watermark under each attack (image_066, cameraman)", y=1.0, fontsize=11)
    fig.tight_layout(); save(fig, "fig_extraction_gallery.png")

    T = 128
    sheet = Image.new("L", (10 * T, 10 * T), 255)
    for i, iid in enumerate(meta.image_id):
        sheet.paste(Image.open(os.path.join(ROOT, "outputs", "watermarked", iid +
"_wm.png")).resize((T, T)), ((i % 10) * T, (i // 10) * T))
    sheet.save(os.path.join(VIS, "contact_sheet_watermarked.png"))
    print(json.dumps({k: out[k] for k in ("E1", "E2")}, indent=1, default=float)[:3000])


if __name__ == "__main__":
    main()
```

4.2 analysis_tech.py

Figures for the technology comparison. 71 lines.

```
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
RES, VIS = os.path.join(ROOT, "results"), os.path.join(ROOT, "visualizations")
BLUE, ORANGE, AQUA, GREY, INK = "#2a78d6", "#eb6834", "#1baf7a", "#8a8a85", "#222222"
plt.rcParams.update({"font.size": 10, "axes.spines.top": False, "axes.spines.right": False,
"axes.grid": True,
```

```
                    "grid.color": "#e4e4e0", "axes.axisbelow": True, "savefig.dpi": 200,
"savefig.bbox": "tight",
                    "legend.frameon": False})

A = pd.read_csv(os.path.join(RES, "tech_benchmark_components.csv"))
fig, ax = plt.subplots(figsize=(9, 8))
labels, vals, cols, ypos = [], [], [], []
y = 0
for op, g in A.groupby("operation", sort=False):
    ax.text(0.0008, y - 0.1, op, fontsize=9, fontweight="bold", va="bottom",
transform=ax.get_yaxis_transform() if False else ax.transData)
    y += 0.9
    for _, r in g.iterrows():
        labels.append(r.implementation); vals.append(r.median_ms)
        cols.append(BLUE if "(project)" in r.implementation else (ORANGE if r.includes_process_and_io
else GREY))
        ypos.append(y); y += 1
    y += 0.5
ax.barh(ypos, vals, color=cols, height=.7)
ax.set_yticks(ypos); ax.set_yticklabels(labels, fontsize=8); ax.invert_yaxis(); ax.set_xscale("log")
for yy, v in zip(ypos, vals):
    ax.text(v * 1.08, yy, f"{v:.2f} ms", va="center", fontsize=7.5)
ax.set_xlim(0.03, 400); ax.set_xlabel("median time per call, ms (log scale; single thread, in-
memory)"); ax.grid(axis="y", visible=False)
ax.set_title("Same operation, different libraries (blue = implementation used by the project;\norange
= includes process start-up and file I/O)", fontsize=10)
fig.tight_layout(); fig.savefig(os.path.join(VIS, "fig_tech_component_speed.png")); plt.close(fig)

S = pd.read_csv(os.path.join(RES, "tech_benchmark_schemes.csv"))
atts = ["No Attack", "JPEG Q=50", "JPEG Q=20", "Gaussian Noise (var=0.02)", "Median Filter 3x3",
"Gaussian Blur (sigma=1.5)",
        "Rotation 5 deg", "Cropping 25%", "Scaling 0.5x"]
order = ["Project: global DWT-DCT-SVD", "blind-watermark 0.4.4 (block DWT-DCT-SVD)", "Baseline: 8x8
block DCT", "Baseline: spatial LSB"]
cols = {order[0]: BLUE, order[1]: ORANGE, order[2]: AQUA, order[3]: GREY}
fig, ax = plt.subplots(figsize=(10, 4.2))
w = 0.2
for i, s in enumerate(order):
    d = S[S.scheme == s].set_index("attack").reindex(atts)
    ax.bar(np.arange(len(atts)) + (i - 1.5) * w, d.mean_ber, width=w * .92, color=cols[s],
           label=f"{s}  (PSNR {d.psnr_db.mean():.1f} dB)")
ax.axhline(0.5, color=INK, lw=.8, ls=":"); ax.text(len(atts) - .5, 0.505, "0.5 = random guess",
ha="right", fontsize=8)
ax.set_xticks(range(len(atts))); ax.set_xticklabels([a.replace(" (", "\n(") for a in atts],
fontsize=8)
ax.set_ylabel("mean bit-error rate of recovered logo (30 images)"); ax.set_ylim(0, .6)
ax.legend(fontsize=8, loc="upper left", bbox_to_anchor=(0, 1.32), ncol=2)
ax.set_title("Same 4095-bit payload, same attacks", y=1.0, loc="right", fontsize=9)
fig.tight_layout(); fig.savefig(os.path.join(VIS, "fig_tech_scheme_ber.png")); plt.close(fig)
print("ok")

best = (A[~A.includes_process_and_io].sort_values("speed_vs_project",
ascending=False).groupby("operation", sort=False).head(1)
        .sort_values("speed_vs_project"))
fig, ax = plt.subplots(figsize=(8.2, 4.6))
y = np.arange(len(best))
cols_ = [BLUE if v > 1.5 else GREY for v in best.speed_vs_project]
ax.barh(y, best.speed_vs_project, color=cols_, height=.62)
ax.set_xscale("log"); ax.set_xlim(0.7, 1500)
ax.axvline(1, color=INK, lw=1)
ax.set_yticks(y); ax.set_yticklabels([o.replace(", 512x512", "").replace(" 512x512", "").replace("
256x256", "") for o in best.operation], fontsize=9.5)
for yy, (_, r) in zip(y, best.iterrows()):
    lab = "project's own library is fastest" if "(project)" in r.implementation else
r.implementation.replace(" (project)", "")
    ax.text(r.speed_vs_project * 1.12, yy, f"{r.speed_vs_project:.1f}×  {lab}", va="center",
fontsize=8.8)
ax.set_xlabel("speed-up of the fastest alternative vs the project's implementation (× , log scale)")
ax.set_title("Best alternative per operation (single thread, in-memory; outputs identical or ≥ 48 dB
PSNR apart)", fontsize=10.5)
ax.grid(axis="y", visible=False)
fig.tight_layout(); fig.savefig(os.path.join(VIS, "fig_tech_speedup.png")); plt.close(fig)
print("speedup ok")
```

4.3 benchmark_tech.py

Library and scheme benchmarks. 433 lines.

```
from __future__ import annotations

import os
for _v in ("OPENBLAS_NUM_THREADS", "OMP_NUM_THREADS", "MKL_NUM_THREADS"):
    os.environ.setdefault(_v, "1")

import io
import json
import shutil
import subprocess
import sys
import tempfile
import time
import warnings

import cv2
import numpy as np
import pandas as pd
import pywt
import scipy
import scipy.fft
import scipy.linalg
import skimage
from PIL import Image, ImageFilter
from scipy import ndimage
from scipy.fftpack import dct as _dct
from skimage import exposure, filters as skfilters, transform as sktransform
from skimage.metrics import peak_signal_noise_ratio as sk_psnr

from attacks import make_attack_suite
from logo import make_watermark_logo
from metrics import psnr
from watermark_core import dct2, embed_watermark, extract_watermark

warnings.filterwarnings("ignore")
cv2.setNumThreads(1)
HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
RES = os.path.join(ROOT, "results")
VIS = os.path.join(ROOT, "visualizations")
DATA = os.path.join(ROOT, "data")


def load(iid):
    return np.asarray(Image.open(os.path.join(DATA, "original", iid + ".png")), dtype=np.float64)


def tmed(fn, repeats=9, warm=2):
    for _ in range(warm):
        fn()
    ts = []
    for _ in range(repeats):
        t = time.perf_counter()
        fn()
        ts.append((time.perf_counter() - t) * 1e3)
    return float(np.median(ts)), float(np.percentile(ts, 25)), float(np.percentile(ts, 75))


def have(cmd):
    return shutil.which(cmd) is not None


def dct_matrix(n):
    k = np.arange(n)[:, None]; i = np.arange(n)[None, :]
    c = np.sqrt(2.0 / n) * np.cos(np.pi * (2 * i + 1) * k / (2 * n))
    c[0] /= np.sqrt(2)
    return c


def haar_numpy(a):
    a00, a01, a10, a11 = a[0::2, 0::2], a[0::2, 1::2], a[1::2, 0::2], a[1::2, 1::2]
    return (a00 + a01 + a10 + a11) / 2.0


def jpeg_pillow(a, q=50):
    b = io.BytesIO(); Image.fromarray(a.astype(np.uint8)).save(b, "JPEG", quality=q); n = b.tell();
b.seek(0)
    return np.array(Image.open(b)).astype(np.float64), n
```

```
def jpeg_cv2(a, q=50):
    ok, buf = cv2.imencode(".jpg", a.astype(np.uint8), [cv2.IMWRITE_JPEG_QUALITY, q])
    return cv2.imdecode(buf, cv2.IMREAD_GRAYSCALE).astype(np.float64), len(buf)


def jpeg_magick(a, q=50, tmp=None):
    src, dst = os.path.join(tmp, "in.png"), os.path.join(tmp, "out.jpg")
    Image.fromarray(a.astype(np.uint8)).save(src)
    subprocess.run(["convert", src, "-quality", str(q), dst], check=True)
    return np.array(Image.open(dst).convert("L")).astype(np.float64), os.path.getsize(dst)


def rot_skimage(a):
    return sktransform.rotate(a / 255.0, 15, resize=False, mode="edge") * 255.0


def rot_cv2(a):
    h, w = a.shape
    M = cv2.getRotationMatrix2D(((w - 1) / 2, (h - 1) / 2), 15, 1.0)
    return cv2.warpAffine(a, M, (w, h), flags=cv2.INTER_LINEAR, borderMode=cv2.BORDER_REPLICATE)


def rot_pillow(a):
    return np.array(Image.fromarray(a.astype(np.uint8)).rotate(15,
resample=Image.BILINEAR)).astype(np.float64)


def scale_skimage(a):
    s = sktransform.resize(a, (a.shape[0] // 2, a.shape[1] // 2), anti_aliasing=True,
preserve_range=True)
    return sktransform.resize(s, a.shape, anti_aliasing=True, preserve_range=True)


def scale_cv2(a):
    s = cv2.resize(a, (a.shape[1] // 2, a.shape[0] // 2), interpolation=cv2.INTER_AREA)
    return cv2.resize(s, (a.shape[1], a.shape[0]), interpolation=cv2.INTER_LINEAR)


def scale_pillow(a):
    im = Image.fromarray(a.astype(np.uint8))
    s = im.resize((a.shape[1] // 2, a.shape[0] // 2), Image.BILINEAR, reducing_gap=None)
    return np.array(s.resize(im.size, Image.BILINEAR)).astype(np.float64)


def part_a(images):
    tmp = tempfile.mkdtemp()
    rows = []
    h512 = images[0]
    ll = h512[:256, :256].copy()
    u8 = h512.astype(np.uint8)
    C = dct_matrix(256)

    ops = {
        "2-D DCT-II 256x256": [
            ("scipy.fftpack.dct (project)", lambda: dct2(ll), "ref"),
            ("scipy.fft.dctn", lambda: scipy.fft.dctn(ll, norm="ortho"), ""),
            ("cv2.dct", lambda: cv2.dct(ll), ""),
            ("NumPy matrix C.A.C^T", lambda: C @ ll @ C.T, ""),
        ],
        "SVD 256x256": [
            ("numpy.linalg.svd (project)", lambda: np.linalg.svd(ll, full_matrices=False)[1], "ref"),
            ("scipy.linalg.svd (gesdd)", lambda: scipy.linalg.svd(ll, full_matrices=False,
check_finite=False)[1], ""),
            ("scipy.linalg.svd (gesvd)", lambda: scipy.linalg.svd(ll, full_matrices=False,
lapack_driver="gesvd", check_finite=False)[1], ""),
            ("cv2.SVDecomp", lambda: cv2.SVDecomp(ll)[0].ravel(), ""),
        ],
        "Haar DWT level-1 (LL) 512x512": [
            ("PyWavelets dwt2 (project)", lambda: pywt.dwt2(h512, "haar")[0], "ref"),
            ("NumPy slicing", lambda: haar_numpy(h512), ""),
        ],
        "JPEG round-trip Q=50, 512x512": [
            ("Pillow (project)", lambda: jpeg_pillow(h512)[0], "ref"),
            ("OpenCV imencode/imdecode", lambda: jpeg_cv2(h512)[0], ""),
        ] + ([("ImageMagick convert (process + file I/O)", lambda: jpeg_magick(h512, tmp=tmp)[0],
"io")] if have("convert") else []),
```

```
        "Median filter 3x3": [
            ("scipy.ndimage (project)", lambda: ndimage.median_filter(h512, size=3), "ref"),
            ("cv2.medianBlur (uint8)", lambda: cv2.medianBlur(u8, 3), ""),
            ("skimage.filters.median (uint8)", lambda: skfilters.median(u8, np.ones((3, 3), bool)),
""),
        ],
        "Gaussian blur sigma=1.5": [
            ("scipy.ndimage (project)", lambda: ndimage.gaussian_filter(h512, 1.5), "ref"),
            ("cv2.GaussianBlur", lambda: cv2.GaussianBlur(h512, (0, 0), 1.5,
borderType=cv2.BORDER_REFLECT), ""),
            ("skimage.filters.gaussian", lambda: skfilters.gaussian(h512, sigma=1.5,
preserve_range=True), ""),
        ],
        "Rotation 15 deg (bilinear)": [
            ("skimage.transform.rotate (project)", lambda: rot_skimage(h512), "ref"),
            ("cv2.warpAffine", lambda: rot_cv2(h512), ""),
            ("Pillow Image.rotate", lambda: rot_pillow(h512), ""),
        ],
        "Down/up-scale 0.5x": [
            ("skimage.transform.resize (project)", lambda: scale_skimage(h512), "ref"),
            ("cv2.resize (AREA down, LINEAR up)", lambda: scale_cv2(h512), ""),
            ("Pillow resize (BILINEAR)", lambda: scale_pillow(h512), ""),
        ],
        "Histogram equalisation": [
            ("skimage.exposure.equalize_hist (project)", lambda: exposure.equalize_hist(h512 / 255.0)
* 255.0, "ref"),
            ("cv2.equalizeHist (uint8)", lambda: cv2.equalizeHist(u8).astype(np.float64), ""),
        ],
        "PSNR 512x512": [
            ("skimage.metrics (project)", lambda: sk_psnr(h512, h512 + 1.0, data_range=255), "ref"),
            ("cv2.PSNR", lambda: cv2.PSNR(h512, h512 + 1.0), ""),
            ("NumPy formula", lambda: 10 * np.log10(255 ** 2 / np.mean((h512 - (h512 + 1.0)) ** 2)),
""),
        ],
    }
    for op, variants in ops.items():
        ref = np.asarray(variants[0][1]())
        for label, fn, tag in variants:
            out = np.asarray(fn())
            med, q1, q3 = tmed(fn)
            row = dict(operation=op, implementation=label, median_ms=med, q25_ms=q1, q75_ms=q3,
                       includes_process_and_io=(tag == "io"))
            if out.shape == ref.shape and out.size > 1:
                if op.startswith("Rotation"):
                    sl = (slice(128, 384), slice(128, 384))
                    d = out[sl] - ref[sl]
                    row["agreement"] = "PSNR %.1f dB (interior)" % (10 * np.log10(255 ** 2 /
max(np.mean(d ** 2), 1e-12)))
                elif op.startswith("SVD") or op.startswith("2-D DCT"):
                    scale = max(np.abs(ref).max(), 1e-12)
                    row["agreement"] = "max rel. diff %.1e" % (np.abs(out - ref).max() / scale)
                elif op.startswith("Haar"):
                    row["agreement"] = "max abs diff %.1e" % np.abs(out - ref).max()
                else:
                    d = out.astype(np.float64) - ref.astype(np.float64)
                    mse = np.mean(d ** 2)
                    row["agreement"] = "identical" if mse == 0 else "PSNR %.1f dB" % (10 *
np.log10(255 ** 2 / mse))
            else:
                row["agreement"] = "%.4f vs %.4f" % (float(np.ravel(out)[0]), float(np.ravel(ref)
[0]))
            rows.append(row)
    j = {"Pillow": jpeg_pillow(h512)[1], "OpenCV": jpeg_cv2(h512)[1]}
    if have("convert"):
        j["ImageMagick"] = jpeg_magick(h512, tmp=tmp)[1]
    shutil.rmtree(tmp, ignore_errors=True)
    return pd.DataFrame(rows), j


NB = 4095


def bits_to_logo(bits):
    b = np.zeros(4096); b[:NB] = np.asarray(bits[:NB], dtype=float)
    return b.reshape(64, 64) * 255.0


class Scheme:
```

```
    name = ""
    blind = None
    needs = ""


class OursScheme(Scheme):
    name = "Project: global DWT-DCT-SVD"
    blind = False
    needs = "owner's key (S_orig, Uw, Vw), ~2.1 MB per image"

    def __init__(self, alpha=0.10):
        self.alpha = alpha
        self.logo = make_watermark_logo()

    def embed(self, host):
        w, self.key = embed_watermark(host, self.logo, alpha=self.alpha)
        return w

    def extract(self, img):
        return extract_watermark(img, self.key) > 127.5


class LSBScheme(Scheme):
    name = "Baseline: spatial LSB"
    blind = True
    needs = "payload length only"

    def __init__(self, bits):
        self.bits = np.asarray(bits[:NB], dtype=np.uint8)

    def embed(self, host):
        flat = np.rint(host).astype(np.uint8).flatten()
        flat[:NB] = (flat[:NB] & 0xFE) | self.bits
        return flat.reshape(host.shape).astype(np.float64)

    def extract(self, img):
        flat = np.clip(np.rint(img), 0, 255).astype(np.uint8).flatten()
        return bits_to_logo(flat[:NB] & 1) > 127.5


class BlockDCTScheme(Scheme):
    name = "Baseline: 8x8 block DCT"
    blind = True
    needs = "payload length only"

    def __init__(self, bits, delta=24.0):
        self.bits = np.asarray(bits[:NB], dtype=np.uint8)
        self.delta = delta
        n = 8
        k = np.arange(n)[:, None]; i = np.arange(n)[None, :]
        c = np.sqrt(2.0 / n) * np.cos(np.pi * (2 * i + 1) * k / (2 * n)); c[0] /= np.sqrt(2)
        self.C = c

    def embed(self, host):
        out = host.astype(np.float64).copy()
        H, W = host.shape
        idx = 0
        for by in range(0, H, 8):
            for bx in range(0, W, 8):
                if idx >= NB:
                    break
                blk = out[by:by + 8, bx:bx + 8]
                d = self.C @ blk @ self.C.T
                a, b = d[4, 3], d[3, 4]
                m = (a + b) / 2
                if self.bits[idx]:
                    d[4, 3], d[3, 4] = m + self.delta / 2, m - self.delta / 2
                else:
                    d[4, 3], d[3, 4] = m - self.delta / 2, m + self.delta / 2
                out[by:by + 8, bx:bx + 8] = self.C.T @ d @ self.C
                idx += 1
        return np.clip(out, 0, 255)

    def extract(self, img):
        H, W = img.shape
        bits = []
        for by in range(0, H, 8):
            for bx in range(0, W, 8):
```

```
                if len(bits) >= NB:
                    break
                d = self.C @ img[by:by + 8, bx:bx + 8] @ self.C.T
                bits.append(1 if d[4, 3] > d[3, 4] else 0)
        return bits_to_logo(bits) > 127.5


class BlindWMScheme(Scheme):
    name = "blind-watermark 0.4.4 (block DWT-DCT-SVD)"
    blind = True
    needs = "two integer passwords + payload length"

    def __init__(self, bits):
        from blind_watermark import WaterMark
        self.WaterMark = WaterMark
        self.bits = np.asarray(bits[:NB], dtype=bool)

    def embed(self, host):
        bwm = self.WaterMark(password_img=1, password_wm=1, processes=1)
        bwm.read_img(img=np.dstack([host.astype(np.uint8)] * 3))
        bwm.read_wm(self.bits, mode="bit")
        e = bwm.embed()
        return np.clip(np.rint(e), 0, 255)

    def extract(self, img3):
        bwm = self.WaterMark(password_img=1, password_wm=1, processes=1)
        ex = bwm.extract(embed_img=np.clip(np.rint(img3), 0, 255).astype(np.uint8), wm_shape=NB,
mode="bit")
        return bits_to_logo(np.asarray(ex) > 0.5) > 127.5


def channelwise(fn, img):
    if img.ndim == 2:
        return fn(img)
    return np.stack([fn(img[..., c]) for c in range(img.shape[2])], axis=-1)


def luma(img):
    return img if img.ndim == 2 else cv2.cvtColor(img.astype(np.float32),
cv2.COLOR_BGR2GRAY).astype(np.float64)


def part_b(ids, attacks):
    logo = make_watermark_logo()
    bits = (logo.flatten() > 127)[:NB]
    ref_logo = bits_to_logo(bits)
    rows = []
    for k, iid in enumerate(ids):
        host = load(iid)
        suite = make_attack_suite(777 + k)
        schemes = [OursScheme(), BlindWMScheme(bits), LSBScheme(bits), BlockDCTScheme(bits)]
        for sch in schemes:
            try:
                t0 = time.perf_counter(); w = sch.embed(host); t_emb = (time.perf_counter() - t0) *
1e3
                wq = np.clip(np.rint(w), 0, 255)
                p = psnr(host, luma(wq))
            except Exception as exc:
                rows.append(dict(image_id=iid, scheme=sch.name, attack="(embed)", status=f"error:
{exc}"))
                continue
            for an in attacks:
                row = dict(image_id=iid, scheme=sch.name, attack=an, psnr_db=p, t_embed_ms=t_emb,
status="ok")
                try:
                    att = channelwise(suite[an], wq)
                    t0 = time.perf_counter(); ex = sch.extract(att); row["t_extract_ms"] =
(time.perf_counter() - t0) * 1e3
                    ex_img = ex.astype(np.float64) * 255.0
                    row["ber"] = float(np.mean((ex_img > 127.5) != (ref_logo > 127.5)))
                    a, b = ref_logo.flatten(), ex_img.flatten()
                    den = np.sqrt((a ** 2).sum()) * np.sqrt((b ** 2).sum())
                    row["nc"] = float((a * b).sum() / den) if den else 0.0
                except Exception as exc:
                    row.update(status=f"error: {type(exc).__name__}: {exc}")
                rows.append(row)
        print("scheme comparison: image", k + 1, "/", len(ids), flush=True)
    return pd.DataFrame(rows)
```

```
def tune_delta(ids):
    logo = make_watermark_logo(); bits = (logo.flatten() > 127)[:NB]
    target = np.mean([psnr(load(i), OursScheme().embed(load(i))) for i in ids])
    best = None
    for d in (8, 12, 16, 20, 24, 28, 32, 40, 48):
        p = np.mean([psnr(load(i), np.rint(BlockDCTScheme(bits, d).embed(load(i)))) for i in ids])
        if best is None or abs(p - target) < abs(best[1] - target):
            best = (d, p)
    return best, target


def main():
    os.makedirs(RES, exist_ok=True)
    meta = pd.read_csv(os.path.join(DATA, "metadata.csv"))
    real = meta[meta.is_synthetic == 0].image_id.tolist()
    rng = np.random.default_rng(20260713)

    comp_ids = list(rng.choice(real, 3, replace=False))
    parts = []
    for iid in comp_ids:
        df, jsz = part_a([load(iid)])
        df["image_id"] = iid
        parts.append(df)
    A = pd.concat(parts)
    Agg = A.groupby(["operation", "implementation"], sort=False).agg(
        median_ms=("median_ms", "median"), q25_ms=("q25_ms", "median"), q75_ms=("q75_ms", "median"),
        includes_process_and_io=("includes_process_and_io", "first"), agreement=("agreement",
"first")).reset_index()
    ref = Agg.groupby("operation", sort=False).median_ms.transform("first")
    Agg["speed_vs_project"] = ref / Agg.median_ms
    Agg.round(4).to_csv(os.path.join(RES, "tech_benchmark_components.csv"), index=False)
    print(Agg.round(3).to_string())

    sample = list(rng.choice(real, 30, replace=False))
    tune_ids = sample[:5]
    (delta, p_d), p_target = tune_delta(tune_ids)
    print("block-DCT delta tuned to", delta, "PSNR", round(p_d, 2), "target", round(p_target, 2))
    BlockDCTScheme.__init__.__defaults__ = (delta,)
    attacks = ["No Attack", "JPEG Q=50", "JPEG Q=20", "Gaussian Noise (var=0.02)", "Median Filter
3x3",
               "Gaussian Blur (sigma=1.5)", "Rotation 5 deg", "Cropping 25%", "Scaling 0.5x"]
    B = part_b(sample, attacks)
    B.to_csv(os.path.join(RES, "tech_benchmark_schemes_raw.csv"), index=False)
    ok = B[B.status == "ok"]
    S = ok.groupby(["scheme", "attack"], sort=False).agg(mean_ber=("ber", "mean"), median_ber=("ber",
"median"),
                                                         mean_nc=("nc", "mean"), psnr_db=("psnr_db",
"mean"),
                                                         t_embed_ms=("t_embed_ms", "median"),
t_extract_ms=("t_extract_ms", "median"),
                                                         ber_le_0_10=("ber", lambda x: (x <=
0.10).mean()), n=("ber", "size")).reset_index()
    S.round(4).to_csv(os.path.join(RES, "tech_benchmark_schemes.csv"), index=False)
    print(S.round(3).to_string())

    env = {"python": sys.version.split()[0], "numpy": np.__version__, "scipy": scipy.__version__,
"scikit-image": skimage.__version__,
           "opencv": cv2.__version__, "Pillow": __import__("PIL").__version__, "PyWavelets":
pywt.__version__,
           "blind-watermark": __import__("importlib.metadata", fromlist=["version"]).version("blind-
watermark"),
           "ImageMagick": (subprocess.run(["convert", "-version"], capture_output=True,
text=True).stdout.splitlines() or [""])[0] if have("convert") else None,
           "ffmpeg": (subprocess.run(["ffmpeg", "-version"], capture_output=True,
text=True).stdout.splitlines() or [""])[0] if have("ffmpeg") else None,
           "cpu_threads_used": 1, "repeats": 9, "warmup": 2, "component_images": comp_ids,
"scheme_images": sample,
           "jpeg_bytes_example": jsz, "blockdct_delta": delta, "blockdct_psnr": p_d,
"project_psnr_tune": p_target}
    json.dump(env, open(os.path.join(RES, "tech_summary.json"), "w"), indent=2, default=str)


if __name__ == "__main__":
    main()
```

4.4 make_diagram.py

Pipeline diagrams. 98 lines.

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch
import os

RESULTS_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "..", "visualizations")


def box(ax, xy, w, h, text, fc="#eaf2f8", ec="#2874a6", fontsize=9.5):
    x, y = xy
    p = FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.02,rounding_size=0.02",
                       fc=fc, ec=ec, linewidth=1.6)
    ax.add_patch(p)
    ax.text(x + w / 2, y + h / 2, text, ha="center", va="center",
            fontsize=fontsize, wrap=True)


def arrow(ax, p1, p2, color="#333333"):
    a = FancyArrowPatch(p1, p2, arrowstyle="-|>", mutation_scale=14,
                        color=color, linewidth=1.5)
    ax.add_patch(a)


def embedding_diagram():
    fig, ax = plt.subplots(figsize=(11, 4.2))
    ax.set_xlim(0, 11); ax.set_ylim(0, 4.2); ax.axis("off")
    ax.set_title("Watermark Embedding Pipeline", fontsize=13, fontweight="bold", pad=16)

    steps = [
        "Host Image\n(512x512, grayscale)",
        "2D DWT\n(Haar, 1-level)\n-> LL sub-band",
        "2-D DCT\nof LL sub-band\n(global)",
        "SVD\nA = U.S.V^T",
        "S' = S + alpha*Sw\n(Sw: singular\nvalues of logo)",
        "Inverse DCT\n+ Inverse DWT",
        "Watermarked\nImage",
    ]
    n = len(steps)
    w, h = 1.35, 1.5
    gap = (11 - n * w) / (n + 1)
    x = gap
    y = 1.1
    centers = []
    for s in steps:
        box(ax, (x, y), w, h, s, fontsize=9.2)
        centers.append((x, x + w))
        x += w + gap
    for i in range(n - 1):
        arrow(ax, (centers[i][1], y + h / 2), (centers[i + 1][0], y + h / 2))

    box(ax, (centers[4][0] - 0.1, y + h + 0.55), 1.5, 0.85, "Watermark\nLogo (64x64)", fc="#fdebd0",
ec="#b9770e", fontsize=9.2)
    arrow(ax, (centers[4][0] + 0.6, y + h + 0.55), (centers[4][0] + 0.6, y + h), color="#b9770e")

    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "diagram_embedding.png"), dpi=170)
    plt.close(fig)


def extraction_diagram():
    fig, ax = plt.subplots(figsize=(11, 4.0))
    ax.set_xlim(0, 11); ax.set_ylim(0, 4.0); ax.axis("off")
    ax.set_title("Watermark Extraction Pipeline (non-blind, key-based)", fontsize=13,
fontweight="bold", pad=16)

    steps = [
        "Received Image\n(possibly attacked:\nJPEG / noise / crop /\nrotate / scale ...)",
        "2D DWT\n-> LL sub-band",
        "2-D DCT\n+ SVD -> S_att",
        "Sw' = (S_att - S_orig)\n/ alpha",
        "Reconstruct:\nUw . Sw' . Vw^T",
        "Extracted\nWatermark",
    ]
```

```
    n = len(steps)
    w, h = 1.55, 1.5
    gap = (11 - n * w) / (n + 1)
    x = gap
    y = 0.9
    centers = []
    for s in steps:
        box(ax, (x, y), w, h, s, fontsize=9.2, fc="#eafaf1", ec="#1e8449")
        centers.append((x, x + w))
        x += w + gap
    for i in range(n - 1):
        arrow(ax, (centers[i][1], y + h / 2), (centers[i + 1][0], y + h / 2))

    box(ax, (centers[3][0] - 0.3, y + h + 0.5), 2.0, 0.75,
        "Embedding Key\n{S_orig, Uw, Vw, alpha, shapes}", fc="#f9ebea", ec="#943126", fontsize=8.6)
    arrow(ax, (centers[3][0] + 0.7, y + h + 0.5), (centers[3][0] + 0.7, y + h), color="#943126")

    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "diagram_extraction.png"), dpi=170)
    plt.close(fig)


if __name__ == "__main__":
    embedding_diagram()
    extraction_diagram()
    print("done")
```

4.5 make_paper_figures.py

Column width figures for the paper. 76 lines.

```
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from PIL import Image

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
RES, TAB, OUT = os.path.join(ROOT, "results"), os.path.join(ROOT, "results", "tables"),
os.path.join(ROOT, "paper", "figs")
os.makedirs(OUT, exist_ok=True)
BLUE, ORANGE, AQUA, GREY, INK = "#2a78d6", "#eb6834", "#1baf7a", "#8a8a85", "#222222"
plt.rcParams.update({"font.family": "DejaVu Serif", "font.size": 7.5, "axes.titlesize": 8,
"axes.labelsize": 7.5, "axes.spines.top": False,
                     "axes.spines.right": False, "axes.grid": True, "grid.color": "#e4e4e0",
"grid.linewidth": .5, "axes.axisbelow": True,
                     "savefig.dpi": 300, "savefig.bbox": "tight", "legend.frameon": False,
"legend.fontsize": 6.5, "xtick.labelsize": 7, "ytick.labelsize": 7})
W = 3.25

rob = pd.read_csv(os.path.join(TAB, "robustness_by_attack.csv"))
sw = pd.read_csv(os.path.join(TAB, "alpha_sweep_summary.csv"))
ctl = pd.read_csv(os.path.join(TAB, "controls_summary.csv"))
sch = pd.read_csv(os.path.join(RES, "tech_benchmark_schemes.csv"))
img = pd.read_csv(os.path.join(RES, "results_images.csv"))

fig, ax = plt.subplots(1, 3, figsize=(W, 1.25))
host = np.asarray(Image.open(os.path.join(ROOT, "data", "original", "image_066.png")), float)
wm = np.asarray(Image.open(os.path.join(ROOT, "outputs", "watermarked", "image_066_wm.png")), float)
for a, im, t, cm, vmax in zip(ax, [host, wm, np.clip(np.abs(host - wm) * 10, 0, 255)], ["original",
"watermarked", "|difference| ×10"], ["gray", "gray", "magma"], [255, 255, 255]):
    a.imshow(im, cmap=cm, vmin=0, vmax=vmax); a.set_title(t, fontsize=7); a.axis("off")
p = img.set_index("image_id").loc["image_066"]
fig.suptitle(f"PSNR {p.psnr_db:.1f} dB, SSIM {p.ssim:.3f}", fontsize=7, y=0.02)
fig.savefig(os.path.join(OUT, "fig1_sample.png")); plt.close(fig)

r = rob.sort_values("pass_rate")
fig, ax = plt.subplots(figsize=(W, 2.9))
ax.barh(r.attack.str.replace("Gaussian ", "Gauss. ").str.replace("Histogram Equalization", "Hist.
equalisation"), r.pass_rate * 100,
        color=[BLUE if v >= .9 else ORANGE for v in r.pass_rate], height=.65)
for i, v in enumerate(r.pass_rate):
    ax.text(min(v * 100 + 2, 101), i, f"{v*100:.0f}", va="center", fontsize=6.5)
ax.set_xlim(0, 112); ax.set_xlabel("images with NC ≥ 0.75 (%), n = 100"); ax.grid(axis="y",
visible=False)
```

```
fig.savefig(os.path.join(OUT, "fig2_passrate.png")); plt.close(fig)

fig, ax = plt.subplots(2, 1, figsize=(W, 3.1), sharex=True)
p0 = sw[sw.attack == "No Attack"]
ax[0].plot(p0.alpha, p0.psnr, color=BLUE, marker="o", ms=3, lw=1.2); ax[0].axhline(30, color=INK,
ls="--", lw=.7)
ax[0].set_ylabel("PSNR (dB)")
cols = {"JPEG Q=50": BLUE, "Gaussian Noise (var=0.02)": ORANGE, "Gaussian Blur (sigma=1.5)": AQUA,
"Rotation 5 deg": GREY}
for an, c in cols.items():
    q = sw[sw.attack == an]
    ax[1].plot(q.alpha, q.pass_rate * 100, color=c, marker="o", ms=2.5, lw=1.1,
label=an.replace("Gaussian ", "Gauss. ").replace(" (var=0.02)", " σ²=0.02").replace(" (sigma=1.5)", "
σ=1.5"))
ax[1].set_ylabel("NC ≥ 0.75 (%)"); ax[1].set_xlabel("embedding strength α"); ax[1].legend(loc="lower
right")
fig.tight_layout(); fig.savefig(os.path.join(OUT, "fig3_alpha.png")); plt.close(fig)

g = ctl.iloc[0]; cc = ctl[ctl.control.str.startswith("c: ")].iloc[0]; ca =
ctl[ctl.control.str.startswith("a:")].iloc[0]; cb = ctl[ctl.control.str.startswith("b:")].iloc[0]
fig, ax = plt.subplots(figsize=(W, 1.9))
labs = ["genuine", "wrong logo\n(owner's key)", "other image\nunmarked", "other image\nwrong key"]
vals = [g.mean_nc, cc.mean_nc, ca.mean_nc, cb.mean_nc]
ax.bar(labs, vals, color=[BLUE, ORANGE, GREY, GREY], width=.55); ax.tick_params(axis="x",
labelsize=6)
for i, v in enumerate(vals):
    ax.text(i, v + .02, f"{v:.3f}", ha="center", fontsize=7)
ax.axhline(0.75, color=INK, ls="--", lw=.7); ax.set_ylim(0, 1.15); ax.set_ylabel("mean NC");
ax.grid(axis="x", visible=False)
fig.savefig(os.path.join(OUT, "fig4_fpp.png")); plt.close(fig)

order = ["Project: global DWT-DCT-SVD", "blind-watermark 0.4.4 (block DWT-DCT-SVD)", "Baseline: 8x8
block DCT", "Baseline: spatial LSB"]
names = ["Project", "blind-watermark", "block DCT", "LSB"]
atts = ["JPEG Q=50", "JPEG Q=20", "Gaussian Noise (var=0.02)", "Gaussian Blur (sigma=1.5)", "Rotation
5 deg", "Cropping 25%", "Scaling 0.5x"]
lab = ["JPEG 50", "JPEG 20", "Noise", "Blur", "Rot. 5°", "Crop", "Scale"]
v = sch.set_index(["scheme", "attack"])["mean_ber"]
fig, ax = plt.subplots(figsize=(W, 2.0))
w = .2
for i, (s, n, c) in enumerate(zip(order, names, [BLUE, ORANGE, AQUA, GREY])):
    ax.bar(np.arange(len(atts)) + (i - 1.5) * w, [v[(s, a)] for a in atts], width=w * .92, color=c,
label=n)
ax.set_xticks(range(len(atts))); ax.set_xticklabels(lab, fontsize=6.5); ax.set_ylabel("mean BER");
ax.legend(ncol=4, loc="upper center", bbox_to_anchor=(.5, 1.18), columnspacing=.8, handlelength=1)
ax.grid(axis="x", visible=False)
fig.savefig(os.path.join(OUT, "fig5_schemes.png")); plt.close(fig)
print("figures ok", os.listdir(OUT))
```

Part 5: Document generators

Scripts that assemble the report, presentation and paper from the results.

5.1 docx_helpers.py

Helper functions for Word documents. 328 lines.

```
from __future__ import annotations

import re

from docx import Document
from docx.enum.section import WD_ORIENT
from docx.enum.table import WD_TABLE_ALIGNMENT
from docx.enum.text import WD_ALIGN_PARAGRAPH, WD_BREAK, WD_TAB_ALIGNMENT, WD_TAB_LEADER
from docx.oxml import OxmlElement
from docx.oxml.ns import qn
from docx.shared import Cm, Pt, RGBColor

NAVY = RGBColor(0x14, 0x21, 0x3D)
TEAL = RGBColor(0x1B, 0x8A, 0x8F)
GREY = RGBColor(0x55, 0x55, 0x55)
BODY_FONT = "Calibri"
CODE_FONT = "Consolas"


def _set_cell_bg(cell, hex_fill):
    tcPr = cell._tc.get_or_add_tcPr()
    shd = OxmlElement("w:shd")
    shd.set(qn("w:val"), "clear")
    shd.set(qn("w:color"), "auto")
    shd.set(qn("w:fill"), hex_fill)
    tcPr.append(shd)


def _set_par_bg(par, hex_fill):
    pPr = par._p.get_or_add_pPr()
    shd = OxmlElement("w:shd")
    shd.set(qn("w:val"), "clear")
    shd.set(qn("w:color"), "auto")
    shd.set(qn("w:fill"), hex_fill)
    pPr.append(shd)


def _field(run, instr):
    for t, txt in (("begin", None), (None, instr), ("separate", None), (None, "1"), ("end", None)):
        if t:
            el = OxmlElement("w:fldChar")
            el.set(qn("w:fldCharType"), t)
            run._r.append(el)
        elif txt == instr:
            it = OxmlElement("w:instrText")
            it.set(qn("xml:space"), "preserve")
            it.text = instr
            run._r.append(it)
        else:
            tt = OxmlElement("w:t")
            tt.text = txt
            run._r.append(tt)


class Report:
    def __init__(self, pages=None):
        self.doc = Document()
        self.pages = pages or {}
        self.headings = []
        self.figures = []
        self.tables = []
        self.h1_no = 0
        self.h2_no = 0
        self.h3_no = 0
        self.fig_no = 0
        self.tab_no = 0
        self.appendix = False
        self._setup()
```

```
    def _setup(self):
        d = self.doc
        s = d.sections[0]
        s.page_height, s.page_width = Cm(29.7), Cm(21.0)
        s.left_margin = s.right_margin = Cm(2.4)
        s.top_margin, s.bottom_margin = Cm(2.3), Cm(2.2)
        s.different_first_page_header_footer = True
        st = d.styles
        n = st["Normal"]
        n.font.name = BODY_FONT
        n.font.size = Pt(11)
        n.element.rPr.rFonts.set(qn("w:eastAsia"), BODY_FONT)
        n.paragraph_format.space_after = Pt(6)
        n.paragraph_format.line_spacing = 1.15
        for name, size, color, before, after in (("Heading 1", 18, NAVY, 18, 8), ("Heading 2", 13.5,
NAVY, 12, 4),
                                                 ("Heading 3", 11.5, TEAL, 8, 3)):
            h = st[name]
            h.font.name = BODY_FONT
            h.font.size = Pt(size)
            h.font.bold = True
            h.font.color.rgb = color
            h.element.rPr.rFonts.set(qn("w:eastAsia"), BODY_FONT)
            h.element.rPr.rFonts.set(qn("w:ascii"), BODY_FONT)
            h.element.rPr.rFonts.set(qn("w:hAnsi"), BODY_FONT)
            h.paragraph_format.space_before = Pt(before)
            h.paragraph_format.space_after = Pt(after)
            h.paragraph_format.keep_with_next = True
        cap = st["Caption"]
        cap.font.name = BODY_FONT
        cap.font.size = Pt(9.5)
        cap.font.italic = True
        cap.font.bold = False
        cap.font.color.rgb = GREY
        cap.paragraph_format.alignment = WD_ALIGN_PARAGRAPH.CENTER
        cap.paragraph_format.space_after = Pt(10)
        fp = s.footer.paragraphs[0]
        fp.alignment = WD_ALIGN_PARAGRAPH.CENTER
        r = fp.add_run()
        r.font.size = Pt(9)
        r.font.color.rgb = GREY
        _field(r, "PAGE")
        s.first_page_header.is_linked_to_previous = False
        s.first_page_footer.is_linked_to_previous = False
        s.first_page_header.paragraphs[0].text = ""
        s.first_page_footer.paragraphs[0].text = ""
        hp = s.header.paragraphs[0]
        hp.alignment = WD_ALIGN_PARAGRAPH.LEFT
        hr = hp.add_run("Digital Image Watermarking - hybrid DWT-DCT-SVD   |   NMIMS MPSTME · Image
and Video Processing")
        hr.font.size = Pt(8.5)
        hr.font.color.rgb = GREY

    def _runs(self, par, text, size=None, color=None, italic=None):
        parts = re.split(r"(\*\*[^*]+\*\*|`[^`]+`|\*[^\s][^*]*\*)", text)
        for p in parts:
            if not p:
                continue
            if p.startswith("**") and p.endswith("**"):
                r = par.add_run(p[2:-2]); r.bold = True
            elif p.startswith("`") and p.endswith("`"):
                r = par.add_run(p[1:-1]); r.font.name = CODE_FONT; r.font.size = Pt((size or 11) -
1.5)
                r._r.rPr.rFonts.set(qn("w:eastAsia"), CODE_FONT)
                r.font.color.rgb = RGBColor(0x8A, 0x1F, 0x1F)
                continue
            elif p.startswith("*") and p.endswith("*") and len(p) > 2:
                r = par.add_run(p[1:-1]); r.italic = True
            else:
                r = par.add_run(p)
            if size:
                r.font.size = Pt(size)
            if color is not None:
                r.font.color.rgb = color
            if italic:
                r.italic = True
        return par
```

```
    def para(self, text, align=None, size=None, space_after=None, keep=False, italic=False,
color=None):
        p = self.doc.add_paragraph()
        self._runs(p, text, size=size, italic=italic, color=color)
        if align == "center":
            p.alignment = WD_ALIGN_PARAGRAPH.CENTER
        elif align == "justify":
            p.alignment = WD_ALIGN_PARAGRAPH.JUSTIFY
        if space_after is not None:
            p.paragraph_format.space_after = Pt(space_after)
        if keep:
            p.paragraph_format.keep_with_next = True
        return p

    def bullets(self, items, style="List Bullet"):
        for it in items:
            p = self.doc.add_paragraph(style=style)
            self._runs(p, it)
            p.paragraph_format.space_after = Pt(2)
        self.doc.paragraphs[-1].paragraph_format.space_after = Pt(8)

    def numbered(self, items):
        for i, it in enumerate(items, 1):
            p = self.doc.add_paragraph()
            p.paragraph_format.left_indent = Cm(0.9)
            p.paragraph_format.first_line_indent = Cm(-0.6)
            p.paragraph_format.space_after = Pt(2)
            p.paragraph_format.tab_stops.add_tab_stop(Cm(0.9))
            self._runs(p, f"{i}.\t{it}")
        self.doc.paragraphs[-1].paragraph_format.space_after = Pt(8)

    def h1(self, text, numbered=True, appendix=None):
        if appendix:
            self.appendix = True
            self.h1_no = appendix if isinstance(appendix, str) else self.h1_no
            num = f"Appendix {appendix}"
            label = f"{num}: {text}"
        elif numbered:
            self.h1_no += 1
            self.h2_no = self.h3_no = 0
            self.fig_no = self.tab_no = 0
            num = str(self.h1_no)
            label = f"{num}. {text}"
        else:
            num, label = "", text
        if appendix:
            self.h2_no = self.h3_no = 0
            self.fig_no = self.tab_no = 0
        p = self.doc.add_heading(label, level=1)
        self.headings.append((1, num, text, label))
        return p

    def h2(self, text):
        self.h2_no += 1
        self.h3_no = 0
        num = f"{self.h1_no}.{self.h2_no}"
        label = f"{num} {text}"
        self.doc.add_heading(label, level=2)
        self.headings.append((2, num, text, label))

    def h3(self, text):
        self.h3_no += 1
        num = f"{self.h1_no}.{self.h2_no}.{self.h3_no}"
        label = f"{num} {text}"
        self.doc.add_heading(label, level=3)
        self.headings.append((3, num, text, label))

    def page_break(self):
        self.doc.add_paragraph().add_run().add_break(WD_BREAK.PAGE)

    def _ref(self):
        return f"{self.h1_no}"

    def figure(self, path, caption, width_cm=15.0):
        self.fig_no += 1
        label = f"Figure {self._ref()}.{self.fig_no}"
        p = self.doc.add_paragraph()
```

```
        p.alignment = WD_ALIGN_PARAGRAPH.CENTER
        p.paragraph_format.keep_with_next = True
        p.paragraph_format.space_after = Pt(2)
        p.add_run().add_picture(path, width=Cm(width_cm))
        c = self.doc.add_paragraph(f"{label}: {caption}", style="Caption")
        self.figures.append((label, caption))
        return label

    def table(self, header, rows, caption, widths=None, size=9, align=None, first_col_bold=False):
        self.tab_no += 1
        label = f"Table {self._ref()}.{self.tab_no}"
        cap = self.doc.add_paragraph(f"{label}: {caption}".replace("`", ""), style="Caption")
        cap.paragraph_format.keep_with_next = True
        cap.paragraph_format.space_after = Pt(3)
        self.tables.append((label, caption))
        t = self.doc.add_table(rows=1, cols=len(header))
        t.style = "Table Grid"
        t.alignment = WD_TABLE_ALIGNMENT.CENTER
        for i, h in enumerate(header):
            c = t.rows[0].cells[i]
            c.text = ""
            _set_cell_bg(c, "14213D")
            r = c.paragraphs[0].add_run(str(h))
            r.bold = True
            r.font.size = Pt(size)
            r.font.color.rgb = RGBColor(0xFF, 0xFF, 0xFF)
            c.paragraphs[0].paragraph_format.space_after = Pt(1)
        trPr = t.rows[0]._tr.get_or_add_trPr()
        th = OxmlElement("w:tblHeader"); th.set(qn("w:val"), "true"); trPr.append(th)
        for ri, row in enumerate(rows):
            cells = t.add_row().cells
            for i, v in enumerate(row):
                cells[i].text = ""
                par = cells[i].paragraphs[0]
                par.paragraph_format.space_after = Pt(1)
                par.paragraph_format.line_spacing = 1.0
                self._runs(par, str(v), size=size)
                if first_col_bold and i == 0:
                    for r in par.runs:
                        r.bold = True
                if align and align[i] == "r":
                    par.alignment = WD_ALIGN_PARAGRAPH.RIGHT
                elif align and align[i] == "c":
                    par.alignment = WD_ALIGN_PARAGRAPH.CENTER
                if ri % 2 == 1:
                    _set_cell_bg(cells[i], "F1F4F9")
        if widths:
            t.autofit = False
            tblPr = t._tbl.tblPr
            lay = OxmlElement("w:tblLayout"); lay.set(qn("w:type"), "fixed"); tblPr.append(lay)
            for i, w in enumerate(widths):
                t.columns[i].width = Cm(w)
            for row in t.rows:
                for i, w in enumerate(widths):
                    row.cells[i].width = Cm(w)
        sp = self.doc.add_paragraph()
        sp.paragraph_format.space_after = Pt(4)
        return label

    def code(self, text, size=8.5):
        for line in text.strip("\n").split("\n"):
            p = self.doc.add_paragraph()
            p.paragraph_format.space_after = Pt(0)
            p.paragraph_format.line_spacing = 1.0
            p.paragraph_format.left_indent = Cm(0.3)
            r = p.add_run(line if line else " ")
            r.font.name = CODE_FONT
            r._r.rPr.rFonts.set(qn("w:eastAsia"), CODE_FONT)
            r.font.size = Pt(size)
            _set_par_bg(p, "F4F4F2")
        self.doc.add_paragraph().paragraph_format.space_after = Pt(2)

    def equation(self, text, number=None):
        p = self.doc.add_paragraph()
        p.alignment = WD_ALIGN_PARAGRAPH.CENTER
        p.paragraph_format.space_after = Pt(6)
        r = p.add_run(text)
        r.font.name = "Cambria Math"
```

```
        r._r.rPr.rFonts.set(qn("w:eastAsia"), "Cambria Math")
        r.font.size = Pt(11.5)
        if number:
            ts = p.paragraph_format.tab_stops
            r2 = p.add_run(f"\t({number})")
            r2.font.size = Pt(10)
            ts.add_tab_stop(Cm(16.2), WD_TAB_ALIGNMENT.RIGHT)

    def callout(self, text, fill="EAF4F4"):
        t = self.doc.add_table(rows=1, cols=1)
        t.alignment = WD_TABLE_ALIGNMENT.CENTER
        c = t.rows[0].cells[0]
        _set_cell_bg(c, fill)
        c.text = ""
        self._runs(c.paragraphs[0], text, size=10.5)
        self.doc.add_paragraph().paragraph_format.space_after = Pt(2)

    def toc_entries(self, entries, title):
        self.doc.add_heading(title, level=1).paragraph_format.page_break_before = True
        for indent, text, key in entries:
            p = self.doc.add_paragraph()
            p.paragraph_format.space_after = Pt(1.5)
            p.paragraph_format.left_indent = Cm(0.6 * indent)
            p.paragraph_format.tab_stops.add_tab_stop(Cm(16.2), WD_TAB_ALIGNMENT.RIGHT,
WD_TAB_LEADER.DOTS)
            r = p.add_run(text + "\t" + str(self.pages.get(key, "0")))
            r.font.size = Pt(10 if indent else 10.5)
            r.bold = indent == 0

    def save(self, path):
        self.doc.save(path)
```

5.2 build_report.py

Builds the project report. 615 lines.

```
from __future__ import annotations

import json
import os
import re
import subprocess
import sys

import pandas as pd

from docx_helpers import Report

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
RES, VIS, TAB = (os.path.join(ROOT, "results"), os.path.join(ROOT, "visualizations"),
os.path.join(ROOT, "results", "tables"))
OUT = os.path.join(ROOT, "report")
os.makedirs(OUT, exist_ok=True)
V = lambda n: os.path.join(VIS, n)

AS = json.load(open(os.path.join(RES, "analysis_summary.json")))
RS = json.load(open(os.path.join(RES, "run_summary.json")))
TS = json.load(open(os.path.join(RES, "tech_summary.json")))
VM = json.load(open(os.path.join(RES, "video_demo_meta.json")))
ROB = pd.read_csv(os.path.join(TAB, "robustness_by_attack.csv"))
FAM = pd.read_csv(os.path.join(TAB, "robustness_by_family.csv"))
SWP = pd.read_csv(os.path.join(TAB, "alpha_sweep_summary.csv"))
CTL = pd.read_csv(os.path.join(TAB, "controls_summary.csv"))
COR = pd.read_csv(os.path.join(TAB, "nc_vs_image_statistics.csv"))
CATF = pd.read_csv(os.path.join(TAB, "nc_by_category_family.csv"))
PRO = pd.read_csv(os.path.join(TAB, "protocol_comparison.csv"))
LEG = pd.read_csv(os.path.join(TAB, "legacy_reproduction_cameraman.csv"))
TCOMP = pd.read_csv(os.path.join(RES, "tech_benchmark_components.csv"))
TSCH = pd.read_csv(os.path.join(RES, "tech_benchmark_schemes.csv"))
VID = pd.read_csv(os.path.join(RES, "video_demo_results.csv"))
META = pd.read_csv(os.path.join(ROOT, "data", "metadata.csv"))
IMG = pd.read_csv(os.path.join(RES, "results_images.csv"))
E1, E2, E4 = AS["E1"], AS["E2"], AS["E4"]
BASE = AS["metric_baselines"]
rob = ROB.set_index("attack")
ctl = CTL.set_index("control")
```

```
def f3(x): return f"{x:.3f}"
def pc(x, d=0): return f"{100 * x:.{d}f} %"


def genuine(): return CTL.iloc[0]
def ctl_row(prefix): return CTL[CTL.control.str.startswith(prefix)].iloc[0]


G, CA, CB, CC, CCR, CD = (genuine(), ctl_row("a:"), ctl_row("b:"), ctl_row("c: "), ctl_row("c-ref"),
ctl_row("d:"))
sw = SWP.set_index(["attack", "alpha"])


def sweep(attack, alpha, col): return float(sw.loc[(attack, alpha), col])


NREAL = int((META.is_synthetic == 0).sum())
NSYN = int((META.is_synthetic == 1).sum())
vsch = TSCH.set_index(["scheme", "attack"])
PROJ, BWM, BDCT, LSB = ("Project: global DWT-DCT-SVD", "blind-watermark 0.4.4 (block DWT-DCT-SVD)",
                        "Baseline: 8x8 block DCT", "Baseline: spatial LSB")


def ber(s, a): return float(vsch.loc[(s, a), "mean_ber"])
def psnr_s(s): return float(TSCH[TSCH.scheme == s].psnr_db.mean())
def tms(s, col): return float(TSCH[TSCH.scheme == s][col].median())


def content(r: Report):
    r.h1("Introduction")
    r.h2("Background and motivation")
    r.para("Digital images and videos are trivially copied, redistributed and repurposed.
Photographers, medical institutions, publishers and manufacturers therefore need a way of embedding
an invisible, verifiable ownership signature in their visual content without degrading it. Digital
watermarking hides auxiliary information directly in the pixel or transform-domain data of the host
media [17][32].")
    r.para("Two broad families exist. Spatial-domain methods, such as least-significant-bit (LSB)
substitution [10], are simple but fragile. Transform-domain methods (DCT, DWT, SVD and their hybrids)
spread the watermark over frequency components and are considerably more robust to compression and
everyday signal processing [1][2][3]. This project implements a hybrid DWT-DCT-SVD scheme and
evaluates it; the present report extends the first submission (two test images) with a 100-image
evaluation, controlled experiments, a comparison with existing technology and a verified literature
review.")
    r.h2("Problem statement")
    r.para("Design an invisible digital image watermarking system that (a) introduces minimal,
imperceptible distortion, (b) survives common real-world distortions such as compression, noise,
filtering and geometric change, and (c) is fully explainable through closed-form linear-algebraic
operations, with no learned component - and **establish, with measurements, how far each of these
properties actually holds**.")
    r.h2("Objectives")
    r.numbered([
        "Implement a hybrid DWT-DCT-SVD embedding and extraction pipeline from first principles
(original objective).",
        "Quantify imperceptibility (PSNR, SSIM) and robustness (NC, BER, and the added ZNCC) under a
suite of 14 attacks (plus an unattacked reference) on **100 unique images** (extended objective).",
        "Characterise the imperceptibility-robustness trade-off as a function of the embedding
strength α.",
        "Identify and explain failure modes, including whether a high NC really proves that *this*
watermark is present (control experiments).",
        "Compare the implementation with established libraries and with an existing open-source
watermarking package, using controlled benchmarks.",
        "Test, as an exploratory extension, whether the scheme can be applied frame by frame to real
video that is re-encoded with lossy codecs.",
    ])
    r.h2("Scope and boundaries")
    r.bullets([
        "8-bit **grey-scale** images; colour is handled only by converting to luminance (ITU-R
BT.601) during dataset preparation.",
        "No machine learning or deep learning at any stage (design constraint of the project);
learned watermarking is discussed only as context.",
        "The scheme is **non-blind**: extraction needs a key that contains the original singular
values.",
        "Video is an **exploratory** test (60 frames from each of two clips); temporal attacks,
colour video and hardware codecs are out of scope.",
        "Only operations present in the project's code are documented as transformations; edge
detection, CLAHE, morphology, perspective warps etc. are not part of this project.",
    ])
```

```
    r.h2("Applications")
    r.para("Ownership marking of photographs, medical-image provenance, tracing of unauthorised
redistribution of publications, and certification of technical imagery. The measurements in Chapter 8
show which of these uses the scheme can support (robust against re-compression and filtering) and
which it cannot (geometric cropping; proof of ownership without additional safeguards).")
    r.h2("Expected outcomes and main contributions")
    r.bullets([
        "A reproducible, tested implementation (43 unit/integration tests, seeded experiments,
config-driven batch runner).",
        f"A category-stratified evaluation set of {len(META)} unique images ({NREAL} real, {NSYN}
clearly labelled synthetic) and 3000 attack trials (+4000 sweep and 1600 control rows) with full
status logging.",
        "**A measured false-positive problem:** a different logo decoded with the owner's key still
yields NC "
        f"{f3(CC.mean_nc)} (genuine {f3(G.mean_nc)}), so NC alone cannot prove ownership.",
        "Corrections to the first submission (global rather than block DCT, non-blind rather than
semi-blind, an α-trade-off figure that exaggerated a 0.05 change, unreproducible noise attacks) -
Appendix F.",
        "A controlled comparison with PyWavelets/SciPy/NumPy/OpenCV/Pillow/scikit-image/ImageMagick
implementations and with the `blind-watermark` package, showing where the project is and is not
competitive.",
    ])

    r.h1("Literature Review")
    r.para("This chapter summarises the literature relevant to the techniques the project actually
uses. The first submission cited Cox et al. [1], Liu and Tan [2], Ganic and Eskicioglu [3], Gonzalez
and Woods [31] and library documentation; it did not state that code was derived from them, so they
are treated here as *foundational literature identified during this review*. Bibliographic details
were checked as recorded in the reference list (status flags in `research/REFERENCES.md`).")
    r.h2("Development of the techniques")
    r.para("**Spatial domain.** Van Schyndel, Tirkel and Osborne [10] are the usual starting point
for digital image watermarking in the spatial domain. LSB embedding offers high capacity and almost
no distortion but no robustness; in our own baseline (Chapter 8) a 4095-bit LSB payload reaches "
           f"{psnr_s(LSB):.1f} dB PSNR yet has a bit-error rate of {ber(LSB, 'JPEG Q=50'):.2f} after
JPEG Q=50 and {ber(LSB, 'Gaussian Noise (var=0.02)'):.2f} after Gaussian noise (chance level is
0.5).")
    r.para("**Spread spectrum in a transform domain.** Cox, Kilian, Leighton and Shamoon [1] argued
that a watermark must be placed in the perceptually significant spectral components if it is to
survive processing; a pseudo-random Gaussian sequence is inserted into the most significant spectral
components and detected by correlation. This established the imperceptibility-robustness trade-off
studied here through α. Block-DCT methods such as Koch and Zhao [11] tie the embedding to the JPEG
model; surveys are given by Hartung and Kutter [17] and Cox *et al.* [32].")
    r.para("**Wavelets.** Mallat's multiresolution theory [12] gives the DWT used here. Embedding in
the approximation (LL) band trades visibility for robustness; embedding in detail bands does the
opposite.")
    r.para("**Singular value decomposition.** Liu and Tan [2] introduced SVD-based ownership
watermarking; Ganic and Eskicioglu [3] combined DWT and SVD and embedded in all sub-bands; Sverdlov,
Dexter and Eskicioglu [6] did the same with the DCT; Lai and Tsai [7] proposed a DWT-SVD scheme.
Hybrids that cascade DWT, DCT and SVD appear in later work, for example Bekkouch and Faraoun [8]
(reversible, two watermarks) and Begum, Ferdush and Uddin [9] (blind, Arnold-map scrambling). These
authors report better robustness or imperceptibility than earlier schemes; **we did not re-evaluate
those claims**.")
    r.h2("The false-positive problem")
    r.para("Zhang and Li [4] published a comment on Liu and Tan [2], and Ting [5] studied ambiguity
attacks on the Ganic-Eskicioglu scheme [3]. Review literature found during the search describes the
common cause: if the watermark's singular values are embedded and its singular vectors are kept as
the key, the detector reconstructs the key owner's watermark regardless of what was embedded. This
report contributes a direct measurement of the effect for this implementation (Section 8.6).")
    r.h2("Geometric attacks, video and learned approaches")
    r.para("Lin *et al.* [22] explain that rotation, scale and translation defeat correlation
detectors unless the detection domain is made invariant (log-polar Fourier representation); our
rotation and cropping results are consistent with that. Hartung and Girod [23] treat watermarking of
uncompressed and compressed video. Learned watermarking (HiDDeN [29], StegaStamp [30]) trains
encoder/decoder networks against a distribution of distortions; the project excludes it by design,
and no experiment here compares against it.")
    r.h2("Paper-to-implementation mapping")
    r.table(["Ref.", "Technique", "Project function / experiment", "Relationship"], [
        ["[1] Cox et al.", "embedding in significant coefficients; α trade-off", "`embed_watermark`
(α); E3", "conceptual foundation"],
        ["[2] Liu & Tan", "singular-value embedding", "`embed_watermark` step 5", "closest ancestor
(the project adds the logo's singular values)"],
        ["[3] Ganic & Eskicioglu", "DWT-SVD hybrid", "`embed_watermark` (DWT + SVD)", "conceptual
foundation"],
        ["[4][5]", "false-positive / ambiguity critique", "experiment E4", "motivates the controls"],
        ["[10][11]", "LSB; block-DCT", "baselines in `benchmark_tech.py`", "comparison"],
        ["[12] Mallat", "DWT theory", "`pywt.dwt2`", "theory"],
        ["[13][14]", "DCT; JPEG", "`dct2`; `jpeg_compression`", "theory / attack"],
        ["[16] Wang et al.", "SSIM", "`metrics.ssim`", "metric"],
```

```
        ["[22] Lin et al.", "RST-resilient detection", "rotation, cropping, scaling attacks",
"explains geometric fragility"],
        ["[23] Hartung & Girod", "video watermarking", "`video_demo.py`", "related work"],
        ["[28] blind-watermark", "block DWT-DCT-SVD, blind", "`BlindWMScheme`", "measured
comparison"],
    ], "Mapping of literature to the implementation.", widths=[3.0, 4.6, 4.6, 4.4], size=8.5)
    r.h2("Research gaps and trade-offs relevant to this project")
    r.numbered([
        "**Identity binding:** the scheme does not bind the logo to the image (Section 8.6).",
        "**Geometry:** global SVD embedding is geometry-fragile; block-based or invariant-domain
embedding [22] is the standard remedy.",
        f"**Metric reliability:** plain NC is dominated by the logo's first singular value - a blank
white image already scores NC = {BASE['nc_constant_white_image']:.3f} and a rank-1 approximation
{BASE['nc_rank1_approximation']:.3f} for this logo.",
        "**Evaluation breadth:** hybrid schemes in [7]-[9] are, to our reading of their abstracts,
evaluated on small standard-image sets; a 100-image stratified evaluation is a contribution of this
work.",
    ])
    r.para("The text of this chapter was written from verified titles, abstracts and reference-list
entries; where only secondary descriptions were available this is stated.", italic=True, size=9.5)

    r.h1("Existing Technology and Problem Analysis")
    r.h2("Existing tools and approaches")
    r.para("The project is an algorithm built on libraries; it is not a general-purpose imaging
toolkit. Relevant technologies are NumPy/SciPy (SVD, DCT, filters), PyWavelets (DWT), scikit-image
(geometry, metrics), Pillow (JPEG), OpenCV (DCT, SVD, filters, video I/O), ImageMagick (command-line
conversion), FFmpeg (video codecs), and at scheme level the `blind-watermark` package [28]. Machine-
learning watermarkers [29][30] are out of scope.")
    r.h2("Feature comparison")
    r.table(["Capability", "NumPy", "SciPy", "PyWavelets", "scikit-image", "Pillow", "OpenCV",
"ImageMagick", "FFmpeg"], [
        ["2-D DCT", "-", "✓", "-", "-", "-", "✓", "-", "-"],
        ["SVD", "✓", "✓", "-", "-", "-", "✓", "-", "-"],
        ["DWT", "-", "-", "✓", "(via PyWavelets)", "-", "-", "-", "-"],
        ["JPEG codec", "-", "-", "-", "(I/O plug-ins)", "✓", "✓", "✓", "MJPEG"],
        ["filters, rotate, resize", "-", "✓", "-", "✓", "✓", "✓", "✓", "✓"],
        ["PSNR / SSIM", "-", "-", "-", "✓ / ✓", "-", "PSNR (SSIM: contrib)", "`compare`", "-"],
        ["video decode/encode", "-", "-", "-", "-", "-", "✓", "-", "✓"],
        ["licence (metadata)", "BSD-3", "BSD", "MIT/BSD-3", "BSD", "MIT-CMU", "Apache-2.0", "IM
licence", "LGPL/GPL"],
    ], "Capability matrix (documentation-based; not benchmarked).", widths=[3.0, 1.4, 1.4, 1.9, 2.0,
1.4, 2.2, 1.9, 1.5], size=8)
    r.h2("Limitations relevant to the project")
    r.para("No single library covers DWT + DCT + SVD + JPEG + metrics, so a combination is necessary;
PyWavelets supplies the DWT that OpenCV lacks. An existing blind watermarking package exists [28] and
outperforms the project in several attacks (Chapter 8), so novelty cannot be claimed on robustness.
The first submission's central design argument - hybrids beat single transforms - was taken from the
literature and is not independently tested here.")
    r.h2("Justification for the chosen methods")
    r.bullets([
        "The singular-value scheme is closed-form and explainable, in line with the project's
constraint of no learned components.",
        "PyWavelets + SciPy + NumPy + Pillow + scikit-image give numerically agreeing results (≤
1.5e-16 relative for DCT/SVD; Section 8.8) and the whole 100-image experiment runs in about 80 s on
four cores, so a change of library is not justified on performance grounds.",
        "The Haar wavelet and global DCT were retained from the first submission to preserve its
functionality and allow direct comparison.",
    ])

    r.h1("System Requirements and Architecture")
    r.h2("Functional requirements")
    r.table(["ID", "Requirement", "Where satisfied"], [
        ["FR1", "Load any common image file safely (colour, alpha, 16-bit, odd sizes) and standardise
it", "`image_io.load_gray_square`"],
        ["FR2", "Embed a binary logo into a grey image with a single strength parameter α",
"`embed_watermark`"],
        ["FR3", "Extract the logo from a possibly attacked image using the key",
"`extract_watermark`"],
        ["FR4", "Apply a reproducible suite of 14 attacks plus an unattacked reference",
"`attacks.make_attack_suite`"],
        ["FR5", "Compute PSNR, SSIM, NC, ZNCC, BER", "`metrics.py`"],
        ["FR6", "Process 100 images in batch, record status, time and parameters per operation,
validate outputs", "`run_batch_experiment.py`"],
        ["FR7", "Produce tables and figures from recorded results only", "`analysis.py`"],
        ["FR8", "Apply the scheme to video frames (exploratory)", "`video_demo.py`"],
    ], "Functional requirements.", widths=[1.2, 10.0, 5.4], size=9)
    r.h2("Non-functional requirements")
    r.bullets([
```

```
        "**Reproducibility:** all stochastic steps seeded; two complete runs gave identical metrics
(maximum absolute difference 0.0).",
        "**Robustness of the tooling:** a failing image or attack is recorded, not fatal; corrupt,
empty and unsupported files raise `ImageLoadError`.",
        "**Performance:** a 512×512 image is embedded in about 37 ms and extracted in about 20 ms on
one CPU core; peak resident memory of a worker is "
        f"{RS['peak_rss_mb_max_over_workers']:.0f} MB.",
        "**Portability:** pure Python with NumPy/SciPy stack; no GPU required.",
    ])
    r.h2("Hardware and software requirements")
    r.para(f"Tested on a 4-vCPU Intel Xeon @ 2.1 GHz with 15 GB RAM, Linux, Python {RS['environment']
['python']}, NumPy {RS['environment']['numpy']}, SciPy {RS['environment']['scipy']}, scikit-image
{RS['environment']['scikit-image']}, Pillow {RS['environment']['Pillow']}, Pandas {RS['environment']
['pandas']}. Optional: OpenCV 5.0.0 and FFmpeg 6.1.1 (video test), ImageMagick 6.9.12 (one benchmark
row), `blind-watermark` 0.4.4 (comparison).")
    r.h2("System architecture and data flow")
    r.para("Two pipelines share one key: an embedding pipeline run once by the owner and an
extraction pipeline run whenever ownership is to be verified (Figures 4.1 and 4.2). In the batch
experiment the watermarked image is rounded to 8 bit, attacked, and then passed to the extractor.")
    r.figure(V("diagram_embedding.png"), "Watermark embedding pipeline (global 2-D DCT of the LL sub-
band; the first submission's \"block DCT\" label was wrong).", 15.5)
    r.figure(V("diagram_extraction.png"), "Watermark extraction pipeline (non-blind, key-based).",
15.5)
    r.h2("Module description")
    r.table(["Module", "Responsibility"], [
        ["`watermark_core.py`", "DWT + DCT + SVD embedding / extraction (numerics unchanged from the
first submission, validation added)"],
        ["`metrics.py`", "MSE, PSNR, SSIM, NC, ZNCC, BER with shape checks"],
        ["`attacks.py`", "14 attacks + reference; `make_attack_suite(seed)`"],
        ["`logo.py`", "procedural 64×64 binary logo"],
        ["`image_io.py`", "safe decoding, standardisation, PNG verification"],
        ["`dataset_manifest.py`, `build_dataset.py`", "sources, licences, near-duplicate rejection,
synthetic controls, `metadata.csv`"],
        ["`run_batch_experiment.py`", "E1-E4 with multiprocessing, per-operation status"],
        ["`analysis.py`, `analysis_tech.py`", "statistics, tables, figures"],
        ["`benchmark_tech.py`", "component and scheme benchmarks"],
        ["`video_demo.py`", "exploratory video test"],
        ["`run_experiment.py`, `make_diagram.py`", "legacy two-image experiment, architecture
diagrams"],
    ], "Software modules (`source_code/src`).", widths=[5.4, 11.2], size=9)
    r.h2("Input and output specification")
    r.bullets([
        "**Input:** any decodable image (PNG/JPEG/…); standardised to a 512×512 8-bit grey array.
Watermark: 2-D array, any size (64×64 binary logo used).",
        "**Output of embedding:** watermarked 512×512 float array in [0,255] (stored as 8-bit PNG)
and a key holding the original singular values (≈ 2.1 MB per image including the unused `U`, `Vᵗ`).",
        "**Output of extraction:** a 64×64 grey image; NC/ZNCC/BER against the logo.",
    ])

    r.h1("Methodology and Algorithms")
    r.h2("Dataset preparation and preprocessing")
    r.para("Every source image is decoded with Pillow, alpha is composited over white, 16-bit data is
rescaled linearly, colour is converted to grey with the BT.601 luma `L = 0.299R + 0.587G + 0.114B`,
the largest centred square is cropped (no aspect distortion), and the result is resampled to 512×512
with Lanczos. The 100-image set and its provenance are described in Chapter 7.")
    r.h2("Mathematical formulation of the scheme")
    r.para("**Haar DWT.** A one-level 2-D Haar transform splits the host H into four sub-bands; for
each 2×2 block (a, b; c, d) the approximation is")
    r.equation("LL = (a + b + c + d) / 2", "1")
    r.para("and the three detail bands are the corresponding sums and differences. The watermark is
embedded in LL (256×256).")
    r.para("**2-D DCT (global).** The orthonormal DCT-II is applied to the **whole** LL band:")
    r.equation("F(u,v) = α(u) α(v) Σₓ Σᵧ f(x,y) cos[(2x+1)uπ / 2M] cos[(2y+1)vπ / 2N]", "2")
    r.para("with α(0) = √(1/M) and α(u>0) = √(2/M). **SVD.** The DCT matrix and the (resized) logo
are factorised:")
    r.equation("A = U · diag(S) · Vᵀ ,        W = U_w · diag(S_w) · V_wᵀ", "3")
    r.para("**Embedding rule.** With embedding strength α (0.10 in the main experiment):")
    r.equation("S′ = S + α · S_w ,        A′ = U · diag(S′) · Vᵀ", "4")
    r.para("followed by the inverse DCT, the inverse DWT and clipping to [0,255]. **Extraction
rule.** From the received image the same DWT-DCT-SVD gives S_att, and")
    r.equation("S_w′ = (S_att - S_orig) / α ,        W′ = U_w · diag(S_w′) · V_wᵀ", "5")
    r.para("The key therefore contains S_orig, U_w, V_wᵀ, α and the shapes; U and Vᵀ of the host are
stored but not used by the extractor. Because S_orig is derived from the unmarked host the scheme is
**non-blind**, and because the logo's singular vectors are in the key the extractor outputs the key
owner's logo whatever singular values it finds (Section 8.6).")
    r.h2("Pseudocode")
    r.code("""EMBED(host, logo, alpha):
    LL, details = haar_dwt2(host)
```

```
    A  = dct2(LL)
    U, S, Vt   = svd(A)
    Wr = resize(logo, shape(LL))
    Uw, Sw, Vwt = svd(Wr)
    S' = S + alpha * Sw
    LL' = idct2(U @ diag(S') @ Vt)
    out = clip(inverse_haar(LL', details), 0, 255)
    key = {S, Uw, Vwt, alpha, shapes}
    return out, key

EXTRACT(image, key):
    LL, _ = haar_dwt2(image);  if shape(LL) != key.shape: LL = resize(LL, key.shape)
    S_att = svd_values(dct2(LL))
    Sw'   = (S_att - key.S) / key.alpha
    W'    = key.Uw @ diag(Sw') @ key.Vwt
    return clip(resize(W', logo_shape), 0, 255)""")
    r.h2("Attack suite")
    r.table(["Family", "Attack (parameters)", "Implementation"], [
        ["compression", "JPEG quality 90, 50, 20", "Pillow, baseline JPEG, floats rounded to 8 bit
first"],
        ["noise", "Gaussian σ²=0.005 and 0.02 (on [0,1]; σ≈18, 36 grey levels); salt-and-pepper 2 %",
"scikit-image `random_noise`, seeded"],
        ["filtering", "median 3×3; Gaussian blur σ=1.5; unsharp mask (σ=1, amount 1)", "SciPy
`ndimage`"],
        ["geometric", "rotation 5° and 15° (centre, bilinear, edge replicate); border zeroing with 25
% of each dimension removed (**43.75 % of the area**); down/up-scaling 0.5×", "scikit-image `rotate`,
`resize`; NumPy"],
        ["global intensity", "histogram equalisation", "scikit-image `equalize_hist`"],
    ], "The attack suite: 14 attacks plus the unattacked reference \"No Attack\" (15 conditions;
identical to the first submission, now seeded). The first report called all 15 \"attacks\".",
widths=[2.6, 8.4, 5.6], size=9)
    r.h2("Evaluation metrics")
    r.equation("PSNR = 10 log₁₀ (255² / MSE) ,     NC = Σ W·W′ / (‖W‖ ‖W′‖) ,     ZNCC = NC of the
mean-centred images", "6")
    r.para("SSIM [16] is used with scikit-image's defaults (7×7 window). BER is the fraction of logo
pixels that differ after thresholding at 127.5. **ZNCC is new:** plain NC is not mean-centred, so any
bright image already scores NC = "
           f"{BASE['nc_constant_white_image']:.3f} against this logo (the square root of its
{pc(BASE['logo_fill_ratio'], 1)} white fraction); ZNCC removes that offset. The 0.75 recognisability
threshold is a convention of the project that we keep for comparability; we could not trace it to a
verified literature source.")
    r.h2("Video workflow (exploratory)")
    r.para("Frames are read with OpenCV, standardised like still images, watermarked individually
(each frame has its own key because the scheme is non-blind), encoded by FFmpeg (FFV1 lossless, H.264
at CRF 18/23/28, MJPEG at q 5/15), decoded, and extracted frame by frame. Temporal redundancy is not
exploited.")
    r.h2("Parameter selection and implementation decisions")
    r.bullets([
        "**α = 0.10** is kept from the first submission. It gives about 36 dB PSNR and, at that
value, 100 % pass rate under JPEG Q=50; Section 8.5 shows how the pass rate under other attacks
changes with α.",
        "**Protocol:** the watermarked image is rounded to 8 bit before attacking (primary), because
that is what a saved file contains; the first submission's float protocol is reported as a check.",
        "**Seeds:** attacks use `20260713 + 100·index (+ offset)`; control images are drawn with the
same base seed.",
    ])
    r.h2("Summary of implemented transformations")
    r.table(["#", "Transformation", "Function", "Library"], [
        ["T1", "grey conversion, crop, Lanczos resize", "`load_gray_square`", "Pillow"],
        ["T2", "1-level Haar DWT and inverse", "`embed/extract_watermark`", "PyWavelets"],
        ["T3", "2-D DCT-II and inverse (global)", "`dct2`, `idct2`", "SciPy fftpack"],
        ["T4", "SVD of host sub-band and logo", "`embed_watermark`", "NumPy"],
        ["T5", "singular-value embedding / extraction", "`embed/extract_watermark`", "NumPy"],
        ["T6", "anti-aliased resize (logo ↔ sub-band)", "inside T5", "scikit-image"],
        ["T7", "procedural binary logo", "`make_watermark_logo`", "Pillow"],
        ["T8-T17", "JPEG, Gaussian & salt-and-pepper noise, median, blur, rotation, border cropping,
scaling, histogram equalisation, sharpening", "`attacks.py`", "Pillow, scikit-image, SciPy"],
        ["T18-T19", "PSNR, SSIM, MSE, NC, ZNCC, BER", "`metrics.py`", "scikit-image, NumPy"],
        ["T20", "frame extraction, per-frame embedding, codec round-trip", "`video_demo.py`",
"OpenCV, FFmpeg"],
    ], "Transformation summary (details, parameters, limitations and complexity:
`documentation/TRANSFORMATIONS_REFERENCE.md`).", widths=[1.6, 7.6, 4.2, 3.2], size=9)

    r.h1("Implementation")
    r.h2("Software structure")
    r.para("The repository keeps the first submission's four modules and adds loaders, a dataset
builder, a batch runner, analysis and benchmark scripts (Table 4.2). Original files are preserved in
`source_code/original_backup/`; every change is listed in `documentation/CHANGELOG.md`.")
```

```
    r.h2("Key functions and code excerpt")
    r.para("The embedding core is unchanged from the first submission (excerpt of
`watermark_core.py`):")
    r.code("""LL, (LH, HL, HH) = pywt.dwt2(host, wavelet)
wm_resized = resize(watermark, target.shape, anti_aliasing=True, preserve_range=True)
target_dct = dct2(target)
U, S, Vt   = np.linalg.svd(target_dct, full_matrices=False)
Uw, Sw, Vtw = np.linalg.svd(wm_resized, full_matrices=False)
S_mod = S + alpha * Sw
target_mod = idct2((U * S_mod) @ Vt)""")
    r.h2("Library usage")
    r.para("NumPy (SVD, array algebra), SciPy (`fftpack.dct`, `ndimage` filters), PyWavelets
(`dwt2`/`idwt2`), scikit-image (resize, rotate, noise, equalisation, PSNR/SSIM), Pillow (decoding,
JPEG), Pandas/Matplotlib (tables, figures), OpenCV and FFmpeg (video test), python-docx/python-pptx
(report and deck builders).")
    r.h2("Execution workflow and error handling")
    r.para("`build_dataset.py` → `run_batch_experiment.py` → `analysis.py` (→ `benchmark_tech.py`,
`video_demo.py`). Each attack/extraction is wrapped so that an exception is stored as `status =
error` with text; image-level failures are logged and the run continues. Outputs are re-opened and
verified (shape, dtype, finiteness). The run summary records "
           f"{RS['images_ok']}/{RS['images_requested']} images,
{RS['attack_trials_ok']}/{RS['attack_trials_total']} attack trials and
{RS['alpha_sweep_rows_ok']}/{RS['alpha_sweep_rows_total']} sweep rows without failure.")
    r.h2("Design decisions")
    r.bullets([
        "Numerical behaviour of the original embed/extract code was **not** modified, so the first
submission's numbers remain reproducible (Appendix E).",
        "Input validation rejects non-2-D, tiny, NaN/Inf or non-numeric input and α ≤ 0.",
        "A variant that embeds the logo's principal component was tried and **rejected**: because the
key then contains the whole logo, an unwatermarked image still decodes at NC = 0.98
(`results/exp_principal_component_variant.txt`).",
        "Per-image parallelism with one BLAS thread per process keeps timings comparable.",
    ])
    r.h2("Tests")
    r.para("43 tests (pytest) cover DCT round trips, embedding shape/range/key, clean recovery,
monotone PSNR in α, odd sizes, resized input, invalid inputs, metric definitions and boundary values,
attack seeding and the cropped area, image loading of
RGB/RGBA/16-bit/bilevel/palette/corrupt/empty/unsupported files, and completeness of the dataset and
results. All 43 pass (`results/test_results.txt`).")

    r.h1("Experimental Setup and Evaluation")
    r.h2("The 100-image dataset")
    cat = META.groupby("category").size().sort_values(ascending=False)
    r.para(f"The set contains {len(META)} unique 512×512 grey images: {NREAL} real images from open
GitHub-hosted sample collections and {NSYN} synthetic control images (gradient, radial gradient,
grating, white noise, low-contrast noise, checkerboard) that are labelled as synthetic everywhere.
Brightness classes: " +
           ", ".join(f"{k} {v}" for k, v in META.brightness_class.value_counts().items()) +
           "; contrast classes: " + ", ".join(f"{k} {v}" for k, v in
META.contrast_class.value_counts().items()) +
           ". Near-duplicates were rejected by a 64-bit difference hash and uniqueness is asserted by
a test. Sources, licences and exclusions are documented in `data/README.md`; per-image provenance is
in `data/metadata.csv`.")
    r.figure(V("dataset_contact_sheet.png"), "All 100 images (standardised grey, 512×512), labelled
with their IDs and source keys.", 15.5)
    half = (len(cat) + 1) // 2
    rows = [[k, int(v)] for k, v in cat.items()]
    rows2 = [[rows[i][0], rows[i][1], rows[i + half][0] if i + half < len(rows) else "", rows[i +
half][1] if i + half < len(rows) else ""] for i in range(half)]
    r.table(["category", "images", "category", "images"], rows2, "Composition of the dataset by
category.", widths=[5.0, 2.0, 5.0, 2.0], size=9)
    r.para("**Provenance and rights.** Hosting-repository licences (Apache-2.0, BSD, MIT,
HPND, \"YOLO\") are recorded per image, but the repository licence is not a statement about each
photograph. Where the upstream project states a licence for an image (CC0 images from scikit-image,
NASA public domain, CC BY 2.0 images from scikit-learn) it is quoted in the metadata; for most others
nothing is documented upstream, which is stated. The images are used only for non-commercial academic
evaluation. Only `raw.githubusercontent.com` and PyPI were reachable when the set was built, which
limits the choice of sources.")
    r.h2("Hardware and software environment")
    r.para(f"4-vCPU Intel Xeon @ 2.1 GHz, 15 GB RAM, Linux 6.18, Python {RS['environment']
['python']}; NumPy {RS['environment']['numpy']}, SciPy {RS['environment']['scipy']}, scikit-image
{RS['environment']['scikit-image']}, Pillow {RS['environment']['Pillow']}, PyWavelets (module
attribute 1.8.0, distribution metadata 1.10.0), OpenCV {TS['opencv']}, FFmpeg
{TS['ffmpeg'].split('version ')[1].split(' Copyright')[0].split('-')[0]}, ImageMagick
{TS['ImageMagick'].split('ImageMagick ')[1].split(' ')[0]}. No GPU.")
    r.h2("Experimental procedure")
    r.table(["ID", "Experiment", "Size", "Recorded"], [
```

```
        ["E1", "embed at α = 0.10, save 8-bit PNG, re-open and verify", "100 images", "PSNR, SSIM,
differences, time, key size"],
        ["E2", "15 conditions (14 attacks + reference) × 2 protocols (uint8 primary, float check)",
"3000 trials", "NC, ZNCC, BER, times, status"],
        ["E3", "α sweep (8 values) × {no attack, JPEG Q=50, Gaussian noise 0.02, blur 1.5, rotation
5°}", "4000 rows", "PSNR, SSIM, NC, ZNCC, BER"],
        ["E4", "false-positive controls (a-d, genuine reference)", "1600 rows", "NC, ZNCC, BER"],
        ["S1", "scheme comparison: project vs `blind-watermark` vs block-DCT vs LSB", "30 images × 9
attacks × 4", "PSNR, BER, NC, time"],
        ["S2", "component benchmark (10 operations, several libraries)", "3 images, 9 repeats",
"median time, agreement"],
        ["V1", "video: 2 clips × 60 frames × 6 codec settings", "720 decoded frames", "PSNR, NC, BER,
bit-rate, time"],
    ], "Experiments.", widths=[1.1, 8.0, 3.2, 4.3], size=9)
    r.h2("Evaluation metrics and their limits")
    r.para("PSNR and SSIM compare the watermarked image with the *unwatermarked original* and are
appropriate for imperceptibility. NC, ZNCC and BER compare the extracted logo with the original logo
and are **conditional on the key**; they do not show that the logo was embedded (Section 8.6). A
higher metric is not automatically better: PSNR is nearly host-independent here, SSIM can collapse on
near-constant images, and edge-detection style metrics do not apply because the project has no edge-
detection operation.")
    r.h2("Statistical methods")
    r.para("Means with 95 % bootstrap confidence intervals (2000 resamples), pass rates with Wilson
95 % intervals, Spearman rank correlations (28 p-values, not corrected for multiplicity), and Mann-
Whitney AUC for genuine versus control NC. The 94 real images are a convenience sample, so intervals
describe sampling variability within this set, not all photographs.")
    r.h2("Benchmarking methodology")
    r.para("Component benchmark: same 512×512 (or 256×256) input, one thread, in-memory (file I/O
excluded), two warm-up calls, nine repeats, median over three randomly chosen real images; process-
based tools (ImageMagick) are flagged. Scheme benchmark: identical 4095-bit payload (the last of the
4096 logo bits is dropped because `blind-watermark` needs strictly fewer bits than its 4096 blocks),
identical attacks, 8-bit rounding, 30 random real images; the block-DCT baseline's strength "
          f"(Δ = {TS['blockdct_delta']}) was tuned on five other images so that its PSNR
({TS['blockdct_psnr']:.1f} dB) matches the project's ({TS['project_psnr_tune']:.1f} dB).")
    r.h2("Reproducibility")
    r.para("Everything is seeded; two complete executions of the batch experiment produced identical
metrics (maximum absolute difference 0.0 over 3000 + 4000 + 1600 + 100 rows; timings vary by 5-10 %).
Commands are in Appendix A.")

    r.h1("Results and Discussion")
    r.h2("Imperceptibility (E1)")
    r.para(f"Across the {E1['n_images']} images at α = 0.10 the mean PSNR of the stored 8-bit
watermarked image is **{E1['psnr_all']['mean']:.2f} dB** (median {E1['psnr_all']['median']:.2f},
minimum {E1['psnr_all']['min']:.2f}, maximum {E1['psnr_all']['max']:.2f}; real images only:
{E1['psnr_real']['mean']:.2f} dB) and the mean SSIM is **{E1['ssim_all']['mean']:.4f}** (real only:
{E1['ssim_real']['mean']:.4f}). No image falls below 30 dB; {E1['n_ssim_below_0.98']} images have
SSIM < 0.98 ({', '.join(E1['ssim_below_0.98_ids'])}). The mean absolute pixel change is
{E1['mean_abs_diff_mean']:.2f} grey levels, and PSNR is almost independent of the host (82 of 100
images lie within 35.5-35.75 dB) because the added energy is set by the logo's singular values, not
by the host.")
    r.figure(V("fig_imperceptibility_hist.png"), "Distribution of PSNR and SSIM of the watermarked
images (blue: real, orange: synthetic).", 14.5)
    r.figure(V("fig_representative_comparison.png"), "Original, watermarked and amplified (×10)
absolute difference for four typical images (image_066, image_064, image_001, image_051) and two
failure cases (image_055: near-black sky; image_100: checkerboard).", 10.3)
    r.para(f"**Failure case.** The near-black astronomy image `image_055` has PSNR 35.9 dB but SSIM
{float(IMG.set_index('image_id').loc['image_055','ssim']):.2f}: with almost no signal, the added
structure and clipping at 0 dominate the local statistics (difference of up to
{E1['max_abs_diff_max']:.0f} grey levels). PSNR alone would have hidden this.")
    r.h2("Robustness under attack (E2)")
    rows = []
    for a in ROB.attack:
        x = rob.loc[a]
        rows.append([a, f"{x.mean_nc:.3f} [{x.ci_lo:.3f}, {x.ci_hi:.3f}]", f"{x.median_nc:.3f}",
f"{100*x.pass_rate:.0f} % [{100*x.pass_lo:.0f}, {100*x.pass_hi:.0f}]", f"{x.mean_zncc:.3f}",
f"{x.mean_ber:.3f}"])
    r.table(["Attack", "mean NC [95 % CI]", "median NC", "NC ≥ 0.75 [Wilson 95 %]", "mean ZNCC",
"mean BER"], rows,
            f"Robustness over {E1['n_images']} images per attack (uint8 protocol). Total:
{E2['n_pass']} of {E2['n_trials']} trials ≥ 0.75 ({pc(E2['pass_rate'], 1)}); mean NC
{E2['mean_nc']:.3f}.", widths=[4.4, 3.6, 1.8, 3.4, 1.7, 1.7], size=8.5)
    r.para(f"**Overall,** {E2['n_pass']} of {E2['n_trials']} trials ({pc(E2['pass_rate'], 1)}) reach
NC ≥ 0.75 when the 100 unattacked reference trials are counted, as the first submission did (25 of
30, 83 %); counting only the 14 attacks it is {E2['n_pass_attacks_only']} of
{E2['n_trials_attacks_only']} ({pc(E2['pass_rate_attacks_only'], 1)}, mean NC
{E2['mean_nc_attacks_only']:.3f}). With the reference included the mean NC is {E2['mean_nc']:.3f}
(real images only {E2['mean_nc_real_only']:.3f}, pass rate {pc(E2['pass_rate_real_only'], 1)}). Clean
```

```
extraction gives a mean NC of {E2['clean_nc_mean']:.3f} (minimum {E2['clean_nc_min']:.3f}, on the
checkerboard). By attack family, mean NC / pass rate are: " +
          "; ".join(f"{row.family} {row.mean_nc:.3f} / {pc(row.pass_rate)}" for row in
FAM.itertuples()) + ".")
    r.figure(V("fig_nc_boxplot.png"), "NC per attack over 100 images (box = interquartile range, line
= median) and the share of images with NC ≥ 0.75.", 14.5)
    r.figure(V("fig_pass_rate.png"), "Detection pass rate per attack with 95 % Wilson intervals.",
13.0)
    r.figure(V("fig_extraction_gallery.png"), "Extracted logo under every attack for one image
(image_066, the cameraman).", 15.5)
    r.para("**Discussion.** The scheme is very robust to JPEG compression (100 % at Q = 90 and 50, 97
% at Q = 20), median filtering (100 %), sharpening (99 %), scaling (98 %), salt-and-pepper noise (97
%), blur (96 %) and mild Gaussian noise (95 %). It is moderately robust to rotation (92 % at 5°, 72 %
at 15°) and strong Gaussian noise (86 %), and it fails under histogram equalisation (44 %) and border
cropping (1 %). The first submission's statement that \"moderate rotation/scaling were recovered with
NC > 0.93 on average\" holds for scaling (0.938) but **not** for 5° rotation (0.880) when 100 images
are used.")
    r.h2("Protocol check")
    mx = PRO.drop(PRO[PRO.attack == "Histogram Equalization"].index).max_abs_diff_per_image.max()
    r.para(f"Attacking the 8-bit image (primary) instead of the float image (first submission)
changes the mean NC by less than 0.001 for every attack except histogram equalisation (0.644 vs
0.638; up to 0.21 for a single image); the largest per-image change for the other attacks is
{mx:.3f}. Mean NC over all trials: {E2['mean_nc']:.4f} (uint8) versus
{E2['mean_nc_float_protocol']:.4f} (float).")
    r.h2("Which images fail?")
    sig = COR[COR.p_contrast < 0.01].sort_values("rho_contrast")
    r.para("Robustness depends on the image. Across the 100 images, NC after most attacks is
**negatively** correlated with image contrast (Spearman ρ): " +
          "; ".join(f"{x.attack} {x.rho_contrast:.2f}" for x in sig.itertuples()) +
          ". Histogram equalisation correlates with mean brightness instead (ρ = " +
f"{float(COR[COR.attack=='Histogram Equalization'].rho_mean_intensity.iloc[0]):.2f}). These are
associations (n = 100, uncorrected p-values), not demonstrated causes.")
    r.figure(V("fig_nc_vs_contrast.png"), "NC after three attacks against image contrast (blue: real,
orange: synthetic).", 15.0)
    cf = CATF.set_index("category")
    worst = cf.loc[cf.index != "synthetic_control"].sort_values("noise").head(2)
    r.para(f"By category (Appendix C; several categories have only 1-4 images, so these are
descriptive): the two document-text images are the least robust to noise (mean NC
{cf.loc['document_text','noise']:.2f}) and filtering ({cf.loc['document_text','filtering']:.2f}), and
portraits and documents are the least robust to histogram equalisation
({cf.loc['person_portrait','global intensity']:.2f} and {cf.loc['document_text','global
intensity']:.2f}), while natural textures and animals are among the most robust.")
    r.figure(V("fig_category_heatmap.png"), "Mean NC by image category and attack family (real
images; number of images per category in brackets).", 13.5)
    r.h2("Embedding strength α (E3)")
    j = "JPEG Q=50"; g = "Gaussian Noise (var=0.02)"
    r.para(f"Mean PSNR falls from {sweep('No Attack',0.02,'psnr'):.1f} dB (α = 0.02) to {sweep('No
Attack',0.10,'psnr'):.1f} dB (0.10) and {sweep('No Attack',0.40,'psnr'):.1f} dB (0.40). The effect on
robustness depends on the attack. **After JPEG Q=50 the mean NC is essentially flat**
({sweep(j,0.02,'nc_mean'):.3f} at α = 0.02, {sweep(j,0.10,'nc_mean'):.3f} at 0.10,
{sweep(j,0.40,'nc_mean'):.3f} at 0.40), so the rise shown in the first submission's trade-off figure
was an artefact: its NC axis spanned only 0.93-0.99 on a second y-axis, and its JPEG simulation
truncated floating-point pixels to 8 bit (a -0.5 grey-level bias), which lowered NC after JPEG at α =
0.02 from 0.981 to 0.929 and had no effect for α ≥ 0.05 (re-creation of the original conditions:
`results/exp_jpeg_truncation.txt`). A real trade-off appears under harsher attacks: under Gaussian
noise (σ² = 0.02) the pass rate rises from {pc(sweep(g,0.02,'pass_rate'))} at α = 0.02 to
{pc(sweep(g,0.05,'pass_rate'))} at 0.05, {pc(sweep(g,0.10,'pass_rate'))} at 0.10,
{pc(sweep(g,0.20,'pass_rate'))} at 0.20 and {pc(sweep(g,0.40,'pass_rate'))} at 0.40, while PSNR drops
below 30 dB for α ≥ 0.20 ({sweep('No Attack',0.20,'psnr'):.1f} dB). The first submission suggested α
≈ 0.15-0.20 \"at a small, still-imperceptible PSNR cost\"; the measured cost is {sweep('No
Attack',0.15,'psnr'):.1f} dB and {sweep('No Attack',0.20,'psnr'):.1f} dB, i.e. at or just above the
conventional 30 dB rule of thumb.")
    r.figure(V("fig_alpha_tradeoff.png"), "α sweep over 100 images: PSNR (left), mean NC (middle) and
pass rate (right) after four attacks; the dashed line in the middle panel marks NC = 0.75.", 16.0)
    r.h2("False-positive controls (E4)")
    r.para(f"E4 asks whether a high NC proves the watermark is present. (a) An *unwatermarked other*
image decoded with the key of image *i* gives a mean NC of {CA.mean_nc:.3f} (95th percentile
{CA.p95:.3f}, maximum {CA['max']:.3f}); (b) a *watermarked other* image (same logo, wrong key) gives
{CB.mean_nc:.3f} (95th percentile {CB.p95:.3f}); {pc(CA.frac_ge_0_75, 1)} and {pc(CB.frac_ge_0_75,
1)} of these controls exceed the 0.75 threshold, but only {pc(CA.frac_ge_0_90, 1)} and
{pc(CB.frac_ge_0_90, 1)} exceed 0.90. (c) The **same host watermarked with a different logo and
decoded with the key of logo A** gives NC(A) = **{CC.mean_nc:.3f}**, against {G.mean_nc:.3f} for the
genuine pair and {CCR.mean_nc:.3f} for the different logo with its own key: the extractor returns the
key owner's logo, whatever was embedded (200 trials; ZNCC {CC.mean_zncc:.3f}). (d) Decoding the
image's own *unwatermarked* host with its own key gives NC = 0 by construction, since S_att = S_orig
- a degenerate case that must not be read as a positive result.")
    r.figure(V("fig_false_positive.png"), "Mean NC for the genuine pair, for a different logo decoded
with the owner's key (orange), and for unrelated images.", 11.0)
```

```
    r.figure(V("fig_controls_hist.png"), "Distribution of NC for genuine extraction and for
unrelated-image controls.", 13.0)
    r.para(f"**Interpretation.** NC measures the fidelity of the recovered singular values *given the
key*; it does not prove that a particular logo is embedded. Control (c) is the false-positive problem
described for SVD schemes in [4][5] (Section 2.2), now measured for this implementation. Wrong-image
controls overlap the 0.75 threshold: the 95th percentile of NC for unrelated images is
{E4['wrong_image_p95']:.2f}, and a detector calibrated to a 5 % false-alarm rate would reject
{pc(E4['attacked_genuine_below_wrong_p95'])} of the attacked genuine trials. A claim of ownership
would need an additional safeguard (for example an image-bound signature); that was not implemented
here.")
    r.figure(V("fig_nc_vs_zncc.png"), "Mean NC and mean ZNCC per attack.", 13.0)
    r.para(f"**NC versus ZNCC.** ZNCC is about 0.02 lower than NC for the clean case
({E2['zncc_clean_mean']:.3f} vs {E2['clean_nc_mean']:.3f}); over all trials the mean ZNCC is
{E2['mean_zncc']:.3f} and {pc(E2['zncc_pass_rate'], 1)} of trials reach 0.75, so the conclusions
based on NC are not an artefact of the brightness offset for this logo.")
    r.h2("Failure cases")
    r.bullets([
        f"**Border cropping:** mean NC {rob.loc['Cropping 25%'].mean_nc:.3f}; with 43.75 % of the
image area zeroed, every singular value changes. The extracted logo is essentially black (BER
{rob.loc['Cropping 25%'].mean_ber:.3f} ≈ the white fraction of the logo).",
        f"**Histogram equalisation:** pass rate {pc(rob.loc['Histogram Equalization'].pass_rate)}; it
remaps the whole intensity distribution, and the effect varies strongly between images (first
submission: 0.10 for the cameraman, 0.66 for the astronaut).",
        "**Near-constant images:** SSIM collapses (0.58) though PSNR is 35.9 dB; the checkerboard has
the lowest clean NC (0.79).",
        "**Rotation > 5°:** 72 % pass at 15°; no re-registration is attempted.",
    ])
    r.h2("Comparison with existing technologies (S1, S2)")
    r.para("**Component level.** Equivalent operations agree numerically (DCT and SVD to ≤ 1.5e-16
relative; Haar DWT to 1e-13; Pillow and OpenCV JPEG bit-identical; 3×3 median identical). OpenCV is
much faster for the attack operations (3×3 median ≈ 340×, Gaussian blur 4.4×, rotation 5.1×, down/up-
scaling 38×, histogram equalisation 19×; part of this reflects `uint8` versus `float64` input),
whereas `scipy.linalg.svd` is only 1.19× faster than NumPy's for the SVD that dominates embedding.
Because the whole experiment takes about 80 s, none of this justifies changing libraries.")
    r.figure(V("fig_tech_component_speed.png"), "Median time per call for equivalent operations in
different libraries (log scale; blue = implementation used by the project; orange includes process
start-up and file I/O).", 15.0)
    rows = []
    for a in ["No Attack", "JPEG Q=50", "JPEG Q=20", "Gaussian Noise (var=0.02)", "Median Filter
3x3", "Gaussian Blur (sigma=1.5)", "Rotation 5 deg", "Cropping 25%", "Scaling 0.5x"]:
        rows.append([a] + [f"{ber(s, a):.3f}" for s in (PROJ, BWM, BDCT, LSB)])
    rows.append(["**PSNR (dB)**"] + [f"{psnr_s(s):.1f}" for s in (PROJ, BWM, BDCT, LSB)])
    rows.append(["**embed / extract (ms)**"] + [f"{tms(s,'t_embed_ms'):.0f} /
{tms(s,'t_extract_ms'):.0f}" for s in (PROJ, BWM, BDCT, LSB)])
    r.table(["Attack (mean BER, lower is better)", "Project", "blind-watermark 0.4.4", "8×8 block
DCT", "LSB"], rows,
            "Scheme comparison on 30 real images with the same 4095-bit payload (BER 0.5 = chance).",
widths=[5.2, 2.6, 3.4, 2.8, 2.4], size=9)
    r.figure(V("fig_tech_scheme_ber.png"), "Mean BER of four schemes under nine attacks (PSNR in the
legend).", 15.0)
    r.para(f"**Scheme level.** At equal PSNR the project is far more robust than LSB and 8×8 block-
DCT embedding, which fall to chance level under JPEG, rescaling and rotation. It is **not superior to
the existing blind block-based hybrid** `blind-watermark`: that package achieves a lower BER under
JPEG Q=50 ({ber(BWM,'JPEG Q=50'):.3f} vs {ber(PROJ,'JPEG Q=50'):.3f}), noise ({ber(BWM,'Gaussian
Noise (var=0.02)'):.3f} vs {ber(PROJ,'Gaussian Noise (var=0.02)'):.3f}), blur, rescaling and cropping
({ber(BWM,'Cropping 25%'):.3f} vs {ber(PROJ,'Cropping 25%'):.3f}), at a higher PSNR
({psnr_s(BWM):.1f} vs {psnr_s(PROJ):.1f} dB) and without a key; but it is
{tms(BWM,'t_embed_ms')/tms(PROJ,'t_embed_ms'):.0f}× slower to embed and
{tms(BWM,'t_extract_ms')/tms(PROJ,'t_extract_ms'):.0f}× slower to extract, and it is worse under
rotation ({ber(BWM,'Rotation 5 deg'):.3f} vs {ber(PROJ,'Rotation 5 deg'):.3f}) and JPEG Q=20
({ber(BWM,'JPEG Q=20'):.3f} vs {ber(PROJ,'JPEG Q=20'):.3f}). Its colour/YUV design and our channel-
wise attacks may favour it. What the project contributes is a short, explainable, key-based reference
implementation and the evaluation of its weaknesses, not a better algorithm.")
    r.h2("Exploratory video test (V1)")
    v = VID.groupby("codec", sort=False).agg(nc=("nc_mean", "mean"), ncmin=("nc_min", "min"),
psnr=("psnr_wm_vs_orig_db", "mean"), psnr_c=("psnr_codec_only_db", "mean"), n=("frames_decoded",
"sum"), kb=("size_wm_kb", "mean")).reset_index()
    rows = [[x.codec, int(x.n), f"{x.psnr:.1f}", f"{x.psnr_c:.1f}" if x.psnr_c < 99 else "∞
(lossless)", f"{x.nc:.3f}", f"{x.ncmin:.3f}", f"{x.kb:.0f}"] for x in v.itertuples()]
    r.table(["Codec setting", "frames decoded", "PSNR vs original (dB)", "PSNR of codec alone (dB)",
"mean NC", "min NC", "file size (kB)"], rows,
            "Per-frame embedding on 60 frames of each of two clips (vtest.avi, Megamind.avi), re-
encoded with FFmpeg; means over the two clips.", widths=[3.3, 1.9, 2.5, 2.6, 1.7, 1.6, 2.1],
size=8.5)
    r.para(f"All 720 decoded frames (2 clips × 60 frames × 6 settings) yielded NC ≥
{VID.nc_min.min():.3f}, so lossy video codecs are not the difficulty for this scheme; embedding takes
{VM['vtest.avi']['embed_ms_per_frame']:.0f} ms per frame (≈ {VM['vtest.avi']
['embed_fps_single_thread']:.0f} frames per second on one core, 512×512). **Caveats:** this is a
```

```
feasibility test, not a video-watermarking benchmark - each frame needs its own key (about 2 MB), temporal attacks (frame dropping, re-ordering, frame-rate change) and colour video were not tested, and 60 frames per clip is small. A naive frame-drop test (every tenth frame removed, keys aligned by position) still gave NC {VID.framedrop_nc_naive_index.dropna().mean():.3f} against {VID.framedrop_nc_with_frame_ids.dropna().mean():.3f} when aligned by frame ID, presumably because neighbouring frames are almost identical - another symptom of the key-dependence discussed in Section 8.6.")
    r.h2("Comparison with the claims of the first submission")
    r.table(["First submission", "This evaluation (100 images)", "Verdict"], [
        ["mean PSNR 35.6 dB / SSIM 0.996 (2 images)", f"{E1['psnr_all']['mean']:.1f} dB / {E1['ssim_all']['mean']:.3f}; {E1['n_ssim_below_0.98']} images below SSIM 0.98", "confirmed, with exceptions"],
        ["25/30 trials ≥ 0.75 (83 %)", f"{E2['n_pass']}/{E2['n_trials']} ({pc(E2['pass_rate'], 1)})", "confirmed"],
        ["JPEG: NC > 0.97", f"mean NC {rob.loc['JPEG Q=90'].mean_nc:.3f} / {rob.loc['JPEG Q=50'].mean_nc:.3f} / {rob.loc['JPEG Q=20'].mean_nc:.3f} (Q 90/50/20); pass {pc(rob.loc['JPEG Q=90'].pass_rate)}/{pc(rob.loc['JPEG Q=50'].pass_rate)}/{pc(rob.loc['JPEG Q=20'].pass_rate)}", "holds for Q ≥ 50; Q=20 slightly lower"],
        ["moderate rotation/scaling NC > 0.93", f"scaling {rob.loc['Scaling 0.5x'].mean_nc:.3f}; rotation 5°/15° {rob.loc['Rotation 5 deg'].mean_nc:.3f}/{rob.loc['Rotation 15 deg'].mean_nc:.3f}", "scaling yes, rotation no"],
        ["cropping NC 0.00; histogram eq. 0.10 / 0.66", f"cropping {rob.loc['Cropping 25%'].mean_nc:.3f} (pass {pc(rob.loc['Cropping 25%'].pass_rate)}); hist-eq mean {rob.loc['Histogram Equalization'].mean_nc:.3f} (pass {pc(rob.loc['Histogram Equalization'].pass_rate)})", "confirmed; hist-eq varies widely by image"],
        ["robustness rises with α (JPEG Q=50)", f"flat: {sweep(j,0.02,'nc_mean'):.3f} → {sweep(j,0.40,'nc_mean'):.3f}", "not supported (axis + truncation artefact); true for noise/blur/rotation"],
        ["α ≈ 0.15-0.20 at \"small\" PSNR cost", f"{sweep('No Attack',0.15,'psnr'):.1f} / {sweep('No Attack',0.20,'psnr'):.1f} dB", "borderline"],
        ["\"block DCT\", \"semi-blind\"", "global DCT; non-blind", "corrected"],
        ["extraction proves ownership (implied)", f"wrong logo decoded at NC {CC.mean_nc:.3f}", "**not supported**"],
        ["hybrids beat single transforms (literature)", "not tested; project beats LSB/block-DCT, `blind-watermark` beats project", "unverified claim"],
    ], "Claims of the first submission against the 100-image evaluation.", widths=[4.7, 7.6, 4.3], size=8.5)

    r.h1("Limitations and Future Scope")
    r.h2("Current limitations")
    r.bullets([
        "**False-positive problem** (Section 8.6): the extractor returns the key owner's logo; not fixed.",
        "**Non-blind**: needs a ≈ 2.1 MB key per image holding the original singular values.",
        "**Geometry:** border cropping and rotation beyond 5° defeat the scheme; no re-synchronisation.",
        "**Histogram equalisation** and strong noise reduce the pass rate to 44 % and 86 %.",
        "**Grey-scale only**; clean extraction is not lossless (BER ≈ 0.9 % from logo resampling).",
        f"**Imperceptibility metrics:** PSNR is almost host-independent; SSIM collapses on near-constant images.",
    ])
    r.h2("Dataset limitations")
    r.para("A convenience sample of 94 real images from GitHub-hosted sample sets (plus six synthetic controls); several categories contain 1-4 images; image rights are mostly not documented upstream; standardisation to grey, centred square crops and 512×512 resampling remove colour, borders and fine detail of the originals. The images are not a recognised benchmark.")
    r.h2("Performance, quality and comparison limits")
    r.para("Timings were taken on four shared vCPUs (5-10 % run-to-run variation, no GPU). Only one existing watermarking package was measured, with channel-wise attacks that may favour it. The video test covers 120 distinct frames from two clips.")
    r.h2("Future scope")
    r.bullets([
        "**Bind the logo to the image** (for example a signature of image features stored in the key) to remove the false-positive problem; verify with the E4 controls.",
        "**Block-based redundant embedding** with per-block keys to address cropping, and a comparison against the `blind-watermark` design as the new baseline.",
        "**Geometric re-synchronisation** (log-polar or template-based registration, cf. [22]).",
        "**Colour:** embed in the luminance channel of a YCbCr image.",
        "**Video:** shared or index-aware keys, temporal redundancy, tests with frame dropping and re-timing; **real-time or embedded** implementations (≈ 37 ms per 512² frame on one CPU core suggests ≈ 27 frames per second without optimisation).",
        "**Learned or hybrid approaches** [29][30] as a comparison, if the no-ML constraint is relaxed.",
    ])

    r.h1("Conclusion")
    r.para(f"The project implemented a closed-form hybrid DWT-DCT-SVD watermarking scheme and, in this extended evaluation, tested it on {len(META)} unique images with 14 attacks and an unattacked
```

```
reference, controls, an α sweep, a technology comparison and a short video test. **Objectives
achieved:** the pipeline is implemented, tested (43 tests) and reproducible; imperceptibility is good
(mean PSNR {E1['psnr_all']['mean']:.1f} dB, SSIM {E1['ssim_all']['mean']:.3f}); {pc(E2['pass_rate'],
1)} of attack trials recover a logo with NC ≥ 0.75, with very high robustness to JPEG, median
filtering, sharpening and rescaling.")
    r.para("**Main findings:** (1) the first submission's headline numbers are reproducible and
largely hold at scale, but moderate rotation and histogram equalisation are weaker than it suggested;
(2) the imperceptibility-robustness trade-off in α exists for noise, blur and rotation but not for
JPEG, and the earlier figure exaggerated it (zoomed axis, truncation bias); (3) **NC is not proof of
ownership** - a different logo decoded with the owner's key is reported at NC "
           f"{CC.mean_nc:.2f} - and unrelated images exceed the 0.75 threshold in 10-13 % of
controls; (4) the scheme is more robust than LSB and block-DCT at equal PSNR but is not better than
an existing blind block-based package, which has lower error rates under most attacks but is 13×
slower to embed and weaker under rotation; (5) per-frame application to re-encoded video works
technically, with per-frame keys.")
    r.para("**Practical significance and lessons.** The scheme is a clear, explainable teaching
implementation and a usable robust mark against re-compression and filtering, provided its key is
protected and ownership claims are not based on NC alone. The most valuable lesson of the extension
is methodological: more images, seeded attacks, controls and honest axes change the conclusions of a
two-image study.")

    r.h1("References", numbered=False)
    refs = open(os.path.join(ROOT, "research", "REFERENCES.md"), encoding="utf-8").read()
    items = []
    for line in refs.splitlines():
        m = re.match(r"\| (\[\d+\]) \| (.*?) \| (.*?) \|\s*$", line)
        if m:
            items.append((int(m.group(1)[1:-1]), m.group(1), re.sub(r"\*([^*]+)\*", r"\1",
m.group(2))))
    for _, tag, txt in sorted(items):
        p = r.para(f"{tag}  {txt}", size=9.5, space_after=3)
    r.para("Verification status of each reference (V1 confirmed against the publisher record, V2
confirmed through secondary listings, V3 not independently verified) is given in
`research/REFERENCES.md`; references that could not be verified were not cited.", size=9,
italic=True)

    r.h1("Installation and execution", appendix="A")
    r.code("""pip install -r source_code/requirements.txt
cd source_code/src
python build_dataset.py
python run_batch_experiment.py
python analysis.py
python benchmark_tech.py && python analysis_tech.py
python video_demo.py --frames 60
python run_experiment.py
cd ../.. && python -m pytest tests -q""", size=8.5)
    r.h1("Dataset sources and attribution", appendix="B")
    g = META[META.is_synthetic == 0].groupby(["source_group",
"repository_license"]).size().reset_index(name="n").sort_values("n", ascending=False)
    r.table(["Source group", "Hosting-repository licence", "images"], [[x.source_group,
x.repository_license, x.n] for x in g.itertuples()],
            "Real-image sources (the repository licence is not a statement about each photograph).",
widths=[7.6, 6.4, 2.0], size=9)
    r.para("Upstream-stated licences used in the metadata: CC0 (scikit-image: brick, camera, chelsea,
coffee, grass, gravel, retina), NASA public domain (astronaut, Hubble field), CC BY 2.0 (scikit-learn
`china.jpg` by *danielbuechele* and `flower.jpg` by *vultilion*; COCO image `hf_cats`, attribution to
the Flickr author per COCO terms - not individually verified), Oxford affine-regions images (K.
Mikolajczyk *et al.*, research use), Middlebury stereo/flow images (research use). Synthetic controls
are generated by `build_dataset.py`. Full per-image attribution: `data/metadata.csv`,
`data/README.md`.", size=10)
    r.h1("Supplementary results", appendix="C")
    cols = ["compression", "filtering", "geometric", "global intensity", "noise", "none"]
    rows = [[x.category, int(x.n_images)] + [f"{getattr(x, c.replace(' ', '_')):.2f}" for c in cols]
for x in CATF.rename(columns=lambda c: c.replace(" ", "_")).itertuples()]
    r.table(["category", "n", "compr.", "filter.", "geom.", "glob.int.", "noise", "clean"], rows,
"Mean NC by category and attack family (all 100 images, uint8 protocol).", widths=[4.2, 1.0, 1.9,
1.9, 1.9, 1.9, 1.9, 1.9], size=8.5)
    rows = []
    for a in ["Gaussian Noise (var=0.02)", "Gaussian Blur (sigma=1.5)", "Rotation 5 deg", "JPEG
Q=50"]:
        for al in (0.02, 0.05, 0.10, 0.20, 0.40):
            rows.append([a, al, f"{sweep(a, al, 'psnr'):.1f}", f"{sweep(a, al, 'nc_mean'):.3f}",
f"{sweep(a, al, 'zncc_mean'):.3f}", f"{sweep(a, al, 'ber_mean'):.3f}", pc(sweep(a, al,
'pass_rate'))])
    r.table(["attack", "α", "PSNR", "mean NC", "mean ZNCC", "mean BER", "NC ≥ 0.75"], rows, "α sweep
(excerpt of `results/tables/alpha_sweep_summary.csv`).", widths=[4.8, 1.2, 1.7, 2.0, 2.2, 2.2, 2.4],
size=8.5)
```

```
    rows = [[x.control[:70], int(x.n), f"{x.mean_nc:.3f}", f"{x.p95:.3f}", f"{x.max:.3f}",
pc(x.frac_ge_0_75, 1), f"{x.mean_zncc:.3f}"] for x in CTL.itertuples()]
    r.table(["control", "n", "mean NC", "p95", "max", "≥ 0.75", "mean ZNCC"], rows, "False-positive
controls (E4).", widths=[6.6, 1.0, 1.7, 1.4, 1.4, 1.7, 1.9], size=8)
    r.figure(V("fig_timing.png"), "Timing of embedding and extraction (single thread, 4 parallel
workers).", 11.0)
    r.figure(V("contact_sheet_watermarked.png"), "Watermarked outputs of all 100 images (same order
as Figure 7.1).", 13.0)
    r.h1("Test results and configuration", appendix="D")
    r.para("`python -m pytest tests -q`: **43 passed**. Configuration (`source_code/config.json`):")
    r.code(json.dumps(RS["config"], indent=1), size=8)
    r.h1("Reproduction of the first submission", appendix="E")
    rows = [[a, f"{x.orig_nc:.4f}", f"{x.rerun_nc:.4f}", f"{x.d_nc:+.4f}"] for a, x in
LEG.set_index(LEG.columns[0]).iterrows()]
    r.table(["attack (cameraman)", "NC (first report)", "NC (re-run)", "difference"], rows, "Re-
execution of the original two-image experiment against Table 5.1 of the first report. Differences are
due to seeded noise and JPEG rounding.", widths=[5.6, 3.6, 3.4, 3.0], size=9)
    r.h1("Corrections to the first report and presentation", appendix="F")
    r.numbered([
        "\"Block DCT\" → **global** 2-D DCT of the LL band (text, diagrams, deck).",
        "\"Semi-blind\" → **non-blind** (needs S_orig of the unmarked host).",
        "Trade-off figure: dual axis with the NC axis zoomed to 0.93-0.99 replaced by separate
panels, four attacks and 100 images; the single low point at α = 0.02 was caused by truncating floats
before JPEG encoding (`results/exp_jpeg_truncation.txt`).",
        "\"Cropping 25%\" explained as 43.75 % of the image area.",
        "\"Fifteen attacks\": the suite has 14 attacks plus the unattacked reference \"No Attack\";
pass rates are given with and without the reference.",
        "The 0.75 NC threshold, 30 dB and 0.98 SSIM rules of thumb are conventions; no verified
literature source was found.",
        "\"Hybrids repeatedly outperform any single transform\" is a reported trend in the
literature, not a result of this project.",
        "Claim that JPEG alignment of the DCT explains the JPEG robustness: not supported (global
DCT, SVD dominates).",
        "Reference list rebuilt with verified entries; textbook and documentation entries
completed.",
        "Added: 100-image evaluation, ZNCC, false-positive controls, technology comparison, video
test, unit tests.",
    ])
    r.h1("Known limitations of this submission", appendix="G")
    r.para("See Chapter 9 and `documentation/CHANGELOG.md`. Items not completed: no Canva design was
edited (no link or connection was provided); no university report template was available; the false-
positive problem is measured but not fixed.", size=10.5)


def front_matter(r: Report, pages, regs):
    d = r.doc
    for _ in range(3):
        d.add_paragraph()
    r.para("NMIMS MPSTME", align="center", size=12, color=None)
    r.para("B.Tech Computer Engineering", align="center", size=10.5)
    d.add_paragraph()
    p = r.para("**DIGITAL IMAGE WATERMARKING**", align="center", size=26)
    r.para("A Hybrid DWT-DCT-SVD Approach for Robust, Imperceptible Copyright Protection",
align="center", size=14, italic=True)
    r.para("Extended evaluation on 100 images, technology comparison, literature review and
exploratory video test", align="center", size=11)
    r.para("(Classical signal-processing method - no machine learning / deep learning)",
align="center", size=10, italic=True)
    for _ in range(2):
        d.add_paragraph()
    r.para("Subject: Image and Video Processing (IVP)", align="center", size=11.5)
    d.add_paragraph()
    r.para("Submitted by", align="center", size=10)
    r.para("**Krishang Vikram Singh**", align="center", size=13)
    r.para("Roll No. B106  |  Batch C1", align="center", size=10.5)
    d.add_paragraph()
    r.para("First submission: July 2026  ·  Revised and extended edition: October 2026",
align="center", size=10)
    r.doc.add_page_break()
    r.h1("Abstract", numbered=False)
    r.para(f"This report presents the design, implementation and evaluation of a digital image
watermarking system built entirely from classical signal-processing techniques - the Discrete Wavelet
Transform (DWT), Discrete Cosine Transform (DCT) and Singular Value Decomposition (SVD). A 64×64
binary logo is embedded into the singular values of the DCT of the LL sub-band of a grey-scale host
image. The first submission evaluated two images; this extended edition evaluates **{len(META)}
unique images ({NREAL} real, {NSYN} labelled synthetic)** under fourteen attacks plus an unattacked
reference (JPEG at three qualities, Gaussian noise at two levels, salt-and-pepper noise, median,
blur, sharpening, rotation at two angles, cropping, scaling, histogram equalisation), with fully
```

```
seeded, logged and validated batch processing ({RS['attack_trials_total']} attack trials,
{RS['alpha_sweep_rows_total']} α-sweep rows, {RS['control_rows']} control rows, 0 failures).")
    r.para(f"Imperceptibility is good (mean PSNR {E1['psnr_all']['mean']:.1f} dB, mean SSIM
{E1['ssim_all']['mean']:.3f}); the normalised correlation of the extracted logo reaches 0.75 in
{pc(E2['pass_rate'], 1)} of the attack trials, with ≥ 95 % pass rates for JPEG (Q ≥ 20), median
filtering, sharpening, scaling, blur and mild noise, 72 % for 15° rotation, 44 % for histogram
equalisation and 1 % for border cropping. Three findings qualify the first submission: the α trade-
off in the earlier figure was an artefact for JPEG - a zoomed NC axis (0.93-0.99) plus a truncation
bias in the JPEG simulation that lowered NC only at α = 0.02 (the trade-off is real for noise, blur
and rotation); the \"block DCT\" and \"semi-blind\" descriptions were wrong (global DCT; non-blind);
and **a high normalised correlation does not prove ownership** - a different logo decoded with the
owner's key scores NC {CC.mean_nc:.3f}, and unrelated images exceed 0.75 in 10-13 % of controls (the
false-positive problem of SVD watermarking).")
    r.para(f"Against existing technology, the scheme is far more robust than LSB and 8×8 block-DCT
embedding at equal PSNR but is not better than the open-source `blind-watermark` package, which has
lower bit-error rates under most attacks at higher PSNR, is
{tms(BWM,'t_embed_ms')/tms(PROJ,'t_embed_ms'):.0f}× slower to embed and is weaker under rotation. An
exploratory per-frame test on two real video clips re-encoded with H.264 and MJPEG recovered the logo
in all 720 decoded frames (NC ≥ {VID.nc_min.min():.2f}). Every number in this report is generated
from the recorded experiment outputs; limitations and unverified items are stated explicitly.")
    r.para("**Keywords:** digital watermarking, DWT, DCT, SVD, robustness, false-positive problem,
PSNR, SSIM, normalised correlation.", size=10)
    entries = []
    for lvl, num, text, label in regs["headings"]:
        if lvl <= 2:
            entries.append((lvl - 1, label, label))
    r.toc_entries([("Abstract" and (0, "Abstract", "Abstract"))] + entries, "Table of Contents")
    r.toc_entries([(0, f"{l}: {c}".replace("`", ""), l) for l, c in regs["figures"]], "List of
Figures")
    r.toc_entries([(0, f"{l}: {c}".replace("`", ""), l) for l, c in regs["tables"]], "List of
Tables")
    r.doc.add_heading("Abbreviations and symbols", level=1).paragraph_format.page_break_before = True
    r.table(["Symbol", "Meaning"], [
        ["DWT / DCT / SVD", "discrete wavelet transform / discrete cosine transform / singular value
decomposition"],
        ["LL", "approximation (low-pass) sub-band of the DWT"],
        ["α", "embedding strength (0.10 unless stated)"],
        ["S, S_w, S_orig, S_att", "singular values of the host sub-band, of the logo, of the original
host, of the attacked image"],
        ["U, V, U_w, V_w", "singular-vector matrices of host and logo"],
        ["PSNR / SSIM", "peak signal-to-noise ratio / structural similarity index"],
        ["NC / ZNCC / BER", "normalised correlation / zero-mean NC / bit error rate"],
        ["LSB", "least significant bit"],
        ["CRF / MJPEG / FFV1", "constant-rate-factor of x264 / motion JPEG / lossless codec"],
        ["FPP", "false-positive problem"],
    ], "Abbreviations and symbols.", widths=[4.4, 12.2], size=9)


def build(pages, path):
    r0 = Report()
    content(r0)
    regs = dict(headings=r0.headings, figures=r0.figures, tables=r0.tables)
    r = Report(pages)
    front_matter(r, pages, regs)
    r.h1_no = r.h2_no = r.h3_no = r.fig_no = r.tab_no = 0
    r.headings, r.figures, r.tables = [], [], []
    r.doc.add_page_break()
    content(r)
    r.save(path)
    return regs


def page_map(pdf, regs):
    n = int(re.search(r"Pages:\s+(\d+)", subprocess.run(["pdfinfo", pdf], capture_output=True,
text=True).stdout).group(1))
    texts = []
    for i in range(1, n + 1):
        t = subprocess.run(["pdftotext", "-layout", "-f", str(i), "-l", str(i), pdf, "-"],
capture_output=True, text=True).stdout
        texts.append(t)
    start = next(i for i, t in enumerate(texts) if "Abbreviations and symbols" in t and "Meaning" in
t) + 1
    pages = {"Abstract": 2}

    def find(label, prefix_len=38, from_page=start):
        key = label[:prefix_len]
        for i in range(from_page, n):
            for line in texts[i].splitlines():
```

```
                s = line.strip()
                if s.startswith(key) and "...." not in s:
                    return i + 1
        return None
    for lvl, num, text, label in regs["headings"]:
        pages[label] = find(label) or 0
    for l, c in regs["figures"]:
        pages[l] = find(l + ":", 12) or 0
    for l, c in regs["tables"]:
        pages[l] = find(l + ":", 12) or 0
    return pages, n


if __name__ == "__main__":
    docx_path = os.path.join(OUT, "Final_Project_Report.docx")
    regs = build({}, docx_path)
    for it in range(2):
        subprocess.run(["soffice", "--headless", "--convert-to", "pdf", "--outdir", OUT, docx_path],
capture_output=True, timeout=300)
        pdf = os.path.join(OUT, "Final_Project_Report.pdf")
        pages, n = page_map(pdf, regs)
        missing = [k for k, v in pages.items() if not v]
        print(f"pass {it + 1}: {n} pages; unresolved registry entries: {len(missing)} {missing[:6]}")
        regs = build(pages, docx_path)
    subprocess.run(["soffice", "--headless", "--convert-to", "pdf", "--outdir", OUT, docx_path],
capture_output=True, timeout=300)
    print("done")
```

5.3 build_deck.py

Builds the presentation. 587 lines.

```
from __future__ import annotations

import json
import os
import re

import pandas as pd
from PIL import Image
from pptx import Presentation
from pptx.dml.color import RGBColor
from pptx.enum.shapes import MSO_SHAPE
from pptx.enum.text import MSO_ANCHOR, PP_ALIGN
from pptx.util import Emu, Inches, Pt

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
RES, VIS, TAB = os.path.join(ROOT, "results"), os.path.join(ROOT, "visualizations"),
os.path.join(ROOT, "results", "tables")
OUT = os.path.join(ROOT, "presentation")
os.makedirs(OUT, exist_ok=True)
V = lambda n: os.path.join(VIS, n)

AS = json.load(open(os.path.join(RES, "analysis_summary.json")))
RS = json.load(open(os.path.join(RES, "run_summary.json")))
TS = json.load(open(os.path.join(RES, "tech_summary.json")))
ROB = pd.read_csv(os.path.join(TAB, "robustness_by_attack.csv")).set_index("attack")
SWP = pd.read_csv(os.path.join(TAB, "alpha_sweep_summary.csv")).set_index(["attack", "alpha"])
CTL = pd.read_csv(os.path.join(TAB, "controls_summary.csv"))
COR = pd.read_csv(os.path.join(TAB, "nc_vs_image_statistics.csv")).set_index("attack")
TSCH = pd.read_csv(os.path.join(RES, "tech_benchmark_schemes.csv")).set_index(["scheme", "attack"])
VID = pd.read_csv(os.path.join(RES, "video_demo_results.csv"))
META = pd.read_csv(os.path.join(ROOT, "data", "metadata.csv"))
E1, E2, E4, BASE = AS["E1"], AS["E2"], AS["E4"], AS["metric_baselines"]
G = CTL.iloc[0]
CA, CB = CTL[CTL.control.str.startswith("a:")].iloc[0], CTL[CTL.control.str.startswith("b:")].iloc[0]
CC = CTL[CTL.control.str.startswith("c: ")].iloc[0]
PROJ, BWM, BDCT, LSB = ("Project: global DWT-DCT-SVD", "blind-watermark 0.4.4 (block DWT-DCT-SVD)",
"Baseline: 8x8 block DCT", "Baseline: spatial LSB")
ber = lambda s, a: float(TSCH.loc[(s, a), "mean_ber"])
psnr_s = lambda s: float(TSCH.reset_index().query("scheme == @s").psnr_db.mean())
tms = lambda s, c: float(TSCH.reset_index().query("scheme == @s")[c].median())
sw = lambda a, al, c: float(SWP.loc[(a, al), c])
pc = lambda x, d=0: f"{100 * x:.{d}f} %"

NAVY, NAVY2, NAVY3 = RGBColor(0x13, 0x23, 0x47), RGBColor(0x0C, 0x18, 0x30), RGBColor(0x1B, 0x2E,
0x57)
```

```
TEAL, TEALD = RGBColor(0x17, 0xB8, 0xB0), RGBColor(0x0E, 0x8F, 0x89)
INK, MUTED, LIGHT, LINE = RGBColor(0x1A, 0x22, 0x33), RGBColor(0x66, 0x70, 0x85), RGBColor(0xF3,
0xF5, 0xF9), RGBColor(0xDC, 0xE2, 0xEB)
WHITE, PALE, RED, REDBG, TEALBG = RGBColor(0xFF, 0xFF, 0xFF), RGBColor(0x93, 0xA8, 0xC7),
RGBColor(0xC1, 0x44, 0x3C), RGBColor(0xFB, 0xEC, 0xEA), RGBColor(0xE6, 0xF6, 0xF5)
FONT = "Calibri"

prs = Presentation()
prs.slide_width, prs.slide_height = Inches(13.333), Inches(7.5)
BLANK = prs.slide_layouts[6]
NOTES = []


def rect(s, x, y, w, h, fill=None, line=None, shape=MSO_SHAPE.RECTANGLE, lw=0.75, radius=None):
    sh = s.shapes.add_shape(shape, Inches(x), Inches(y), Inches(w), Inches(h))
    if fill is None:
        sh.fill.background()
    else:
        sh.fill.solid(); sh.fill.fore_color.rgb = fill
    if line is None:
        sh.line.fill.background()
    else:
        sh.line.color.rgb = line; sh.line.width = Pt(lw)
    sh.shadow.inherit = False
    if radius is not None and shape == MSO_SHAPE.ROUNDED_RECTANGLE:
        sh.adjustments[0] = radius
    return sh


def text(s, x, y, w, h, content, size=12, color=INK, bold=False, align=None, anchor=MSO_ANCHOR.TOP,
italic=False, spacing=None, line_sp=None):
    tb = s.shapes.add_textbox(Inches(x), Inches(y), Inches(w), Inches(h))
    tf = tb.text_frame
    tf.word_wrap = True
    tf.margin_left = tf.margin_right = Inches(0.04)
    tf.margin_top = tf.margin_bottom = Inches(0.02)
    tf.vertical_anchor = anchor
    paras = content if isinstance(content, list) else [content]
    for i, ptxt in enumerate(paras):
        p = tf.paragraphs[0] if i == 0 else tf.add_paragraph()
        if align:
            p.alignment = align
        if spacing is not None:
            p.space_after = Pt(spacing)
        if line_sp:
            p.line_spacing = line_sp
        for part in re.split(r"(\*\*[^*]+\*\*|`[^`]+`)", ptxt):
            if not part:
                continue
            r = p.add_run()
            if part.startswith("**"):
                r.text = part[2:-2]; r.font.bold = True
            elif part.startswith("`"):
                r.text = part[1:-1]; r.font.name = "Consolas"; r.font.bold = bold
                r.font.size = Pt(size - 1); r.font.color.rgb = TEALD if color == INK else color
                continue
            else:
                r.text = part; r.font.bold = bold
            r.font.name = FONT
            r.font.size = Pt(size)
            r.font.color.rgb = color
            r.font.italic = italic
    return tb


def bullets(s, x, y, w, h, items, size=12.5, color=INK, gap=5, dot=TEAL):
    tb = s.shapes.add_textbox(Inches(x), Inches(y), Inches(w), Inches(h))
    tf = tb.text_frame; tf.word_wrap = True
    tf.margin_left = tf.margin_right = Inches(0.04)
    for i, it in enumerate(items):
        p = tf.paragraphs[0] if i == 0 else tf.add_paragraph()
        p.space_after = Pt(gap)
        r0 = p.add_run(); r0.text = "▪  "; r0.font.size = Pt(size); r0.font.color.rgb = dot;
r0.font.name = FONT
        for part in re.split(r"(\*\*[^*]+\*\*|`[^`]+`)", it):
            if not part:
                continue
            r = p.add_run()
```

```
            if part.startswith("**"):
                r.text = part[2:-2]; r.font.bold = True
            elif part.startswith("`"):
                r.text = part[1:-1]; r.font.name = "Consolas"; r.font.size = Pt(size - 1);
r.font.color.rgb = TEALD
                continue
            else:
                r.text = part
            r.font.name = FONT; r.font.size = Pt(size); r.font.color.rgb = color
    return tb


def picture(s, path, x, y, w=None, h=None):
    iw, ih = Image.open(path).size
    ar = iw / ih
    if w is not None and h is not None:
        if w / h > ar:
            pw, ph = h * ar, h
        else:
            pw, ph = w, w / ar
        px, py = x + (w - pw) / 2, y + (h - ph) / 2
    elif w is not None:
        pw, ph, px, py = w, w / ar, x, y
    else:
        pw, ph, px, py = h * ar, h, x, y
    return s.shapes.add_picture(path, Inches(px), Inches(py), Inches(pw), Inches(ph))


def tile(s, x, y, w, h, big, label, fill=NAVY, bigc=TEAL, labc=PALE, bigsize=28):
    rect(s, x, y, w, h, fill=fill, shape=MSO_SHAPE.ROUNDED_RECTANGLE, radius=0.08)
    text(s, x + 0.15, y + 0.1, w - 0.3, h * 0.55, big, size=bigsize, color=bigc, bold=True,
anchor=MSO_ANCHOR.MIDDLE)
    text(s, x + 0.15, y + h * 0.58, w - 0.3, h * 0.4, label, size=10.5, color=labc)


def card(s, x, y, w, h, title, body, fill=LIGHT, line=LINE, tcolor=NAVY, size=11):
    rect(s, x, y, w, h, fill=fill, line=line, shape=MSO_SHAPE.ROUNDED_RECTANGLE, radius=0.04)
    text(s, x + 0.15, y + 0.1, w - 0.3, 0.35, title, size=12.5, color=tcolor, bold=True)
    if isinstance(body, list):
        bullets(s, x + 0.12, y + 0.5, w - 0.25, h - 0.55, body, size=size, gap=3)
    else:
        text(s, x + 0.15, y + 0.5, w - 0.3, h - 0.55, body, size=size, color=INK)


def table(s, x, y, w, rows, header, widths, size=10.5, row_h=0.34, header_fill=NAVY, zebra=True,
bold_first=False):
    nrows, ncols = len(rows) + 1, len(header)
    shp = s.shapes.add_table(nrows, ncols, Inches(x), Inches(y), Inches(w), Inches(row_h * nrows))
    tb = shp.table
    tot = sum(widths)
    for i, wd in enumerate(widths):
        tb.columns[i].width = Inches(w * wd / tot)
    for r_i in range(nrows):
        tb.rows[r_i].height = Inches(row_h)
        for c_i in range(ncols):
            cell = tb.cell(r_i, c_i)
            cell.margin_left = cell.margin_right = Inches(0.06)
            cell.margin_top = cell.margin_bottom = Inches(0.025)
            cell.vertical_anchor = MSO_ANCHOR.MIDDLE
            val = header[c_i] if r_i == 0 else rows[r_i - 1][c_i]
            tf = cell.text_frame; tf.word_wrap = True
            p = tf.paragraphs[0]
            for part in re.split(r"(\*\*[^*]+\*\*|`[^`]+`)", str(val)):
                if not part:
                    continue
                r = p.add_run()
                if part.startswith("**"):
                    r.text = part[2:-2]; r.font.bold = True
                elif part.startswith("`"):
                    r.text = part[1:-1]; r.font.name = "Consolas"; r.font.size = Pt(size - 1.5)
                    r.font.color.rgb = TEALD if r_i else WHITE
                    continue
                else:
                    r.text = part
                r.font.name = FONT; r.font.size = Pt(size)
                if r_i == 0:
                    r.font.bold = True; r.font.color.rgb = WHITE
                else:
```

```
                    r.font.color.rgb = INK
                    if bold_first and c_i == 0:
                        r.font.bold = True
            cell.fill.solid()
            cell.fill.fore_color.rgb = header_fill if r_i == 0 else (LIGHT if (zebra and r_i % 2 ==
0) else WHITE)
    return shp


count = [0]


def slide(kicker, title, notes, dark=False):
    s = prs.slides.add_slide(BLANK)
    count[0] += 1
    n = count[0]
    if dark:
        rect(s, 0, 0, 13.333, 7.5, fill=NAVY2)
    box = rect(s, 0.55, 0.42, 0.36, 0.36, fill=TEALD if not dark else TEAL,
shape=MSO_SHAPE.ROUNDED_RECTANGLE, radius=0.15)
    text(s, 0.55, 0.42, 0.36, 0.36, f"{n:02d}", size=10, color=WHITE, bold=True,
align=PP_ALIGN.CENTER, anchor=MSO_ANCHOR.MIDDLE)
    text(s, 1.05, 0.44, 11.7, 0.34, kicker.upper(), size=9.5, color=TEALD if not dark else TEAL,
bold=True, anchor=MSO_ANCHOR.MIDDLE)
    text(s, 0.55, 0.86, 12.2, 0.95, title, size=23, color=NAVY if not dark else WHITE, bold=True,
anchor=MSO_ANCHOR.TOP, line_sp=0.95)
    text(s, 0.55, 7.08, 8, 0.28, "NMIMS MPSTME  ·  Image & Video Processing", size=9, color=MUTED if
not dark else PALE)
    text(s, 12.2, 7.08, 0.6, 0.28, f"{n:02d}", size=9, color=MUTED if not dark else PALE,
align=PP_ALIGN.RIGHT)
    s.notes_slide.notes_text_frame.text = notes
    NOTES.append((n, title, notes))
    return s


def src(s, txt, y=6.78):
    text(s, 0.55, y, 12.2, 0.28, txt, size=8.5, color=MUTED, italic=True)


s = prs.slides.add_slide(BLANK)
count[0] += 1
rect(s, 0, 0, 13.333, 7.5, fill=NAVY2)
for k, r_ in enumerate([5.2, 4.2, 3.2, 2.2, 1.2]):
    c = rect(s, 10.0 - r_ / 2 + 0.2, 3.6 - r_ / 2, r_, r_, fill=None, line=NAVY3 if k % 2 else
RGBColor(0x1F, 0x3A, 0x6B), shape=MSO_SHAPE.OVAL, lw=1)
rect(s, 10.13, 3.53, 0.14, 0.14, fill=TEAL, shape=MSO_SHAPE.OVAL)
text(s, 0.8, 0.7, 9, 0.35, "NMIMS  MPSTME   ·   B.TECH COMPUTER ENGINEERING", size=10.5, color=TEAL,
bold=True)
text(s, 0.8, 1.7, 9, 1.8, ["Digital Image", "Watermarking"], size=50, color=WHITE, bold=True,
line_sp=0.9)
text(s, 0.8, 3.65, 8.6, 0.9, "A hybrid DWT-DCT-SVD approach - extended evaluation on 100 images,
controls, technology comparison and a video feasibility test", size=15, color=PALE)
rect(s, 0.8, 4.75, 5.95, 0.42, fill=None, line=TEAL, shape=MSO_SHAPE.ROUNDED_RECTANGLE, radius=0.3)
text(s, 0.8, 4.75, 5.95, 0.42, "CLASSICAL SIGNAL PROCESSING  -  NO MACHINE LEARNING / DEEP LEARNING",
size=9.5, color=TEAL, bold=True, align=PP_ALIGN.CENTER, anchor=MSO_ANCHOR.MIDDLE)
text(s, 0.8, 5.95, 9, 0.9, ["**Krishang Vikram Singh**", "Roll No. B106  ·  Batch C1", "Subject:
Image and Video Processing (IVP)  ·  First submission July 2026 · extended edition October 2026"],
size=11.5, color=WHITE, spacing=2)
text(s, 0.55, 7.08, 8, 0.28, "NMIMS MPSTME  ·  Image & Video Processing", size=9, color=PALE)
s.notes_slide.notes_text_frame.text = ("Opening. This is the extended version of my watermarking
project. The first submission tested a hybrid DWT-DCT-SVD scheme on two images. "
    "Since then I audited the code, rebuilt the evaluation on 100 unique images, added control
experiments, compared the scheme with existing technology, and tried it on real video. "
    "I will show what held up, what did not, and what I corrected.")
NOTES.append((1, "Title", s.notes_slide.notes_text_frame.text))

s = slide("Introduction · motivation", "Copying an image takes a second - proving who owns it is the
hard part",
          "Motivation: images are copied in one click. An invisible watermark carries an ownership
signal inside the pixels. Three constraints must hold at once: imperceptible, robust, explainable. "
          "The project's third constraint is explainability: every step is a closed-form linear-
algebra operation, no learned component. In this extended work I test whether the first two
constraints hold at scale.")
card(s, 0.55, 2.0, 3.95, 2.55, "Imperceptible", "The watermark must not visibly change the image.
Measured with PSNR and SSIM against the **unwatermarked original**.", size=14)
card(s, 4.69, 2.0, 3.95, 2.55, "Robust", "It must survive compression, noise, filtering and geometric
change. Measured with NC / ZNCC / BER of the recovered logo.", size=14)
```

```
card(s, 8.83, 2.0, 3.95, 2.55, "Explainable", "DWT, DCT and SVD are closed-form and verifiable by
hand - no black-box model. A deliberate scope choice, not a claim of superiority.", size=14)
rect(s, 0.55, 4.85, 12.23, 1.75, fill=NAVY, shape=MSO_SHAPE.ROUNDED_RECTANGLE, radius=0.05)
text(s, 0.85, 4.95, 11.6, 0.35, "WHAT THIS EXTENDED WORK ADDS", size=10, color=TEAL, bold=True)
bullets(s, 0.85, 5.3, 11.7, 1.3, ["**100 unique images** instead of two, with seeded attacks, status
logging and 43 unit tests",
        "**Controls** that ask whether a high NC really proves the watermark is present",
        "**Technology comparison** (libraries and an existing watermarking package) and an
exploratory **video** test"], size=12.5, color=WHITE, gap=3)

s = slide("Objectives and scope", "Six objectives - three inherited from the first submission, three
new",
          "Objectives one to three are from the first submission: implement, measure
imperceptibility, measure robustness. The new ones are: characterise alpha, find and explain failure
modes including false positives, and compare with existing technology and video. "
          "Scope boundaries matter: grey-scale only, no machine learning, non-blind scheme, video
exploratory.")
table(s, 0.55, 1.95, 7.6, [
    ["1", "Implement the hybrid DWT-DCT-SVD embed/extract pipeline", "inherited"],
    ["2", "Quantify imperceptibility (PSNR, SSIM)", "inherited · now 100 images"],
    ["3", "Quantify robustness (NC, BER + ZNCC) under 14 attacks and a no-attack reference",
"inherited · now 100 images"],
    ["4", "Characterise the α trade-off", "re-done, 4 attacks"],
    ["5", "Find failure modes - including false positives (controls)", "**new**"],
    ["6", "Compare with libraries and an existing package; test video", "**new**"],
], ["#", "Objective", "Status"], [0.4, 5.2, 2.0], size=11, row_h=0.52)
card(s, 8.45, 1.95, 4.33, 3.55, "Scope boundaries", [
    "8-bit **grey-scale** images (colour only via luminance)",
    "**No** machine / deep learning",
    "Scheme is **non-blind**: needs a key with the original singular values",
    "Video = **exploratory** (2 clips × 60 frames)",
    "Only operations that exist in the code are documented"], size=13)
rect(s, 0.55, 5.75, 12.23, 0.85, fill=TEALBG, line=TEAL, shape=MSO_SHAPE.ROUNDED_RECTANGLE,
radius=0.08)
text(s, 0.8, 5.8, 11.8, 0.75, "**Honesty rule:** every number in this deck comes from a recorded
experiment file; limits, unverified claims and corrections to the first submission are stated
explicitly.", size=12.5, color=NAVY, anchor=MSO_ANCHOR.MIDDLE)

s = slide("Existing technology · literature", "From LSB to hybrids - and an old warning about SVD
schemes",
          "A short timeline. Spatial LSB marks by van Schyndel et al. 1994 are fragile. Cox et al.
1997 put a spread-spectrum watermark into significant spectral coefficients. "
          "Liu and Tan 2002 introduced SVD ownership watermarking; Ganic and Eskicioglu 2004 combined
DWT and SVD. Later hybrids cascade DWT, DCT and SVD. "
          "The important warning: comments by Zhang and Li in 2005 and Ting in 2005 concern false
positives and ambiguity attacks on this family. I verified the bibliographic details of each
reference; "
          "I did not re-run those papers' experiments. The claim in my first report that hybrids
always beat single transforms is a literature trend, not something I tested.")
xs = [0.55, 3.0, 5.45, 7.9, 10.35]
items = [("1994", "LSB marking", "van Schyndel et al. [10]", "high capacity, fragile"), ("1997",
"Spread spectrum", "Cox et al. [1]", "significant spectral coefficients"),
         ("2002-04", "SVD & DWT-SVD", "Liu & Tan [2]; Ganic & Eskicioglu [3]", "singular-value
embedding"),
         ("2010-22", "Hybrid DWT-DCT-SVD", "[7] [8] [9]", "reported gains; not re-tested here"),
("2005", "False-positive critique", "Zhang & Li [4]; Ting [5]", "key owner's mark can be 'found'
anywhere")]
rect(s, 0.8, 2.52, 11.8, 0.05, fill=LINE)
for x, (yr, head, who, note) in zip(xs, items):
    red = head.startswith("False")
    rect(s, x + 0.9, 2.4, 0.28, 0.28, fill=RED if red else TEAL, shape=MSO_SHAPE.OVAL)
    text(s, x, 1.95, 2.2, 0.35, yr, size=12, color=RED if red else TEALD, bold=True,
align=PP_ALIGN.CENTER)
    card(s, x, 2.95, 2.3, 2.1, head, [who, note], size=12, fill=REDBG if red else LIGHT, line=RED if
red else LINE)
rect(s, 0.55, 5.35, 12.23, 1.3, fill=NAVY, shape=MSO_SHAPE.ROUNDED_RECTANGLE, radius=0.06)
bullets(s, 0.85, 5.45, 11.7, 1.2, ["Gap 1 - **identity binding**: SVD schemes can report the key
owner's logo whatever was embedded (measured on slide 15)",
        "Gap 2 - **geometry**: global embedding breaks under cropping/rotation, the standard remedy
being invariant or block-based domains [22]",
        "Gap 3 - **evaluation breadth**: published hybrids are typically shown on a few standard
images; here 100"], size=11.5, color=WHITE, gap=2)
src(s, "Bibliographic details verified against publisher / secondary records; status flags in
research/REFERENCES.md.", 6.72)

s = slide("System architecture · embedding", "Embedding: add the logo's singular values to the host's
- once, by the owner",
```

```
          "The embedding pipeline: host image, one-level Haar DWT, take the LL band, apply a 2-D DCT
to the whole band, take the SVD, add alpha times the singular values of the logo, and invert the
transforms. "
          "Two corrections to the first submission are visible here: the DCT is global, not block-
wise. The result is an 8-bit image that looks like the original; the owner keeps a key.")
picture(s, V("diagram_embedding.png"), 0.55, 1.9, 12.2, 4.7)
src(s, "Correction vs first submission: the DCT is applied to the whole LL band (not 8×8 blocks).
Alpha = 0.10 in the main experiment.", 6.72)

s = slide("System architecture · verification", "Extraction needs the owner's key - and rebuilds the
owner's logo from it",
          "Extraction repeats DWT, DCT and SVD on the received, possibly attacked image and subtracts
the stored original singular values, divides by alpha and rebuilds the logo from the key's singular
vectors. "
          "Second correction: the scheme is non-blind, not semi-blind, because the key contains the
original singular values. And note the consequence we will measure later: because the logo's singular
vectors are in the key, the extractor returns the key owner's logo whatever singular values it
finds.")
picture(s, V("diagram_extraction.png"), 0.55, 1.9, 12.2, 4.7)
src(s, "Correction vs first submission: extraction is non-blind (needs S_orig of the unmarked host);
the key holds U_w and V_w, i.e. the logo's identity.", 6.72)

s = slide("Mathematical foundations", "One tunable parameter: add α times the logo's singular
values",
          "The mathematics in five lines. Haar DWT gives the approximation band. A global orthonormal
DCT-II moves it to a frequency domain. SVD factors the DCT matrix and the logo. "
          "The embedding rule adds alpha times the logo's singular values to the host's singular
values. Extraction subtracts the stored original singular values and divides by alpha, then rebuilds
the logo from the key's singular vectors. "
          "Important consequence: the identity of the logo lives in the key, not in the image.")
eqs = [("DWT (Haar, LL)", "LL = (a + b + c + d) / 2   for each 2×2 block"), ("DCT-II (global,
orthonormal)", "F(u,v) = α(u)α(v) ΣΣ f(x,y) cos[(2x+1)uπ/2M] cos[(2y+1)vπ/2N]"),
       ("SVD", "A = U·diag(S)·Vᵀ        W = U_w·diag(S_w)·V_wᵀ"), ("Embedding", "S′ = S + α·S_w ,   A
′ = U·diag(S′)·Vᵀ  →  IDCT → IDWT → clip"),
       ("Extraction (key: S_orig, U_w, V_w, α)", "S_w′ = (S_att - S_orig) / α ,   W′ = U_w·diag(S_w
′)·V_wᵀ")]
for i, (h, e) in enumerate(eqs):
    y = 1.95 + i * 0.82
    rect(s, 0.55, y, 8.2, 0.7, fill=LIGHT, line=LINE, shape=MSO_SHAPE.ROUNDED_RECTANGLE, radius=0.08)
    text(s, 0.7, y + 0.03, 7.9, 0.28, h.upper(), size=9, color=TEALD, bold=True)
    text(s, 0.7, y + 0.28, 7.9, 0.4, e, size=14, color=NAVY, bold=False)
card(s, 9.05, 1.95, 3.73, 4.05, "What follows from the maths", [
    "**Non-blind:** needs S_orig of the unmarked host",
    "**Global DCT:** not tied to JPEG's 8×8 blocks - JPEG robustness comes from SVD",
    "**Identity in the key:** `U_w`, `V_w` rebuild *the owner's* logo from any singular values",
    f"Metric caution: a blank white image already scores NC =
**{BASE['nc_constant_white_image']:.2f}** against this logo"], size=12.5)
src(s, "Equations (1)-(5) of the report; numerics unchanged from the first submission (re-run
reproduces its table within 0.004 NC).", 6.72)

s = slide("Transformations implemented", "Twenty documented operations - only those that exist in the
code",
          "Every operation in the code is documented in TRANSFORMATIONS_REFERENCE with its
mathematics, parameters, limitations and complexity. The core is T2 to T5. "
          "T8 to T17 are the attacks used for testing. T20, video, is new and exploratory. Generic
operations such as Sobel, CLAHE or morphology are not in this project and are not claimed.")
table(s, 0.55, 1.95, 12.23, [
    ["T1", "Grey conversion, centre-crop, Lanczos resize", "Pillow", "T11-T12", "Median 3×3, Gaussian
blur σ=1.5"],
    ["T2", "1-level Haar DWT / inverse", "PyWavelets", "T13-T15", "Rotation 5°/15°, border cropping,
0.5× scaling"],
    ["T3", "2-D DCT-II / inverse (global)", "SciPy", "T16-T17", "Histogram equalisation, unsharp
mask"],
    ["T4", "SVD of host sub-band and logo", "NumPy", "T18", "PSNR, SSIM, MSE"],
    ["T5", "Singular-value embedding and extraction", "NumPy", "T19", "NC, ZNCC (new), BER"],
    ["T6-T7", "Anti-aliased resize; procedural logo", "scikit-image, Pillow", "T20", "Frame I/O, per-
frame embed, codec round trip (new)"],
    ["T8", "JPEG Q=90/50/20", "Pillow", "T9-T10", "Gaussian σ²=0.005/0.02; salt-and-pepper 2 %"],
], ["#", "Transformation", "Library", "#", "Transformation"], [0.7, 4.2, 1.6, 0.9, 4.2], size=10.5,
row_h=0.5)
rect(s, 0.55, 5.9, 12.23, 0.75, fill=TEALBG, line=TEAL, shape=MSO_SHAPE.ROUNDED_RECTANGLE,
radius=0.08)
text(s, 0.8, 5.93, 11.8, 0.7, "Pipeline order matters: DWT → DCT → SVD → embed → **round to 8 bit** →
attack → DWT → DCT → SVD → extract. The first submission attacked the float image; the 8-bit protocol
is primary here (difference < 0.001 mean NC except histogram equalisation).", size=11.5, color=NAVY,
anchor=MSO_ANCHOR.MIDDLE)
```

```
s = slide("Dataset design", f"{len(META)} unique images: {int((META.is_synthetic == 0).sum())} real,
{int((META.is_synthetic == 1).sum())} labelled synthetic - rights and provenance recorded",
          "The dataset: 94 real images from open sample collections hosted on GitHub - OpenCV,
scikit-image, torchvision, three.js textures and others - plus six synthetic controls. "
          "They are grey-standardised, square-cropped and resized to 512. Near duplicates were
rejected by a hash, and a test asserts uniqueness. "
          "Important caveat: only raw GitHub and PyPI were reachable, so this is a convenience
sample, not a benchmark, and rights for most photographs are not documented upstream - I record
exactly what upstream says.")
picture(s, V("dataset_contact_sheet.png"), 0.55, 1.9, 7.0, 4.85)
cat = META.groupby("category").size().sort_values(ascending=False)
tile(s, 7.85, 1.95, 1.55, 1.15, "94", "real images", bigsize=26)
tile(s, 9.5, 1.95, 1.55, 1.15, "6", "synthetic controls", bigsize=26)
tile(s, 11.15, 1.95, 1.63, 1.15, "18", "categories", bigsize=26)
card(s, 7.85, 3.3, 4.93, 3.3, "Provenance, diversity, limits", [
    "Sources: OpenCV (+extra), scikit-image, darknet, torchvision, three.js, Pillow tests, …",
    f"Brightness: {', '.join(f'{k} {v}' for k, v in META.brightness_class.value_counts().items())};
contrast: {', '.join(f'{k} {v}' for k, v in META.contrast_class.value_counts().items())}",
    "SHA-256, URL, licence and rights note per image in `metadata.csv`",
    "Convenience sample - **not** a benchmark; colour removed; 1-4 images in several categories"],
size=12)

s = slide("Quantitative evaluation methodology", "Seven experiments, 14 attacks plus a reference,
four metrics - all seeded and logged",
          "E1 imperceptibility, E2 robustness with fifteen conditions - fourteen attacks plus the
unattacked reference, which the first report counted as an attack - under two protocols, "
          "E3 an alpha sweep with four attacks, E4 false-positive controls. S1 and S2 are the
technology comparisons and V1 is the video test. Metrics: PSNR and SSIM for imperceptibility; NC,
ZNCC and BER for the recovered logo. "
          "The 0.75 threshold is the project's convention; I could not find a primary literature
source for it, and I measure its false-alarm rate in E4.")
table(s, 0.55, 1.95, 7.9, [
    ["E1", "embed α = 0.10, save 8-bit PNG, verify", "100 images"],
    ["E2", "15 conditions × 2 protocols", "3000 trials"],
    ["E3", "α sweep × 4 attacks", "4000 rows"],
    ["E4", "false-positive controls (a-d)", "1600 rows"],
    ["S1/S2", "scheme / component benchmarks", "30 imgs / 10 ops"],
    ["V1", "video: 2 clips × 60 frames × 6 codecs", "720 frames"],
], ["ID", "What", "Size"], [0.8, 5.0, 2.0], size=11, row_h=0.46)
card(s, 8.75, 1.95, 4.03, 3.2, "Metrics", [
    "**PSNR / SSIM** vs the unwatermarked original",
    "**NC**, **ZNCC** (mean-centred), **BER** vs the logo",
    "Recognised if NC ≥ 0.75 (project convention)",
    "95 % bootstrap CIs; Wilson intervals for pass rates"], size=13)
rect(s, 0.55, 5.35, 12.23, 1.3, fill=NAVY, shape=MSO_SHAPE.ROUNDED_RECTANGLE, radius=0.06)
text(s, 0.85, 5.42, 11.7, 1.2, [f"**{RS['images_ok']}/{RS['images_requested']}** images ·
**{RS['attack_trials_ok']}/{RS['attack_trials_total']}** attack trials ·
**{RS['alpha_sweep_rows_ok']}/{RS['alpha_sweep_rows_total']}** sweep rows - **0 failures**",
        "Two full runs gave **identical metrics** (max |Δ| = 0.0) · 43 tests pass · ≈ 80 s on 4
cores"], size=13, color=WHITE, spacing=4)

s = slide("Results · imperceptibility", "Invisible on typical images - but PSNR alone hides some
failures",
          f"Across 100 images the mean PSNR is {E1['psnr_all']['mean']:.1f} dB and mean SSIM
{E1['ssim_all']['mean']:.3f}. Notice that PSNR is almost the same for most images, about 35.6 dB, "
          "because the added energy is set by the logo's singular values, not by the host. SSIM
exposes exceptions: the near-black Milky Way image has PSNR 35.9 but SSIM 0.58, and the checkerboard
0.86. "
          "So PSNR alone would have hidden these failures.")
tile(s, 0.55, 1.95, 2.3, 1.2, f"{E1['psnr_all']['mean']:.1f} dB", "mean PSNR (min 35.5)", bigsize=28)
tile(s, 0.55, 3.3, 2.3, 1.2, f"{E1['ssim_all']['mean']:.3f}", f"mean SSIM · {E1['n_ssim_below_0.98']}
images < 0.98", bigsize=28)
tile(s, 0.55, 4.65, 2.3, 1.2, "0", "images below 30 dB", bigsize=28)
picture(s, V("fig_imperceptibility_hist.png"), 3.05, 1.9, 5.3, 2.2)
picture(s, V("fig_representative_comparison.png"), 8.6, 1.85, 4.2, 4.9)
bullets(s, 3.05, 4.25, 5.4, 2.4, ["PSNR ≈ 35.6 dB for **82 of 100** images - set by the logo, not the
host",
        "**Near-black sky** (`image_055`): PSNR 35.9 dB but **SSIM 0.58** - clipping dominates",
        "Checkerboard: SSIM 0.86; synthetic images reach 43.8 dB"], size=11.5, gap=4)
src(s, "Right: original · watermarked · |difference|×10 for typical images and two failure cases (E1,
100 images).", 6.78)

s = slide("Results · robustness under attack", f"{pc(E2['pass_rate'], 1)} of trials recovered -
strong for compression and filtering, weak for cropping and equalisation",
          f"Headline: {E2['n_pass']} of 1500 trials reach NC at least 0.75 when the unattacked
reference is included, as in the first submission, and {E2['n_pass_attacks_only']} of
{E2['n_trials_attacks_only']} counting only the 14 attacks. "
```

```
          "Very robust: JPEG, median filter, sharpening, scaling, salt and pepper, blur, mild noise.
Moderate: five and fifteen degree rotation, strong noise. "
          "Weak: histogram equalisation, 44 percent, and border cropping, 1 percent. The first
submission said moderate rotation and scaling had mean NC above 0.93; with 100 images that holds for
scaling, 0.938, but not for 5 degree rotation, 0.880.")
picture(s, V("fig_nc_boxplot.png"), 0.55, 1.85, 7.4, 4.85)
cols = [("JPEG Q=50", "JPEG Q=50"), ("Median 3×3", "Median Filter 3x3"), ("Scaling 0.5×", "Scaling
0.5x"), ("Noise σ²=0.02", "Gaussian Noise (var=0.02)"),
        ("Rotation 15°", "Rotation 15 deg"), ("Hist. equalisation", "Histogram Equalization"),
("Border cropping", "Cropping 25%")]
rows = [[a, f"{ROB.loc[b].mean_nc:.3f}", f"{100 * ROB.loc[b].pass_rate:.0f} %"] for a, b in cols]
table(s, 8.2, 1.95, 4.58, rows, ["Attack", "mean NC", "NC ≥ 0.75"], [2.2, 1.1, 1.3], size=11,
row_h=0.42)
rect(s, 8.2, 5.5, 4.58, 1.15, fill=TEALBG, line=TEAL, shape=MSO_SHAPE.ROUNDED_RECTANGLE, radius=0.08)
text(s, 8.35, 5.53, 4.3, 1.1, "First submission: \"rotation/scaling NC > 0.93\". At n = 100: scaling
0.938 ✓, **5° rotation 0.880 ✗**.", size=11, color=NAVY, anchor=MSO_ANCHOR.MIDDLE)
src(s, f"1500 trials = 100 images × 15 conditions (14 attacks + reference); attacks only:
{E2['n_pass_attacks_only']}/{E2['n_trials_attacks_only']} = {pc(E2['pass_rate_attacks_only'], 1)}.",
6.78)

s = slide("Results · which images fail?", "Robustness depends on the image: higher-contrast images
lose more under noise and rotation",
          "Across the 100 images, NC after noise and rotation is negatively correlated with image
contrast - Spearman rho about minus 0.65 for strong Gaussian noise, minus 0.68 for salt and pepper
and minus 0.55 for 15 degree rotation. "
          "Histogram equalisation instead correlates with mean brightness. These are associations,
with uncorrected p-values and n of 100, not demonstrated causes. "
          "By category, the two document-text images are the weakest, portraits and documents are
most sensitive to equalisation; several categories have only one to four images so I treat them as
descriptive.")
picture(s, V("fig_nc_vs_contrast.png"), 0.55, 1.9, 6.4, 2.25)
picture(s, V("fig_category_heatmap.png"), 7.2, 1.85, 5.6, 4.9)
bullets(s, 0.55, 4.35, 6.4, 2.4, [f"Noise σ²=0.02: ρ(contrast) = **{COR.loc['Gaussian Noise
(var=0.02)'].rho_contrast:.2f}**; salt-and-pepper **{COR.loc['Salt & Pepper
(2%)'].rho_contrast:.2f}**; 15° rotation **{COR.loc['Rotation 15 deg'].rho_contrast:.2f}**",
        f"Hist. equalisation tracks mean brightness (ρ = {COR.loc['Histogram
Equalization'].rho_mean_intensity:.2f}), not contrast",
        "Weakest categories: **document text** (noise 0.53) and **portraits** under equalisation
(0.40)",
        "n = 100, uncorrected p-values - associations, not causes"], size=11.5, gap=4)

s = slide("Results · embedding strength", "The α trade-off is real for noise and rotation - but flat
for JPEG, so the earlier figure misled",
          "This slide corrects the first submission. Its trade-off figure used two y-axes with the NC
axis zoomed to 0.93 to 0.99, and its JPEG simulation truncated floats, which lowered NC only at alpha
0.02. "
          "I re-created its conditions exactly and got the same numbers, then showed that rounding
removes the dip. With 100 images and four attacks the picture is clear: after JPEG the NC is flat, "
          "but under noise, blur and rotation robustness grows with alpha while PSNR falls. For
Gaussian noise the pass rate goes from 9 percent at alpha 0.02 to 85 percent at 0.10 and 95 percent
at 0.40, while PSNR falls below 30 dB from alpha 0.20.")
picture(s, V("fig_alpha_tradeoff.png"), 0.55, 1.9, 12.2, 3.6)
g = "Gaussian Noise (var=0.02)"
tile(s, 0.55, 5.65, 2.9, 1.05, f"{pc(sw(g, 0.02, 'pass_rate'))} → {pc(sw(g, 0.10, 'pass_rate'))}",
"noise pass rate, α 0.02 → 0.10", bigsize=22)
tile(s, 3.6, 5.65, 2.9, 1.05, f"{sw('No Attack', 0.10, 'psnr'):.1f} → {sw('No Attack', 0.20,
'psnr'):.1f} dB", "PSNR, α 0.10 → 0.20", bigsize=22)
tile(s, 6.65, 5.65, 2.9, 1.05, f"{sw('JPEG Q=50', 0.02, 'nc_mean'):.2f} → {sw('JPEG Q=50', 0.40,
'nc_mean'):.2f}", "mean NC after JPEG Q=50, α 0.02 → 0.40", bigsize=22)
rect(s, 9.7, 5.65, 3.08, 1.05, fill=REDBG, line=RED, shape=MSO_SHAPE.ROUNDED_RECTANGLE, radius=0.08)
text(s, 9.82, 5.68, 2.85, 1.0, "Old figure: axis 0.93-0.99 + float truncation before JPEG →
**artefact** (`exp_jpeg_truncation.py`)", size=10.5, color=INK, anchor=MSO_ANCHOR.MIDDLE)

s = slide("Results · false-positive controls", f"A different logo decoded with the owner's key still
scores NC {CC.mean_nc:.2f} - NC does not prove ownership",
          "This is the most important finding. Control C: embed a different logo in the same host and
decode with the key of logo A. The extractor returns the key owner's logo, NC 0.975 against 0.977 for
the genuine pair. "
          "That is the false-positive problem described for SVD schemes in the literature, now
measured for this implementation. Controls A and B: unrelated images decoded with a wrong key score
0.37 to 0.40 on average, "
          "and about 10 to 13 percent exceed the 0.75 threshold. A claim of ownership therefore needs
an additional safeguard such as an image-bound signature; I did not implement one. "
          "I also tried embedding the principal component instead; that made things worse, the
unwatermarked image still decoded at 0.98.")
picture(s, V("fig_false_positive.png"), 0.55, 1.9, 6.3, 3.4)
picture(s, V("fig_controls_hist.png"), 6.95, 1.9, 5.85, 3.15)
```

```
tile(s, 0.55, 5.5, 2.95, 1.15, f"{CC.mean_nc:.3f}", "wrong logo, owner's key (genuine 0.977)",
bigc=RGBColor(0xFF, 0x8A, 0x80), bigsize=26)
tile(s, 3.65, 5.5, 2.95, 1.15, f"{pc(CA.frac_ge_0_75, 1)} / {pc(CB.frac_ge_0_75, 1)}", "unrelated-
image controls above NC 0.75", bigsize=22)
tile(s, 6.75, 5.5, 2.95, 1.15, f"{E4['wrong_image_p95']:.2f}", "95th percentile NC of unrelated
images", bigsize=26)
rect(s, 9.85, 5.5, 2.93, 1.15, fill=TEALBG, line=TEAL, shape=MSO_SHAPE.ROUNDED_RECTANGLE,
radius=0.08)
text(s, 9.97, 5.52, 2.7, 1.1, "Needed: an **image-bound signature** - not implemented; the principal-
component variant was **worse** (NC 0.98 on an unmarked image)", size=10, color=NAVY,
anchor=MSO_ANCHOR.MIDDLE)

s = slide("Results · failure case analysis", "Four failure modes - explained from the data, not
hidden",
          "Border cropping: mean NC 0.02; with 43.75 percent of the area zeroed every singular value
changes. The first report called this 25 percent cropping; 25 percent of each dimension is 43.75
percent of the area. "
          "Histogram equalisation: 44 percent pass; it varies between images, 0.10 for the cameraman
and 0.66 for the astronaut in the first submission. "
          "Near-constant images: SSIM collapses though PSNR is fine. Rotation: 72 percent pass at 15
degrees; there is no re-registration.")
card(s, 0.55, 1.95, 2.95, 3.9, "Border cropping", [f"mean NC **{ROB.loc['Cropping
25%'].mean_nc:.3f}**, pass **{pc(ROB.loc['Cropping 25%'].pass_rate)}**", "43.75 % of the *area*
zeroed (25 % per dimension)", "all singular values change; logo extracts as black (BER 0.14 ≈ white
fraction)"], fill=REDBG, line=RED, tcolor=RED, size=12.5)
card(s, 3.65, 1.95, 2.95, 3.9, "Histogram equalisation", [f"mean NC **{ROB.loc['Histogram
Equalization'].mean_nc:.3f}**, pass **{pc(ROB.loc['Histogram Equalization'].pass_rate)}**", "remaps
the whole intensity distribution", "varies by image (first report: 0.10 vs 0.66)", "tracks mean
brightness (ρ -0.31)"], fill=REDBG, line=RED, tcolor=RED, size=12.5)
card(s, 6.75, 1.95, 2.95, 3.9, "Rotation > 5°", [f"5°: **{pc(ROB.loc['Rotation 5 deg'].pass_rate)}**,
15°: **{pc(ROB.loc['Rotation 15 deg'].pass_rate)}**", "no re-registration attempted", "consistent
with RST theory [22]", "block-based schemes break too (blind-watermark BER 0.21)"], fill=REDBG,
line=RED, tcolor=RED, size=12.5)
card(s, 9.85, 1.95, 2.93, 3.9, "Near-constant images", ["near-black sky: PSNR **35.9 dB**, SSIM
**0.58**", "checkerboard: lowest clean NC (0.79)", "PSNR alone would have hidden both"], fill=REDBG,
line=RED, tcolor=RED, size=12.5)
rect(s, 0.55, 6.05, 12.23, 0.6, fill=NAVY, shape=MSO_SHAPE.ROUNDED_RECTANGLE, radius=0.1)
text(s, 0.8, 6.07, 11.8, 0.56, f"By contrast: JPEG Q ≥ 20, median 3×3, sharpening, 0.5× scaling,
blur, salt-and-pepper and mild noise all reach ≥ 95 % pass rate.", size=12, color=WHITE,
anchor=MSO_ANCHOR.MIDDLE)

s = slide("Comparison with existing technologies · components", "Same operation, different libraries:
numerically equal, speed rarely matters here",
          "Component benchmark: the same operation on the same input in different libraries, one
thread, in memory, warm-up and repeats. Results agree numerically: DCT and SVD to ten to the minus
sixteen, "
          "Pillow and OpenCV JPEG bit-identical. OpenCV is much faster for the attack operations -
about 340 times for the 3 by 3 median - partly because it works on uint8. But the whole 100-image
experiment takes about 80 seconds, "
          "so changing libraries is not justified on speed. ImageMagick as a command line tool costs
about 49 milliseconds per call, dominated by process start-up.")
picture(s, V("fig_tech_speedup.png"), 0.55, 1.95, 6.9, 4.2)
card(s, 7.65, 1.95, 5.13, 4.7, "What the measurements say", [
    "DCT / SVD / Haar agree to ≤ 1.5e-16 (DWT 1e-13); Pillow ≡ OpenCV JPEG",
    "SVD dominates embedding (≈ 23 of 37 ms); `scipy.linalg.svd` is **1.19×** NumPy's",
    "OpenCV: median **≈ 340×**, blur 4.4×, rotation 5.1×, rescale 38×, equalise 19× (partly `uint8`
vs `float64`)",
    "PyWavelets Haar is **11× slower** than four lines of NumPy - still the right general tool",
    "ImageMagick CLI ≈ **49 ms** per JPEG (process + disk)",
    "Whole experiment ≈ **80 s** on 4 cores → no performance case for switching"], size=11.5)
src(s, "Documentation-based capability matrix (not benchmarked) is in the report, Table 3.1.", 6.78)

s = slide("Comparison with existing technologies · schemes", "Beats LSB and block-DCT at equal PSNR -
but not an existing blind block-based hybrid",
          "Scheme-level comparison with the same 4095-bit payload and the same attacks on 30 images.
The block-DCT baseline was tuned to match the project's PSNR. LSB is imperceptible but destroyed by
any processing. "
          "The open-source blind-watermark package, which is also DWT-DCT-SVD but block-based and
blind, has lower bit-error rates under most attacks at higher PSNR, 42 against 36 dB. "
          "But it is 13 times slower to embed, 18 times slower to extract, and worse under rotation
and strong JPEG. Caveat: its YUV design and my channel-wise attacks may favour it. "
          "So the project is a simple, explainable, key-based alternative, not a better algorithm.")
picture(s, V("fig_tech_scheme_ber.png"), 0.55, 1.85, 7.9, 3.4)
rows = [["PSNR (dB)"] + [f"{psnr_s(x):.1f}" for x in (PROJ, BWM, BDCT, LSB)],
        ["JPEG Q=50 BER"] + [f"{ber(x, 'JPEG Q=50'):.3f}" for x in (PROJ, BWM, BDCT, LSB)],
        ["Rotation 5° BER"] + [f"{ber(x, 'Rotation 5 deg'):.3f}" for x in (PROJ, BWM, BDCT, LSB)],
        ["Cropping BER"] + [f"{ber(x, 'Cropping 25%'):.3f}" for x in (PROJ, BWM, BDCT, LSB)],
```

```
        ["embed / extract ms"] + [f"{tms(x, 't_embed_ms'):.0f} / {tms(x, 't_extract_ms'):.0f}" for x
in (PROJ, BWM, BDCT, LSB)],
        ["blind?"] + ["**no** (key)", "yes", "yes", "yes"]]
table(s, 0.55, 5.3, 7.9, rows, ["", "Project", "blind-watermark", "block DCT", "LSB"], [2.2, 1.5,
1.8, 1.4, 1.2], size=10, row_h=0.235)
card(s, 8.7, 1.95, 4.08, 4.7, "Reading the comparison", [
    "Project ≫ LSB and block-DCT under JPEG, scaling, rotation",
    f"`blind-watermark`: lower BER under JPEG Q=50 ({ber(BWM, 'JPEG Q=50'):.3f} vs {ber(PROJ, 'JPEG
Q=50'):.3f}), noise, blur, cropping; **higher PSNR** ({psnr_s(BWM):.1f} dB)",
    f"…but **{tms(BWM, 't_embed_ms') / tms(PROJ, 't_embed_ms'):.0f}× slower** and worse under
rotation ({ber(BWM, 'Rotation 5 deg'):.3f} vs {ber(PROJ, 'Rotation 5 deg'):.3f}) and JPEG Q=20",
    "Its colour/YUV design and channel-wise attacks may favour it",
    "**No claim of superiority** - contribution = explainable reference implementation + measured
weaknesses"], size=12)

s = slide("Exploratory video test", "Per-frame embedding survives H.264 and MJPEG re-encoding -
feasibility, not a video benchmark",
          "Because the project's future-scope slide mentioned video, I tested it: 60 frames from each
of two real clips, read with OpenCV, watermarked frame by frame, each with its own key, re-encoded
with FFmpeg as lossless FFV1, H.264 at three quality levels, and MJPEG at two. "
          "All 720 decoded frames gave NC at least 0.972. So lossy codecs are not the difficulty. The
caveats are important: keys are per frame, about 2 megabytes each; temporal attacks such as frame
dropping or re-timing were not tested; "
          "and a naive misaligned frame-drop test still gave NC 0.97, presumably because neighbouring
frames are similar - another sign that NC does not bind a mark to a frame.")
v = VID.groupby("codec", sort=False).agg(nc=("nc_mean", "mean"), ncmin=("nc_min", "min"),
psnr=("psnr_wm_vs_orig_db", "mean"), kb=("size_wm_kb", "mean")).reset_index()
rows = [[x.codec, f"{x.psnr:.1f}", f"{x.nc:.3f}", f"{x.ncmin:.3f}", f"{x.kb:.0f}"] for x in
v.itertuples()]
table(s, 0.55, 1.95, 7.6, rows, ["Codec setting", "PSNR (dB)", "mean NC", "min NC", "size (kB)"],
[2.6, 1.2, 1.2, 1.2, 1.2], size=11, row_h=0.46)
tile(s, 8.45, 1.95, 2.1, 1.2, "720", "decoded frames, 0 failed", bigsize=28)
tile(s, 10.68, 1.95, 2.1, 1.2, f"≥ {VID.nc_min.min():.2f}", "NC in every frame", bigsize=28)
card(s, 8.45, 3.35, 4.33, 2.7, "Caveats", ["2 clips × 60 frames - small", "one **key per frame** (≈ 2
MB)", "no frame dropping, re-timing, colour or hardware codecs", "naive frame-drop still NC 0.97
(similar neighbours)"], size=12, fill=REDBG, line=RED, tcolor=RED)
text(s, 0.55, 5.35, 7.6, 1.0, f"Embedding ≈ 32-41 ms per 512² frame on one core (≈ 24-31 frames/s) ·
clips: `vtest.avi` (768×576), `Megamind.avi` (720×528) from the OpenCV sample data", size=11,
color=MUTED)

s = slide("Claims versus evidence", "What held up and what did not: ten claims of the first
submission checked at n = 100",
          "A direct comparison of the first submission's claims with the 100-image evidence. The
headline numbers are reproducible and mostly hold. "
          "What changed: moderate rotation is weaker than claimed, the alpha trade-off figure was an
artefact for JPEG, the block-DCT and semi-blind labels were wrong, "
          "the suite is fourteen attacks plus a reference, and - most importantly - extraction does
not prove ownership.")
rows = [
    ["PSNR 35.6 dB / SSIM 0.996 (2 images)", f"{E1['psnr_all']['mean']:.1f} dB / {E1['ssim_all']
['mean']:.3f}; 3 images < 0.98", "✓ with exceptions"],
    ["25 / 30 trials ≥ 0.75", f"{E2['n_pass']} / 1500 ({pc(E2['pass_rate'], 1)}); attacks only
{pc(E2['pass_rate_attacks_only'], 1)}", "✓"],
    ["JPEG: NC > 0.97", f"{ROB.loc['JPEG Q=90'].mean_nc:.3f} / {ROB.loc['JPEG Q=50'].mean_nc:.3f} /
{ROB.loc['JPEG Q=20'].mean_nc:.3f} (Q 90/50/20)", "✓ (Q ≥ 50)"],
    ["Rotation/scaling NC > 0.93", f"scaling {ROB.loc['Scaling 0.5x'].mean_nc:.3f}; rotation 5°
{ROB.loc['Rotation 5 deg'].mean_nc:.3f}", "scaling ✓ · rotation ✗"],
    ["Cropping fails; hist-eq 0.10 / 0.66", f"pass {pc(ROB.loc['Cropping 25%'].pass_rate)}; hist-eq
mean {ROB.loc['Histogram Equalization'].mean_nc:.3f} (44 % pass)", "✓ (hist-eq varies)"],
    ["Robustness rises with α (JPEG)", f"flat {sw('JPEG Q=50', 0.02, 'nc_mean'):.3f} → {sw('JPEG
Q=50', 0.40, 'nc_mean'):.3f}", "✗ artefact"],
    ["\"Block DCT\", \"semi-blind\"", "global DCT, non-blind", "corrected"],
    ["\"Fifteen attacks\"", "14 attacks + the unattacked reference", "corrected"],
    ["Extraction proves ownership", f"wrong logo decoded at NC {CC.mean_nc:.3f}", "✗ **not
supported**"],
    ["Hybrids beat single transforms", "not tested; blind-watermark beats the project",
"unverified"],
]
table(s, 0.55, 1.9, 12.23, rows, ["First submission", "This evaluation (100 images)", "Verdict"],
[4.4, 5.6, 2.2], size=12, row_h=0.45)

s = slide("Implementation · quality · reproducibility", "Tested, seeded and logged: 43 tests, zero
failures, identical re-runs",
          "Implementation: the embed and extract numerics are unchanged, so the first submission's
numbers remain reproducible - I re-ran its experiment and got the same table within 0.004. "
          "Added: input validation, a safe image loader, a dataset builder, a multiprocessing batch
runner that records status and time per operation and re-opens every output, seeded attacks, and 43
tests. "
```

```
          "Bug found during development, for transparency: a debug flag overwrote the metadata file;
fixed, with a regression test.")
card(s, 0.55, 1.95, 4.0, 3.0, "Engineering", ["numerics of `embed/extract_watermark` unchanged",
"validation: 2-D, finite, size, α > 0", "`image_io`: alpha, 16-bit, corrupt/empty files", "config-
driven batch runner, 4 workers"], size=12.5)
card(s, 4.7, 1.95, 4.0, 3.0, "Verification", ["**43 tests** pass (functions, edge cases, loaders,
completeness)", "legacy re-run reproduces the first report (≤ 0.004 NC)", "two full runs: identical
metrics", "all 100 output PNGs re-opened and validated"], size=12.5)
card(s, 8.85, 1.95, 3.93, 3.0, "Traceability", ["originals preserved in `original_backup/`",
"CHANGELOG lists every change and why", "negative results saved: principal-component variant; JPEG
truncation", "PROJECT_AUDIT: 15 findings, F1-F15"], size=12.5)
rect(s, 0.55, 5.2, 12.23, 1.45, fill=NAVY, shape=MSO_SHAPE.ROUNDED_RECTANGLE, radius=0.05)
text(s, 0.85, 5.28, 11.7, 0.3, "REPRODUCE", size=10, color=TEAL, bold=True)
text(s, 0.85, 5.58, 11.7, 1.0, ["`python build_dataset.py  →  run_batch_experiment.py  →  analysis.py
→  benchmark_tech.py  →  video_demo.py`", "`python -m pytest tests -q`    ·   Python 3.13, NumPy 2.5,
SciPy 1.18, scikit-image 0.26, Pillow 12.3, OpenCV 5.0, FFmpeg 6.1"], size=11.5, color=WHITE,
spacing=3)

s = slide("Limitations · future scope", "Known limits - and the next steps the evidence supports",
          "Limitations: false positives, non-blind key, cropping and rotation, histogram
equalisation, grey-scale only, a lossy clean extraction of about 1 percent BER, and a dataset that is
a convenience sample. "
          "Future work is ordered by evidence: first bind the logo to the image; then block-based
redundant embedding for cropping; then geometric re-synchronisation; colour via luminance; and a
proper video design with shared or index-aware keys.")
card(s, 0.55, 1.95, 5.95, 4.7, "Limitations (measured)", [
    "**False-positive problem** - not fixed",
    "**Non-blind**: ≈ 2.1 MB key per image",
    "**Cropping** 1 %, **rotation 15°** 72 %, **hist-eq** 44 %",
    "Grey-scale only; clean BER ≈ 0.9 % (logo resampling)",
    "Dataset: convenience sample, rights mostly undocumented",
    "Only one existing package compared; no GPU; ±5-10 % timing noise",
    "Video: exploratory, per-frame keys, no temporal attacks"], size=13, fill=REDBG, line=RED,
tcolor=RED)
card(s, 6.7, 1.95, 6.08, 4.7, "Future scope (in priority order)", [
    "**1 · Image-bound signature** to remove false positives - verify with the E4 controls",
    "**2 · Block-based redundant embedding** for cropping; use `blind-watermark` as the new
baseline",
    "**3 · Geometric re-synchronisation** (log-polar / template) [22]",
    "**4 · Colour:** embed in the Y channel of YCbCr",
    "**5 · Video:** index-aware keys, temporal redundancy, frame-drop and re-timing tests; real-time
port (≈ 27 fps single-core as is)",
    "**6 ·** Learned watermarkers [29][30] as a comparison, if the no-ML rule is relaxed"], size=13)

s = slide("Conclusion", "A transparent scheme that survives everyday processing - but NC alone is not
proof of ownership",
          "Conclusion. The scheme is transparent and survives re-compression and filtering well: 85
percent of all trials, 84 percent of attack trials. "
          "Imperceptibility is good on typical images with exceptions. The extension changed the
story in four ways: the alpha trade-off exists only for harsher attacks, rotation and equalisation
are weaker than claimed, "
          "a high NC does not prove ownership, and an existing package beats the project on most
attacks. What I would do next is bind the logo to the image. Thank you; questions welcome.",
dark=True)
tile(s, 0.55, 1.95, 3.9, 1.5, f"{E1['psnr_all']['mean']:.1f} dB / {E1['ssim_all']['mean']:.3f}",
"mean PSNR / SSIM over 100 images", fill=NAVY3, bigsize=28)
tile(s, 4.7, 1.95, 3.9, 1.5, f"{pc(E2['pass_rate'], 1)}", "of 1500 trials with NC ≥ 0.75",
fill=NAVY3, bigsize=28)
tile(s, 8.85, 1.95, 3.93, 1.5, f"NC {CC.mean_nc:.2f}", "for a DIFFERENT logo with the owner's key",
fill=NAVY3, bigc=RGBColor(0xFF, 0x8A, 0x80), bigsize=28)
text(s, 0.55, 3.75, 12.2, 0.3, "KEY TAKEAWAYS", size=10, color=TEAL, bold=True)
bullets(s, 0.55, 4.1, 12.2, 2.7, ["**Works:** JPEG Q ≥ 20, median, sharpening, scaling, blur, mild
noise - ≥ 95 % pass rate; per-frame video re-encoding too",
        "**Does not:** border cropping (1 %), histogram equalisation (44 %), rotation 15° (72 %);
first report's α-trade-off figure was an artefact for JPEG",
        "**Cannot claim:** ownership from NC alone (false-positive problem) or superiority over
existing technology",
        "**Next:** bind the logo to the image, block-based redundancy, geometric re-
synchronisation"], size=15, color=WHITE, gap=9)

s = slide("References", "Verified sources (IEEE style) - status flags in research/REFERENCES.md",
          "References. All bibliographic details were checked against publisher or secondary records;
entries that could not be verified were not cited. The full list with verification status is in
research/REFERENCES.md. Thank you.")
refs = [
    "[1] I. J. Cox et al., “Secure spread spectrum watermarking for multimedia,” IEEE Trans. Image
Process., 6(12):1673-1687, 1997.",
```

```
    "[2] R. Liu and T. Tan, “An SVD-based watermarking scheme for protecting rightful ownership,”
IEEE Trans. Multimedia, 4:121-128, 2002.",
    "[3] E. Ganic and A. M. Eskicioglu, “Robust DWT-SVD domain image watermarking: embedding data in
all frequencies,” Proc. MM&Sec ’04, ACM, pp. 166-174, 2004.",
    "[4] X.-P. Zhang and K. Li, “Comments on ‘An SVD-based watermarking scheme…’,” IEEE Trans.
Multimedia, 7:593-594, 2005.",
    "[5] G. C.-W. Ting, “Ambiguity attacks on the Ganic-Eskicioglu DWT-SVD scheme,” Proc. ICISC, pp.
378-388, 2005.",
    "[7] C.-C. Lai and C.-C. Tsai, “Digital image watermarking using DWT and SVD,” IEEE Trans.
Instrum. Meas., 59(11):3060-3063, 2010.",
    "[8] S. Bekkouch and K. M. Faraoun, “…combined DCT-DWT-SVD transforms,” J. Inf. Process. Syst.,
11(3):406-420, 2015.",
    "[9] M. Begum, J. Ferdush, M. S. Uddin, “A hybrid robust watermarking system based on DCT, DWT
and SVD,” J. King Saud Univ. CIS, 34(8):5856-5867, 2022.",
    "[10] R. G. van Schyndel et al., “A digital watermark,” Proc. ICIP, vol. 2, pp. 86-90, 1994.
[11] E. Koch and J. Zhao, Proc. IEEE NSIP Workshop, pp. 452-455, 1995.",
    "[12] S. G. Mallat, IEEE Trans. PAMI, 11(7):674-693, 1989.   [13] N. Ahmed, T. Natarajan, K. R.
Rao, IEEE Trans. Comput., C-23(1):90-93, 1974.",
    "[16] Z. Wang et al., “Image quality assessment: from error visibility to structural similarity,”
IEEE Trans. Image Process., 13(4):600-612, 2004.",
    "[17] F. Hartung and M. Kutter, “Multimedia watermarking techniques,” Proc. IEEE, 87(7):1079-
1107, 1999.",
    "[22] C.-Y. Lin et al., “Rotation, scale, and translation resilient watermarking for images,”
IEEE Trans. Image Process., 10(5):767-782, 2001.",
    "[23] F. Hartung and B. Girod, “Watermarking of uncompressed and compressed video,” Signal
Process., 66(3):283-301, 1998.",
    "[28] `blind-watermark` 0.4.4, github.com/guofei9987/blind_watermark   ·   [29] J. Zhu et al.,
HiDDeN, ECCV 2018   ·   [30] M. Tancik et al., StegaStamp, CVPR 2020",
    "Software: NumPy [19], SciPy [20], scikit-image [21], PyWavelets [18], Pillow, OpenCV, FFmpeg,
ImageMagick.   Dataset sources and attributions: data/README.md.",
]
text(s, 0.55, 1.85, 12.2, 4.95, refs, size=11.5, color=INK, spacing=5)

prs.save(os.path.join(OUT, "Final_Presentation.pptx"))

with open(os.path.join(OUT, "SPEAKER_NOTES.md"), "w", encoding="utf-8") as f:
    f.write("# Speaker notes - Final_Presentation.pptx\n\nTarget length: ≈ 15-18 minutes for 24
slides (≈ 40-45 s per slide; slides 12, 14, 15 and 18 deserve more time).\n\n")
    for n, title, notes in NOTES:
        f.write(f"## Slide {n} - {title}\n\n{notes}\n\n")
print("slides:", count[0])
```

5.4 build_paper.py

Builds the research paper. 395 lines.

```
from __future__ import annotations

import json
import os
import re
import subprocess

import pandas as pd
from docx import Document
from docx.enum.section import WD_SECTION
from docx.enum.table import WD_TABLE_ALIGNMENT
from docx.enum.text import WD_ALIGN_PARAGRAPH, WD_BREAK, WD_TAB_ALIGNMENT
from docx.oxml import OxmlElement
from docx.oxml.ns import qn
from docx.shared import Cm, Pt, RGBColor

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
RES, TAB, FIG = os.path.join(ROOT, "results"), os.path.join(ROOT, "results", "tables"),
os.path.join(ROOT, "paper", "figs")
OUTDIR = os.path.join(ROOT, "paper")
ROLL = "B106"
FONT = "Times New Roman"

AS = json.load(open(os.path.join(RES, "analysis_summary.json")))
RS = json.load(open(os.path.join(RES, "run_summary.json")))
ROB = pd.read_csv(os.path.join(TAB, "robustness_by_attack.csv")).set_index("attack")
SWP = pd.read_csv(os.path.join(TAB, "alpha_sweep_summary.csv")).set_index(["attack", "alpha"])
CTL = pd.read_csv(os.path.join(TAB, "controls_summary.csv"))
COR = pd.read_csv(os.path.join(TAB, "nc_vs_image_statistics.csv")).set_index("attack")
TSCH = pd.read_csv(os.path.join(RES, "tech_benchmark_schemes.csv")).set_index(["scheme", "attack"])
VID = pd.read_csv(os.path.join(RES, "video_demo_results.csv"))
```

```
META = pd.read_csv(os.path.join(ROOT, "data", "metadata.csv"))
E1, E2, E4, BASE = AS["E1"], AS["E2"], AS["E4"], AS["metric_baselines"]
G = CTL.iloc[0]
CA, CB = CTL[CTL.control.str.startswith("a:")].iloc[0], CTL[CTL.control.str.startswith("b:")].iloc[0]
CC = CTL[CTL.control.str.startswith("c: ")].iloc[0]
PROJ, BWM, BDCT, LSB = ("Project: global DWT-DCT-SVD", "blind-watermark 0.4.4 (block DWT-DCT-SVD)",
"Baseline: 8x8 block DCT", "Baseline: spatial LSB")
ber = lambda s, a: float(TSCH.loc[(s, a), "mean_ber"])
psnr_s = lambda s: float(TSCH.reset_index().query("scheme == @s").psnr_db.mean())
tms = lambda s, c: float(TSCH.reset_index().query("scheme == @s")[c].median())
sw = lambda a, al, c: float(SWP.loc[(a, al), c])
pc = lambda x, d=0: f"{100 * x:.{d}f}%"

REFS = {}
for line in open(os.path.join(ROOT, "research", "REFERENCES.md"), encoding="utf-8"):
    m = re.match(r"\| \[(\d+)\] \| (.*?) \| (.*?) \|\s*$", line)
    if m:
        REFS[int(m.group(1))] = re.sub(r"\*([^*]+)\*", r"\1", m.group(2))
order: list[int] = []


def cite(m):
    nums = [int(x) for x in m.group(1).split(",")]
    out = []
    for n in nums:
        if n not in order:
            order.append(n)
        out.append(order.index(n) + 1)
    out.sort()
    return ", ".join(f"[{k}]" for k in out)


def C(text):
    return re.sub(r"<<([\d,]+)>>", cite, text)


doc = Document()


def set_cols(section, n, space_twips=360):
    sectPr = section._sectPr
    for el in sectPr.findall(qn("w:cols")):
        sectPr.remove(el)
    cols = OxmlElement("w:cols")
    cols.set(qn("w:num"), str(n)); cols.set(qn("w:space"), str(space_twips))
    sectPr.append(cols)


def page_setup(section):
    section.page_height, section.page_width = Cm(29.7), Cm(21.0)
    section.left_margin = section.right_margin = Cm(1.9)
    section.top_margin, section.bottom_margin = Cm(1.9), Cm(2.0)


def field(run, instr):
    for t, txt in (("begin", None), (None, instr), ("separate", None), (None, "1"), ("end", None)):
        if t:
            el = OxmlElement("w:fldChar"); el.set(qn("w:fldCharType"), t); run._r.append(el)
        elif txt == instr:
            it = OxmlElement("w:instrText"); it.set(qn("xml:space"), "preserve"); it.text = instr;
run._r.append(it)
        else:
            tt = OxmlElement("w:t"); tt.text = txt; run._r.append(tt)


s0 = doc.sections[0]
page_setup(s0)
normal = doc.styles["Normal"]
normal.font.name = FONT
normal.font.size = Pt(10)
normal.element.rPr.rFonts.set(qn("w:eastAsia"), FONT)
normal.paragraph_format.space_after = Pt(3)
normal.paragraph_format.line_spacing = 1.0
fp = s0.footer.paragraphs[0]
fp.alignment = WD_ALIGN_PARAGRAPH.CENTER
r_ = fp.add_run(); r_.font.size = Pt(9); r_.font.name = FONT; field(r_, "PAGE")
```

```
def para(text, size=10, bold=False, italic=False, align="justify", after=3, before=0, indent=None,
keep=False, color=None):
    p = doc.add_paragraph()
    p.alignment = {"justify": WD_ALIGN_PARAGRAPH.JUSTIFY, "center": WD_ALIGN_PARAGRAPH.CENTER,
"left": WD_ALIGN_PARAGRAPH.LEFT}[align]
    p.paragraph_format.space_after, p.paragraph_format.space_before = Pt(after), Pt(before)
    if indent is not None:
        p.paragraph_format.first_line_indent = Cm(indent)
    if keep:
        p.paragraph_format.keep_with_next = True
    for part in re.split(r"(\*\*[^*]+\*\*|\*[^*\s][^*]*\*|`[^`]+`)", C(text)):
        if not part:
            continue
        if part.startswith("**"):
            r = p.add_run(part[2:-2]); r.bold = True
        elif part.startswith("`"):
            r = p.add_run(part[1:-1]); r.font.name = "Courier New"; r.font.size = Pt(size - 1.5);
continue
        elif part.startswith("*") and len(part) > 2:
            r = p.add_run(part[1:-1]); r.italic = True
        else:
            r = p.add_run(part)
        r.font.size = Pt(size); r.font.name = FONT
        if bold:
            r.bold = True
        if italic:
            r.italic = True
        if color is not None:
            r.font.color.rgb = color
    return p


def body(text):
    return para(text, indent=0.45)


ROMAN = ["I", "II", "III", "IV", "V", "VI", "VII", "VIII"]
h1n = [0]
h2n = [0]


def H1(title):
    h2n[0] = 0
    p = para(f"{ROMAN[h1n[0]]}. {title.upper()}", size=9.5, align="center", before=7, after=3,
keep=True)
    for r in p.runs:
        r.font.small_caps = False
    h1n[0] += 1
    return p


def H2(title):
    h2n[0] += 1
    return para(f"{chr(64 + h2n[0])}. {title}", size=10, italic=True, align="left", before=3,
after=2, keep=True)


fig_n = [0]
tab_n = [0]


def figure(path, caption, width=8.2):
    fig_n[0] += 1
    p = doc.add_paragraph(); p.alignment = WD_ALIGN_PARAGRAPH.CENTER
    p.paragraph_format.space_after = Pt(1); p.paragraph_format.keep_with_next = True
    p.add_run().add_picture(path, width=Cm(width))
    c = para(f"Fig. {fig_n[0]}. {caption}", size=8, align="center", after=5)
    return c


def shade(cell, fill):
    tcPr = cell._tc.get_or_add_tcPr()
    shd = OxmlElement("w:shd"); shd.set(qn("w:val"), "clear"); shd.set(qn("w:color"), "auto");
shd.set(qn("w:fill"), fill); tcPr.append(shd)


def table(header, rows, caption, widths, size=7.5):
    tab_n[0] += 1
```

```
    para(f"TABLE {ROMAN[tab_n[0] - 1]}", size=8, align="center", after=0, keep=True)
    para(caption.upper() if False else caption, size=8, align="center", after=2, keep=True)
    t = doc.add_table(rows=1, cols=len(header))
    t.style = "Table Grid"; t.alignment = WD_TABLE_ALIGNMENT.CENTER; t.autofit = False
    lay = OxmlElement("w:tblLayout"); lay.set(qn("w:type"), "fixed"); t._tbl.tblPr.append(lay)
    mar = OxmlElement("w:tblCellMar")
    for side in ("left", "right"):
        e = OxmlElement(f"w:{side}"); e.set(qn("w:w"), "40"); e.set(qn("w:type"), "dxa");
mar.append(e)
    t._tbl.tblPr.append(mar)
    for i, h in enumerate(header):
        c = t.rows[0].cells[i]; c.text = ""; shade(c, "DDE3EC")
        r = c.paragraphs[0].add_run(h); r.bold = True; r.font.size = Pt(size); r.font.name = FONT
        c.paragraphs[0].paragraph_format.space_after = Pt(0)
    for row in rows:
        cells = t.add_row().cells
        for i, v in enumerate(row):
            cells[i].text = ""
            p = cells[i].paragraphs[0]; p.paragraph_format.space_after = Pt(0)
            r = p.add_run(str(v)); r.font.size = Pt(size); r.font.name = FONT
            if i > 0:
                p.alignment = WD_ALIGN_PARAGRAPH.CENTER
    for i, w in enumerate(widths):
        t.columns[i].width = Cm(w)
        for row in t.rows:
            row.cells[i].width = Cm(w)
    for row in t.rows[:-1]:
        for c in row.cells:
            for pp in c.paragraphs:
                pp.paragraph_format.keep_with_next = True
    for row in t.rows:
        trPr = row._tr.get_or_add_trPr(); cs = OxmlElement("w:cantSplit"); trPr.append(cs)
    doc.add_paragraph().paragraph_format.space_after = Pt(2)


def equation(text, n):
    p = doc.add_paragraph(); p.paragraph_format.space_after = Pt(3)
    p.paragraph_format.tab_stops.add_tab_stop(Cm(4.1), WD_TAB_ALIGNMENT.CENTER)
    p.paragraph_format.tab_stops.add_tab_stop(Cm(8.2), WD_TAB_ALIGNMENT.RIGHT)
    r = p.add_run("\t" + text); r.font.name = "Cambria Math"; r._r.rPr.rFonts.set(qn("w:eastAsia"),
"Cambria Math"); r.font.size = Pt(10)
    r2 = p.add_run(f"\t({n})"); r2.font.size = Pt(10); r2.font.name = FONT


para("Robustness and Verification Limits of a Hybrid DWT-DCT-SVD Image Watermark: A 100-Image
Evaluation with False-Positive Controls",
     size=18, bold=True, align="center", after=8, before=2)
para("Krishang Vikram Singh", size=11, align="center", after=0)
para("Roll No. B106 · Batch C1 · B.Tech Computer Engineering", size=9.5, italic=True, align="center",
after=0)
para("NMIMS Mukesh Patel School of Technology Management & Engineering (MPSTME), Mumbai, India",
size=9.5, italic=True, align="center", after=8)

n_real, n_syn = int((META.is_synthetic == 0).sum()), int((META.is_synthetic == 1).sum())
abstract = (f"**Abstract-**Singular-value watermarking in a wavelet-cosine transform domain is widely
used for ownership marking, yet reported evaluations typically use a handful of images "
            f"and score only the correlation between the extracted and the owner's logo. We re-
evaluate a classical hybrid DWT-DCT-SVD scheme on {len(META)} unique 512×512 images ({n_real} real,
{n_syn} labelled synthetic) "
            f"under 14 attacks plus an unattacked reference ({RS['attack_trials_total']} seeded
trials), with an embedding-strength sweep, false-positive controls, a comparison with an open-source
blind block-based hybrid and two classical baselines, "
            f"and an exploratory per-frame video test. The scheme is imperceptible (mean PSNR
{E1['psnr_all']['mean']:.1f} dB, SSIM {E1['ssim_all']['mean']:.3f}) and recovers the logo (NC ≥ 0.75)
in {pc(E2['pass_rate'], 1)} of trials: ≥ 95 % of images survive JPEG (Q ≥ 20), "
            f"median filtering, sharpening, rescaling and mild noise, but only {pc(ROB.loc['Rotation
15 deg'].pass_rate)} survive 15° rotation, {pc(ROB.loc['Histogram Equalization'].pass_rate)}
histogram equalisation and {pc(ROB.loc['Cropping 25%'].pass_rate)} border cropping. "
            f"Two findings qualify common practice. First, the robustness-imperceptibility trade-off
in the embedding strength α is flat for JPEG and appears only under harsher attacks. Second, a
different logo decoded with the owner's key still yields NC {CC.mean_nc:.3f} "
            f"(genuine pair {G.mean_nc:.3f}), and {pc(CA.frac_ge_0_75)}-{pc(CB.frac_ge_0_75)} of
unrelated-image controls exceed the customary NC 0.75 threshold: a high NC does not show that the
claimed watermark is present. "
            f"The blind block-based hybrid achieves lower bit-error rates under most attacks but is
{tms(BWM, 't_embed_ms') / tms(PROJ, 't_embed_ms'):.0f}× slower and weaker under rotation. Code, data
manifest and all results accompany the paper.")
para(abstract, size=9, bold=False, after=3)
```

```
para("**Index Terms-**digital watermarking, DWT, DCT, SVD, robustness, false-positive problem,
normalised correlation, PSNR, SSIM.", size=9, italic=True, after=6)

s1 = doc.add_section(WD_SECTION.CONTINUOUS)
set_cols(s1, 2)

H1("Introduction")
body("Images are copied at no cost, so owners embed invisible signatures that can later be extracted
to support an ownership claim <<17,32>>. Spatial-domain marks such as least-significant-bit (LSB)
substitution are fragile <<10>>, whereas transform-domain schemes place the mark in perceptually
significant spectral components <<1>>. "
     "SVD-based schemes for ownership protection were introduced by Liu and Tan <<2>>, combined with
the wavelet transform by Ganic and Eskicioglu <<3>>, and later cascaded with the DCT in hybrid DWT-
DCT-SVD designs <<8,9>>.")
body("Two gaps motivate this study. First, evaluations of such schemes usually rely on a few standard
images and report one number, the normalised correlation (NC) with the owner's logo. Second, SVD
schemes that store the logo's singular vectors in the key have long been criticised for false
positives and ambiguity attacks <<4,5>>, "
     "but the effect is rarely quantified, so the common NC ≥ 0.75 rule is used without knowing its
false-alarm rate.")
body("**Contributions.** (1) A seeded, fully logged evaluation of a classical DWT-DCT-SVD scheme on
100 unique images with 14 attacks and an unattacked reference. (2) Control experiments that measure
the false-positive problem and a metric analysis (a blank image already scores NC "
     f"{BASE['nc_constant_white_image']:.2f} against our logo; a mean-centred variant, ZNCC, is
added). (3) An embedding-strength study showing that the trade-off depends on the attack, correcting
a single-attack plot. "
     "(4) A measured comparison with an existing open-source blind block-based hybrid <<28>> and two
classical baselines, and an exploratory per-frame video test.")

H1("Related Work")
body("**Foundations.** Cox *et al.* <<1>> argued that a watermark must reside in perceptually
significant coefficients and be detected by correlation; Koch and Zhao <<11>> tie block-DCT embedding
to the JPEG model; Hartung and Kutter <<17>> survey the field. "
     "The DWT <<12>> and DCT <<13>> underlie JPEG-family codecs <<14>>. **SVD schemes.** Liu and Tan
<<2>> modify singular values for rightful-ownership protection; Ganic and Eskicioglu <<3>> embed in
the wavelet sub-bands; Lai and Tsai <<7>> and later hybrids <<8,9>> report improved robustness "
     "(we did not re-evaluate those claims). **Critique.** Zhang and Li <<4>> commented on <<2>>, and
Ting <<5>> studied ambiguity attacks on <<3>>: when the logo's singular vectors are part of the key,
the detector can reconstruct the key owner's logo regardless of what was embedded. "
     "**Geometry and video.** Lin *et al.* <<22>> show that rotation, scale and translation defeat
correlation detectors unless the detection domain is invariant; Hartung and Girod <<23>> treat
compressed video. Learned watermarking <<29,30>> is outside our scope.")

H1("Method")
H2("Embedding and extraction")
body("A one-level Haar DWT splits the 512×512 host into four sub-bands; the 256×256 approximation LL
is transformed by a global orthonormal 2-D DCT (not block-wise) and factorised by SVD, A =
U·diag(S)·Vᵀ. "
     "The 64×64 binary logo W is resized to the sub-band size and factorised, W = U_w·diag(S_w)·V_wᵀ.
With embedding strength α (0.10 unless stated),")
equation("S′ = S + α·S_w ,   A′ = U·diag(S′)·Vᵀ", 1)
body("followed by the inverse DCT, the inverse DWT and clipping to [0, 255]. The key is K = {S_orig,
U_w, V_wᵀ, α}. Given a (possibly attacked) image, the same transforms give singular values S_att,
and")
equation("S_w′ = (S_att - S_orig)/α ,   W′ = U_w·diag(S_w′)·V_wᵀ", 2)
body("The scheme is therefore **non-blind** (S_orig is derived from the unmarked host), and the
logo's identity is carried by the key (U_w, V_w), not by the image - the structural reason for the
false-positive behaviour measured in Section V-E. "
     "The embedding and extraction code is unchanged from the first implementation of this project;
numerical results of that implementation were reproduced to within 0.004 NC.")
H2("Attacks and protocol")
body("Fourteen attacks plus the unattacked reference: JPEG at quality 90/50/20; Gaussian noise (σ² =
0.005, 0.02 on [0,1]); salt-and-pepper (2 %); 3×3 median; Gaussian blur (σ = 1.5); unsharp-mask
sharpening; rotation by 5° and 15°; border zeroing with 25 % of each dimension removed (43.75 % of
the area); 0.5× down/up-scaling; "
     "and histogram equalisation. The watermarked image is rounded to 8 bit before attacking (primary
protocol); attacking the floating-point image, as in the first implementation, changed mean NC by
less than 0.001 except for equalisation (0.644 vs 0.638). All stochastic attacks are seeded.")
H2("Metrics")
body("Imperceptibility is measured against the unwatermarked original by PSNR = 10 log₁₀(255²/MSE)
and SSIM <<16>>. Recovery is measured by NC = Σ W·W′ / (‖W‖‖W′‖), by BER after thresholding at 127.5,
and by ZNCC, the NC of the mean-centred images. "
     f"For this logo a blank white image scores NC = {BASE['nc_constant_white_image']:.3f} and a
rank-1 approximation {BASE['nc_rank1_approximation']:.3f}, so NC alone flatters bright outputs. A
logo is called recovered if NC ≥ 0.75, a convention we keep for comparability with earlier work; we
could not trace it to a primary source and estimate its false-alarm rate in Section V-E.")

H1("Experimental Setup")
```

```
body(f"**Data.** {len(META)} unique images ({n_real} real, {n_syn} synthetic controls: gradients,
grating, noise, checkerboard) were drawn from open sample collections hosted on GitHub (OpenCV,
scikit-image, three.js textures and others), converted to grey (ITU-R BT.601), centre-cropped and
resized to 512×512; "
     "near-duplicates were removed by a difference hash. The set is a convenience sample covering
animals, urban scenes, objects, textures, portraits, paintings, documents, astronomy and medical
images with dark/bright and low/high-contrast cases; it is not a recognised benchmark, and per-image
provenance, licences and rights notes are recorded in the metadata.")
body(f"**Experiments.** E1 imperceptibility ({E1['n_images']} images); E2 robustness
({RS['attack_trials_total']} trials = 100 images × 15 conditions × 2 protocols); E3 an α sweep over
eight values and four attacks ({RS['alpha_sweep_rows_total']} rows); "
     f"E4 false-positive controls ({RS['control_rows']} rows): unmarked and marked *other* images
decoded with the key of image *i*, and the same host marked with a *different* logo decoded with the
key of logo A. "
     "S1 compares the scheme, at equal PSNR and with an identical 4095-bit payload, with `blind-
watermark` 0.4.4 <<28>> (blind, block-based DWT-DCT-SVD), an 8×8 block-DCT baseline and an LSB
baseline on 30 random real images. V1 applies the scheme frame by frame to two real clips.")
body("**Statistics and reproducibility.** Means carry 95 % bootstrap intervals, pass rates Wilson
intervals, associations Spearman correlations. Two complete executions gave identical metrics
(maximum absolute difference 0.0); 43 unit tests pass. "
     "Software: NumPy <<19>>, SciPy <<20>>, scikit-image <<21>>, PyWavelets <<18>>, Pillow, OpenCV
and FFmpeg; four vCPUs, single-thread BLAS.")

H1("Results and Discussion")
H2("Imperceptibility")
body(f"The mean PSNR of the stored 8-bit watermarked image is {E1['psnr_all']['mean']:.2f} dB (range
{E1['psnr_all']['min']:.1f}-{E1['psnr_all']['max']:.1f}; none below 30 dB) and the mean SSIM is
{E1['ssim_all']['mean']:.3f} (Fig. 1). "
     "PSNR is almost host-independent (82 of 100 images lie in 35.5-35.75 dB) because the added
energy is set by the logo's singular values. SSIM exposes exceptions: the near-black astronomy image
reaches 35.9 dB but SSIM 0.58, and three images fall below 0.98; PSNR alone would have hidden this.")
figure(os.path.join(FIG, "fig1_sample.png"), "Original, watermarked and amplified difference
(image_066, α = 0.10).")

H2("Robustness")
rows = []
for a in ROB.index:
    x = ROB.loc[a]
    rows.append([a.replace("Gaussian ", "Gauss. ").replace("Histogram Equalization", "Hist.
equalisation"), f"{x.mean_nc:.3f} ({x.ci_lo:.3f}-{x.ci_hi:.3f})", f"{100 * x.pass_rate:.0f}",
f"{x.mean_zncc:.3f}", f"{x.mean_ber:.3f}"])
table(["Attack", "mean NC (95 % CI)", "≥0.75 (%)", "ZNCC", "BER"], rows, f"Robustness over 100 images
per condition (8-bit protocol).", [2.45, 2.75, 1.0, 0.95, 0.95], size=7)
body(f"{E2['n_pass']} of {E2['n_trials']} trials ({pc(E2['pass_rate'], 1)}) reach NC ≥ 0.75 including
the 100 unattacked references, as counted in earlier work; counting only the 14 attacks it is
{pc(E2['pass_rate_attacks_only'], 1)} (mean NC {E2['mean_nc_attacks_only']:.3f}). "
     f"ZNCC is about 0.02 lower than NC and yields {pc(E2['zncc_pass_rate'], 1)} overall, so the
conclusions are not an artefact of the brightness offset. Table I and Fig. 2 show strong robustness
to compression and filtering and failure under cropping and equalisation. "
     f"Rotation is weaker than sometimes assumed: mean NC {ROB.loc['Rotation 5 deg'].mean_nc:.3f} at
5° and {ROB.loc['Rotation 15 deg'].mean_nc:.3f} at 15° (92 % and 72 % pass). The scheme's robustness
to JPEG stems from the SVD stage; the DCT is global and not aligned with JPEG's 8×8 blocks.")
figure(os.path.join(FIG, "fig2_passrate.png"), "Share of the 100 images with NC ≥ 0.75 per attack.")

H2("Which images fail")
body(f"Robustness is image-dependent. NC after attack is negatively correlated with image contrast
(Spearman ρ: salt-and-pepper {COR.loc['Salt & Pepper (2%)'].rho_contrast:.2f}, Gaussian noise σ²=0.02
{COR.loc['Gaussian Noise (var=0.02)'].rho_contrast:.2f}, 15° rotation {COR.loc['Rotation 15
deg'].rho_contrast:.2f}); "
     f"histogram equalisation instead tracks mean brightness (ρ = {COR.loc['Histogram
Equalization'].rho_mean_intensity:.2f}). Document-text images are the weakest category under noise
(mean NC 0.53). These are associations with uncorrected p-values (n = 100), not demonstrated
causes.")

H2("Embedding strength")
j, g = "JPEG Q=50", "Gaussian Noise (var=0.02)"
body(f"Mean PSNR falls from {sw('No Attack', 0.02, 'psnr'):.1f} dB (α = 0.02) to {sw('No Attack',
0.10, 'psnr'):.1f} dB (0.10) and {sw('No Attack', 0.40, 'psnr'):.1f} dB (0.40). After JPEG Q = 50 the
mean NC is essentially flat in α ({sw(j, 0.02, 'nc_mean'):.3f} at 0.02, {sw(j, 0.40, 'nc_mean'):.3f}
at 0.40). "
     f"The trade-off appears under harsher attacks: for Gaussian noise (σ² = 0.02) the pass rate
rises from {pc(sw(g, 0.02, 'pass_rate'))} at α = 0.02 to {pc(sw(g, 0.10, 'pass_rate'))} at 0.10 and
{pc(sw(g, 0.40, 'pass_rate'))} at 0.40 while PSNR drops below 30 dB for α ≥ 0.20 (Fig. 3). "
     "A single-attack plot on a second, zoomed NC axis - and a JPEG simulation that truncates
floating-point pixels, which lowers NC only at α = 0.02 from 0.981 to 0.929 - had earlier suggested a
steep trade-off for JPEG; we reproduced that figure exactly and traced the artefact.")
figure(os.path.join(FIG, "fig3_alpha.png"), "Effect of α: PSNR (top) and pass rate after four attacks
(bottom), 100 images.")
```

```
H2("False-positive controls")
rows = [[x.control.split(":")[0] if ":" in x.control[:3] else "genuine", x.control.split(": ", 1)[-1]
[:46] if ": " in x.control else x.control[:46], f"{x.mean_nc:.3f}", f"{x.p95:.3f}", f"{100 *
x.frac_ge_0_75:.1f}"] for x in CTL.itertuples()]
rows = [["ref", "genuine: own key, clean image", f"{G.mean_nc:.3f}", f"{G.p95:.3f}", f"{100 *
G.frac_ge_0_75:.1f}"],
        ["a", "other image, unmarked, key of i", f"{CA.mean_nc:.3f}", f"{CA.p95:.3f}", f"{100 *
CA.frac_ge_0_75:.1f}"],
        ["b", "other image, marked, wrong key", f"{CB.mean_nc:.3f}", f"{CB.p95:.3f}", f"{100 *
CB.frac_ge_0_75:.1f}"],
        ["c", "other logo B, decoded with key A", f"{CC.mean_nc:.3f}", f"{CC.p95:.3f}", f"{100 *
CC.frac_ge_0_75:.1f}"]]
table(["", "Control (NC against the owner's logo)", "mean", "p95", "≥0.75 (%)"], rows, "False-
positive controls (E4).", [0.7, 4.6, 1.0, 1.0, 1.1], size=7)
body(f"Embedding a *different* logo in the same host and decoding with the key of logo A returns the
owner's logo: NC {CC.mean_nc:.3f}, indistinguishable from the genuine {G.mean_nc:.3f} (Fig. 4, 200
trials) - the extractor reproduces the key owner's logo whatever singular values it finds. "
     f"Unrelated images decoded with a wrong key score {CA.mean_nc:.2f}-{CB.mean_nc:.2f} on average,
with 95th percentile {E4['wrong_image_p95']:.2f}; {pc(CA.frac_ge_0_75, 1)} (unmarked) and
{pc(CB.frac_ge_0_75, 1)} (marked) exceed 0.75, but only about 1 % exceed 0.90. "
     f"A detector calibrated to a 5 % false-alarm rate would reject
{pc(E4['attacked_genuine_below_wrong_p95'])} of the attacked genuine trials. "
     "Decoding the unmarked host with its own key gives NC = 0 by construction (S_att = S_orig) and
must not be read as evidence. "
     "An attempt to cure the problem by embedding the logo's principal component made it worse (an
unmarked image still decoded at NC 0.98), because the key then contains the whole logo. Claims of
ownership therefore require an additional safeguard, for example an image-bound signature, which is
future work.")
figure(os.path.join(FIG, "fig4_fpp.png"), "Mean NC for the genuine pair and for the controls (dashed:
0.75).")

H2("Comparison with existing technology")
rows = []
for a in ["JPEG Q=50", "JPEG Q=20", "Gaussian Noise (var=0.02)", "Rotation 5 deg", "Cropping 25%",
"Scaling 0.5x"]:
    rows.append([a.replace("Gaussian Noise (var=0.02)", "Gauss. noise").replace(" deg",
"°").replace("Cropping 25%", "Cropping").replace("Scaling 0.5x", "Scaling 0.5×")] + [f"{ber(s,
a):.3f}" for s in (PROJ, BWM, BDCT, LSB)])
rows.append(["PSNR (dB)"] + [f"{psnr_s(s):.1f}" for s in (PROJ, BWM, BDCT, LSB)])
rows.append(["embed/extract (ms)"] + [f"{tms(s, 't_embed_ms'):.0f}/{tms(s, 't_extract_ms'):.0f}" for
s in (PROJ, BWM, BDCT, LSB)])
table(["BER (30 images)", "Project", "blind-wm.", "block DCT", "LSB"], rows, "Same 4095-bit payload
and attacks.", [2.7, 1.4, 1.5, 1.5, 1.2], size=7)
body(f"At equal PSNR the project is far more robust than the LSB and 8×8 block-DCT baselines, which
approach chance (BER 0.5) under JPEG, rescaling and rotation (Fig. 5). It is, however, **not**
superior to the existing blind block-based hybrid <<28>>: that package has lower BER under JPEG Q =
50 "
     f"({ber(BWM, 'JPEG Q=50'):.3f} vs {ber(PROJ, 'JPEG Q=50'):.3f}), noise, blur, scaling and
cropping at a higher PSNR ({psnr_s(BWM):.1f} vs {psnr_s(PROJ):.1f} dB) without a key, but is
{tms(BWM, 't_embed_ms') / tms(PROJ, 't_embed_ms'):.0f}× slower to embed and "
     f"{tms(BWM, 't_extract_ms') / tms(PROJ, 't_extract_ms'):.0f}× slower to extract, and worse under
rotation ({ber(BWM, 'Rotation 5 deg'):.3f} vs {ber(PROJ, 'Rotation 5 deg'):.3f}) and JPEG Q = 20
({ber(BWM, 'JPEG Q=20'):.3f} vs {ber(PROJ, 'JPEG Q=20'):.3f}). "
     "Its YUV design and our channel-wise attacks may favour it. Library-level checks showed that
DCT, SVD, Haar DWT and JPEG implementations in SciPy, OpenCV, NumPy and Pillow agree to ≤ 1.5·10⁻¹⁶
(relative) or bit-identically, "
     "and that OpenCV is faster for attack operations (e.g. 3×3 median ≈ 340×, partly because it
works on 8-bit data); the whole experiment runs in about 80 s on four cores, so speed was not a
constraint.")
figure(os.path.join(FIG, "fig5_schemes.png"), "Mean bit-error rate of four schemes (lower is better;
0.5 = chance).")

H2("Video feasibility")
body(f"Applied frame by frame (each frame with its own key) to 60 frames of two real clips and re-
encoded with H.264 (CRF 18/23/28) and MJPEG (q 5/15), the logo was recovered in all
{int(VID.frames_decoded.sum())} decoded frames (NC ≥ {VID.nc_min.min():.2f}); embedding takes about
32-41 ms per frame on one core. "
     "This is a feasibility check only: temporal attacks, colour video and hardware codecs were not
tested, and a naive frame-drop test with position-aligned keys still gave NC 0.97 because
neighbouring frames are nearly identical - a further symptom of key-dependence.")

H1("Limitations")
body("(i) The false-positive problem is measured, not solved. (ii) The scheme is non-blind and needs
a key of about 2 MB per image. (iii) Border cropping and rotation beyond 5° defeat it; equalisation
is only partly survived. (iv) Grey-scale only; clean extraction is not lossless (BER ≈ 0.9 % from
logo resampling). "
```

```
    "(v) The 94 real images are a convenience sample from GitHub-hosted sets with mostly
undocumented photograph rights, standardised to grey squares; category-level results rest on 1-14
images. "
    "(vi) Only one existing watermarking package was benchmarked; learned methods <<29,30>> and GPU
variants were not. (vii) Timing varies by 5-10 % between runs. (viii) The video test covers 120
distinct frames from two clips.")

H1("Conclusion and Future Work")
body(f"A classical DWT-DCT-SVD watermark is imperceptible (PSNR {E1['psnr_all']['mean']:.1f} dB, SSIM
{E1['ssim_all']['mean']:.3f}) and survives everyday processing - JPEG, median filtering, sharpening,
rescaling, mild noise - on at least 95 % of 100 images, but not cropping, equalisation or strong
rotation. "
    "Larger-scale testing changes two common conclusions: the α trade-off depends on the attack, and
a high normalised correlation does not prove that the claimed watermark is present, because a
different logo decoded with the owner's key scores almost the same. "
    "An existing blind block-based hybrid outperforms the scheme on most attacks, so the scheme's
value lies in its transparency and compactness, not in robustness. "
    "Future work: bind the logo to the image (signature of image features), block-based redundant
embedding for cropping, geometric re-synchronisation <<22>>, colour via luminance, and index-aware
keys with temporal redundancy for video <<23>>.")

H1_title = para("REFERENCES", size=9.5, align="center", before=7, after=3, keep=True)
for k, old in enumerate(order, 1):
    p = para(f"[{k}] {REFS[old]}", size=7.5, align="justify", after=1.5)
    p.paragraph_format.left_indent = Cm(0.6); p.paragraph_format.first_line_indent = Cm(-0.6)

s2 = doc.add_section(WD_SECTION.NEW_PAGE)
set_cols(s2, 1)
page_setup(s2)
s2.left_margin = s2.right_margin = Cm(1.6)
para("APPENDIX A - SOURCE CODE", size=14, bold=True, align="center", after=4)
para("Python 3.13 · NumPy · SciPy · scikit-image · PyWavelets · Pillow · pandas · matplotlib (OpenCV
and FFmpeg for the video test). "
    "Repository: github.com/krishangvikramsingh/claude-code, branch claude/elegant-tesla-c3k9fk,
folder final_submission/source_code/src. "
    "The listing contains the algorithm, attacks, metrics, dataset builder, batch experiment,
analysis and benchmark scripts; "
    "the data manifest (dataset_manifest.py), the tests and the report/presentation generators are
in the repository.", size=9, align="justify", after=6)

CODE_FILES = [("watermark_core.py", "Hybrid DWT-DCT-SVD embedding and extraction (algorithm core)"),
("metrics.py", "PSNR, SSIM, MSE, NC, ZNCC, BER"),
              ("attacks.py", "The 14 attacks plus the unattacked reference, seeded"), ("logo.py",
"Procedural 64×64 binary watermark logo"),
              ("image_io.py", "Safe image loading and standardisation"), ("run_experiment.py",
"Original two-image experiment (reproduces the first report)"),
              ("build_dataset.py", "Reproducible 100-image dataset builder"),
("run_batch_experiment.py", "100-image batch experiment (E1-E4)"),
              ("analysis.py", "Statistics, tables and figures from the recorded results"),
("benchmark_tech.py", "Library and scheme benchmarks (S1, S2)"),
              ("video_demo.py", "Exploratory per-frame video test (V1)"), ("make_diagram.py",
"Architecture diagrams")]
mono = "Consolas"
total_lines = 0
for i, (fn, desc) in enumerate(CODE_FILES, 1):
    path = os.path.join(HERE, fn)
    src = open(path, encoding="utf-8").read().rstrip("\n").split("\n")
    total_lines += len(src)
    h = para(f"A.{i}  {fn}  -  {desc}  ({len(src)} lines)", size=10.5, bold=True, align="left",
before=10, after=3, keep=True)
    for line in src:
        p = doc.add_paragraph()
        p.paragraph_format.space_after = Pt(0); p.paragraph_format.line_spacing = 1.0
        r = p.add_run(line.replace("\t", "    ") if line else " ")
        r.font.name = mono; r._r.rPr.rFonts.set(qn("w:eastAsia"), mono); r.font.size = Pt(7.5)

doc.core_properties.title = ROLL
doc.core_properties.author = "Krishang Vikram Singh"
doc.core_properties.subject = "Research paper and source code - Image and Video Processing"
os.makedirs(OUTDIR, exist_ok=True)
out = os.path.join(OUTDIR, f"{ROLL}.docx")
doc.save(out)
print("saved", out, "| references:", len(order), "| code lines:", total_lines, "| figures:",
fig_n[0], "| tables:", tab_n[0])
subprocess.run(["soffice", "--headless", "--convert-to", "pdf", "--outdir", OUTDIR, out],
capture_output=True, timeout=300)
```

5.5 validate_submission.py

Checks the finished package. 82 lines.

```
import json
import os
import re
import sys

import numpy as np
import pandas as pd
from PIL import Image

ROOT = os.path.abspath(os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", ".."))
P = lambda *a: os.path.join(ROOT, *a)
lines, fails = [], []


def check(name, ok, detail=""):
    lines.append(f"[{'PASS' if ok else 'FAIL'}] {name}" + (f" - {detail}" if detail else ""))
    if not ok:
        fails.append(name)


req = ["README.md", "source_code/src/watermark_core.py",
"source_code/original_backup/watermark_core.py", "source_code/requirements.txt",
       "source_code/config.json", "tests/test_watermark_core.py", "data/metadata.csv",
"data/README.md",
       "results/results_attacks.csv", "results/results_images.csv",
"results/results_alpha_sweep.csv", "results/results_controls.csv",
       "results/run_summary.json", "results/analysis_summary.json",
"results/tech_benchmark_components.csv", "results/tech_benchmark_schemes.csv",
       "results/video_demo_results.csv", "research/LITERATURE_REVIEW.md", "research/REFERENCES.md",
"research/TECHNOLOGY_COMPARISON.md",
       "documentation/PROJECT_AUDIT.md", "documentation/TRANSFORMATIONS_REFERENCE.md",
"documentation/EXPERIMENT_METHODOLOGY.md",
       "documentation/CHANGELOG.md", "report/Final_Project_Report.docx",
"report/Final_Project_Report.pdf",
       "presentation/Final_Presentation.pptx", "presentation/Final_Presentation.pdf",
"presentation/SPEAKER_NOTES.md"]
missing = [f for f in req if not os.path.exists(P(f))]
check("all required deliverables present", not missing, f"missing: {missing}" if missing else
f"{len(req)} files")

meta = pd.read_csv(P("data", "metadata.csv"))
check("metadata has 100 rows and unique IDs", len(meta) == 100 and meta.image_id.is_unique)
arrs = [np.asarray(Image.open(P("data", "original", i + ".png"))) for i in meta.image_id]
check("100 images are 512x512 uint8 and pixel-unique", all(a.shape == (512, 512) and a.dtype ==
np.uint8 for a in arrs) and len({a.tobytes() for a in arrs}) == 100)
check("synthetic images labelled", set(meta[meta.is_synthetic == 1].category) ==
{"synthetic_control"} and (meta.is_synthetic == 1).sum() == 6)
check("real images have source URL, licence, rights note, SHA-256", meta[meta.is_synthetic == 0]
[["source_url", "repository_license", "rights_note", "source_sha256"]].notna().all().all())
check("processing status recorded for every image", (meta.processing_status == "ok").all() and
(meta.attack_trials_ok == 30).all())

att = pd.read_csv(P("results", "results_attacks.csv")); img = pd.read_csv(P("results",
"results_images.csv"))
sw = pd.read_csv(P("results", "results_alpha_sweep.csv")); ctl = pd.read_csv(P("results",
"results_controls.csv"))
check("3000 attack trials, all ok", len(att) == 3000 and (att.status == "ok").all())
check("every image x protocol has 15 conditions", att.groupby(["image_id",
"protocol"]).size().eq(15).all())
check("4000 sweep rows ok, 1600 control rows", len(sw) == 4000 and (sw.status == "ok").all() and
len(ctl) == 1600)
check("100 watermarked PNGs exist and were validated", img.output_valid.all() and
all(os.path.exists(P("outputs", "watermarked", i + "_wm.png")) for i in meta.image_id))
check("metrics in valid ranges", att.nc.between(0, 1).all() and att.ber.between(0, 1).all() and
att.zncc.between(-1, 1).all())

AS = json.load(open(P("results", "analysis_summary.json")))
a = att[att.protocol == "uint8"]
pass_rate = float((a.nc >= 0.75).mean()); mean_nc = float(a.nc.mean())
check("headline pass rate recomputed from raw CSV matches summary", abs(pass_rate - AS["E2"]
["pass_rate"]) < 1e-12, f"{pass_rate:.4f}")
check("mean PSNR recomputed matches summary", abs(img.psnr_db.mean() - AS["E1"]["psnr_all"]["mean"])
< 1e-9, f"{img.psnr_db.mean():.3f} dB")
fp = ctl[ctl.control.str.startswith("c: ")].nc.mean()
```

```
check("false-positive control mean NC recomputed", abs(fp - 0.975) < 0.0015, f"{fp:.4f}")

from docx import Document
from pptx import Presentation
d = Document(P("report", "Final_Project_Report.docx"))
dtext = "\n".join(p.text for p in d.paragraphs) + "\n".join(c.text for t in d.tables for r in t.rows
for c in r.cells)
pres = Presentation(P("presentation", "Final_Presentation.pptx"))
ptext = "\n".join(sh.text_frame.text for s in pres.slides for sh in s.shapes if sh.has_text_frame)
check("report opens (python-docx) with figures and tables", len(d.inline_shapes) >= 15 and
len(d.tables) >= 15, f"{len(d.inline_shapes)} images, {len(d.tables)} tables")
check("deck opens (python-pptx) with 24 slides, all with notes", len(pres.slides) == 24 and
all(s.has_notes_slide and s.notes_slide.notes_text_frame.text.strip() for s in pres.slides))
bad = re.compile(r"\b(TODO|TBD|lorem ipsum|placeholder|XXX|FIXME)\b", re.I)
check("no placeholder text in report/deck", not bad.search(dtext) and not bad.search(ptext))
check("report and deck agree on headline numbers", all(x in dtext for x in ("85.1 %", "0.975")) and
all(x in ptext for x in ("85.1 %", "0.98")), "85.1 % pass rate; wrong-logo NC 0.975")
check("report mentions Canva not edited / no template", "Canva" in dtext)
pdf_pages = int(re.search(r"Pages:\s+(\d+)", os.popen(f'pdfinfo "{P("report",
"Final_Project_Report.pdf")}"').read()).group(1))
check("report PDF has pages and TOC page numbers resolved", pdf_pages >= 30, f"{pdf_pages} pages")
toc_zero = len(re.findall(r"\.{5,}0\s*$", os.popen(f'pdftotext -layout "{P("report",
"Final_Project_Report.pdf")}" - 2>/dev/null').read(), re.M))
check("no unresolved (0) page numbers in contents/lists", toc_zero == 0)
refs = open(P("research", "REFERENCES.md"), encoding="utf-8").read()
cited = set(re.findall(r"\[(\d+)\]", dtext))
listed = set(re.findall(r"^\| \[(\d+)\] ", refs, re.M))
check("every citation number in the report exists in the reference list", cited <= listed,
f"unlisted: {sorted(cited - listed, key=int)}" if not cited <= listed else f"{len(cited)} cited /
{len(listed)} listed")
orig_same = all(open(P("source_code/original_backup", f), "rb").read() != open(P("source_code/src",
f), "rb").read() for f in ["watermark_core.py", "metrics.py", "attacks.py"])
check("original source files preserved separately (and modified versions differ)", orig_same)

out = "\n".join(lines) + f"\n\n{len(lines) - len(fails)}/{len(lines)} checks passed\n"
open(P("results", "validation_report.txt"), "w", encoding="utf-8").write(out)
print(out)
sys.exit(1 if fails else 0)
```

Part 6: Tests

Automated tests (pytest).

6.1 conftest.py

Test configuration. 6 lines.

```
import os
import sys

SRC = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "source_code", "src"))
if SRC not in sys.path:
    sys.path.insert(0, SRC)
```

6.2 test_watermark_core.py

Tests for embedding and extraction. 83 lines.

```
import numpy as np
import pytest

from logo import make_watermark_logo
from metrics import normalized_correlation, psnr
from watermark_core import dct2, embed_watermark, extract_watermark, idct2


@pytest.fixture(scope="module")
def host():
    rng = np.random.default_rng(0)
    from scipy import ndimage
    return np.clip(128 + 40 * ndimage.gaussian_filter(rng.standard_normal((256, 256)), 3) * 6, 0,
255)


@pytest.fixture(scope="module")
def logo():
    return make_watermark_logo(32, "TEST")


def test_dct_roundtrip():
    a = np.random.default_rng(1).random((16, 16))
    assert np.allclose(idct2(dct2(a)), a)


def test_embed_shape_range_and_key(host, logo):
    w, key = embed_watermark(host, logo, alpha=0.1)
    assert w.shape == host.shape
    assert w.min() >= 0 and w.max() <= 255
    assert {"S_original", "Uw", "Vtw", "alpha", "target_shape", "wm_shape"} <= set(key)


def test_clean_roundtrip_recovers_logo(host, logo):
    w, key = embed_watermark(host, logo, alpha=0.1)
    ext = extract_watermark(w, key)
    assert ext.shape == logo.shape
    assert normalized_correlation(logo, ext) > 0.9


def test_imperceptibility_alpha_monotone(host, logo):
    p = [psnr(host, embed_watermark(host, logo, alpha=a)[0]) for a in (0.02, 0.1, 0.4)]
    assert p[0] > p[1] > p[2]


def test_unwatermarked_with_own_key_gives_zero(host, logo):
    _, key = embed_watermark(host, logo, alpha=0.1)
    assert normalized_correlation(logo, extract_watermark(host, key)) == 0.0


def test_odd_sized_host(logo):
    rng = np.random.default_rng(2)
    h = rng.random((101, 77)) * 255
    w, key = embed_watermark(h, logo)
    assert w.shape == h.shape
    assert extract_watermark(w, key).shape == logo.shape
```

```
def test_extract_handles_resized_image(host, logo):
    w, key = embed_watermark(host, logo)
    ext = extract_watermark(w[:200, :200], key)
    assert ext.shape == logo.shape


@pytest.mark.parametrize("bad", [np.zeros((4, 4)), np.zeros((8, 8, 3)), np.full((16, 16), np.nan),
np.zeros(16)])
def test_invalid_host_rejected(bad, logo):
    with pytest.raises((ValueError, TypeError)):
        embed_watermark(bad, logo)


@pytest.mark.parametrize("alpha", [0, -0.1, float("nan")])
def test_invalid_alpha(host, logo, alpha):
    with pytest.raises(ValueError):
        embed_watermark(host, logo, alpha=alpha)


def test_invalid_subband(host, logo):
    with pytest.raises(ValueError):
        embed_watermark(host, logo, subband="XX")


def test_string_array_rejected(logo):
    with pytest.raises(TypeError):
        embed_watermark(np.array([["a"] * 16] * 16), logo)
```

6.3 test_metrics.py

Tests for the metrics. 48 lines.

```
import numpy as np
import pytest

from metrics import (PSNR_CAP_DB, bit_error_rate, mse, normalized_correlation, psnr, ssim,
                     zero_normalized_correlation)


def test_psnr_identical_is_capped():
    a = np.random.default_rng(0).random((32, 32)) * 255
    assert psnr(a, a) == PSNR_CAP_DB


def test_psnr_known_value():
    a = np.zeros((16, 16)); b = np.full((16, 16), 10.0)
    assert mse(a, b) == pytest.approx(100.0)
    assert psnr(a, b) == pytest.approx(10 * np.log10(255 ** 2 / 100.0))


def test_ssim_identical_is_one():
    a = np.random.default_rng(1).random((32, 32)) * 255
    assert ssim(a, a) == pytest.approx(1.0)


def test_nc_basic():
    w = np.zeros((8, 8)); w[2:5, 2:5] = 255
    assert normalized_correlation(w, w) == pytest.approx(1.0)
    assert normalized_correlation(w, np.zeros_like(w)) == 0.0
    assert normalized_correlation(w, np.full_like(w, 255)) == pytest.approx(np.sqrt(9 / 64))


def test_zncc_properties():
    rng = np.random.default_rng(3)
    w = (rng.random((16, 16)) > .8) * 255.0
    assert zero_normalized_correlation(w, w) == pytest.approx(1.0)
    assert zero_normalized_correlation(w, 255 - w) == pytest.approx(-1.0)
    assert zero_normalized_correlation(w, np.full_like(w, 5)) == 0.0
    assert zero_normalized_correlation(w, 0.5 * w + 30) == pytest.approx(1.0)


def test_ber():
    a = np.zeros((4, 4)); b = a.copy(); b[0, :] = 255
    assert bit_error_rate(a, b) == pytest.approx(0.25)
```

```
@pytest.mark.parametrize("fn", [psnr, ssim, normalized_correlation, zero_normalized_correlation,
bit_error_rate, mse])
def test_shape_mismatch_raises(fn):
    with pytest.raises(ValueError):
        fn(np.zeros((8, 8)), np.zeros((9, 9)))
```

6.4 test_attacks.py

Tests for the attacks. 43 lines.

```
import numpy as np
import pytest

from attacks import (ATTACK_NAMES, cropping_attack, jpeg_compression, make_attack_suite,
rotation_attack,
                     scaling_attack)


@pytest.fixture(scope="module")
def img():
    return np.random.default_rng(0).random((128, 128)) * 255


def test_suite_complete_and_shape_preserving(img):
    suite = make_attack_suite(1)
    assert list(suite) == ATTACK_NAMES and len(suite) == 15
    for name, fn in suite.items():
        out = fn(img)
        assert out.shape == img.shape, name
        assert np.isfinite(out).all(), name


def test_stochastic_attacks_are_seeded(img):
    a = make_attack_suite(7)["Gaussian Noise (var=0.02)"](img)
    b = make_attack_suite(7)["Gaussian Noise (var=0.02)"](img)
    c = make_attack_suite(8)["Gaussian Noise (var=0.02)"](img)
    assert np.array_equal(a, b) and not np.array_equal(a, c)


def test_cropping_removes_expected_area():
    out = cropping_attack(np.full((512, 512), 200.0), 0.25)
    kept = (out > 0).mean()
    assert kept == pytest.approx(0.5625, abs=1e-6)


def test_jpeg_quality_bounds(img):
    with pytest.raises(ValueError):
        jpeg_compression(img, 0)
    assert jpeg_compression(img, 90).shape == img.shape


def test_zero_rotation_and_unit_scale_identity(img):
    assert np.allclose(rotation_attack(img, 0), img, atol=1e-6)
    assert np.allclose(scaling_attack(img, 1.0), img, atol=1e-6)
```

6.5 test_image_io.py

Tests for image loading. 66 lines.

```
import numpy as np
import pytest
from PIL import Image

from image_io import ImageLoadError, load_gray_square, save_gray_png, verify_png


def _save(tmp_path, name, im):
    p = tmp_path / name
    im.save(p)
    return str(p)


def test_rgb_wide_image_is_cropped_to_square(tmp_path):
    arr = np.zeros((100, 300, 3), np.uint8); arr[..., 0] = 255
    a, info = load_gray_square(_save(tmp_path, "w.png", Image.fromarray(arr)), 64)
    assert a.shape == (64, 64) and (info.width, info.height) == (300, 100)
    assert a.mean() == pytest.approx(0.299 * 255, abs=1.0)
```

```
def test_rgba_composited_on_white(tmp_path):
    arr = np.zeros((64, 64, 4), np.uint8)
    a, _ = load_gray_square(_save(tmp_path, "t.png", Image.fromarray(arr)), 32)
    assert a.min() > 250


def test_16bit_rescaled(tmp_path):
    arr = (np.arange(64 * 64).reshape(64, 64) * 10).astype(np.uint16)
    a, _ = load_gray_square(_save(tmp_path, "s.png", Image.fromarray(arr)), 32)
    assert 245 <= a.max() <= 255 and 0 <= a.min() <= 10


def test_palette_and_bilevel(tmp_path):
    im = Image.fromarray((np.random.default_rng(0).random((64, 64)) > .5).astype(np.uint8) *
255).convert("1")
    a, _ = load_gray_square(_save(tmp_path, "b.png", im), 32)
    assert a.shape == (32, 32)


def test_missing_empty_corrupt_unsupported(tmp_path):
    with pytest.raises(ImageLoadError):
        load_gray_square(str(tmp_path / "nope.png"))
    e = tmp_path / "empty.png"; e.write_bytes(b"")
    with pytest.raises(ImageLoadError):
        load_gray_square(str(e))
    c = tmp_path / "corrupt.png"; c.write_bytes(b"\x89PNG\r\n\x1a\n" + b"garbage" * 20)
    with pytest.raises(ImageLoadError):
        load_gray_square(str(c))
    t = tmp_path / "doc.txt"; t.write_text("not an image")
    with pytest.raises(ImageLoadError):
        load_gray_square(str(t))


def test_tiny_and_constant_16bit_rejected(tmp_path):
    with pytest.raises(ImageLoadError):
        load_gray_square(_save(tmp_path, "tiny.png", Image.fromarray(np.zeros((8, 8), np.uint8))))
    with pytest.raises(ImageLoadError):
        load_gray_square(_save(tmp_path, "c16.png", Image.fromarray(np.full((64, 64), 500,
np.uint16))))


def test_save_and_verify_roundtrip(tmp_path):
    p = str(tmp_path / "sub" / "x.png")
    save_gray_png(np.full((20, 30), 300.7), p)
    assert verify_png(p, (20, 30))
    assert not verify_png(p, (30, 20))
    assert not verify_png(str(tmp_path / "missing.png"))
    assert np.asarray(Image.open(p)).max() == 255
```

6.6 test_dataset_and_results.py

Checks on the dataset and results. 45 lines.

```
import os

import numpy as np
import pandas as pd
import pytest
from PIL import Image

ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
META = os.path.join(ROOT, "data", "metadata.csv")
pytestmark = pytest.mark.skipif(not os.path.exists(META), reason="dataset not built")


@pytest.fixture(scope="module")
def meta():
    return pd.read_csv(META)


def test_exactly_100_unique_images(meta):
    assert len(meta) == 100
    assert meta.image_id.is_unique and meta.key.is_unique
    hashes = []
    for iid in meta.image_id:
        a = np.asarray(Image.open(os.path.join(ROOT, "data", "original", iid + ".png")))
```

```
        assert a.shape == (512, 512) and a.dtype == np.uint8
        hashes.append(a.tobytes().__hash__())
    assert len(set(hashes)) == 100


def test_synthetic_labelled(meta):
    assert set(meta[meta.is_synthetic == 1].category) == {"synthetic_control"}
    assert (meta[meta.is_synthetic == 0].category != "synthetic_control").all()
    assert meta[meta.is_synthetic == 0].source_sha256.str.len().eq(64).all()


@pytest.mark.skipif(not os.path.exists(os.path.join(ROOT, "results", "results_attacks.csv")),
reason="no results")
def test_batch_completeness(meta):
    att = pd.read_csv(os.path.join(ROOT, "results", "results_attacks.csv"))
    assert len(att) == 100 * 15 * 2
    assert (att.status == "ok").all()
    assert att.groupby(["image_id", "protocol"]).size().eq(15).all()
    assert att.nc.between(0, 1).all() and att.ber.between(0, 1).all()
    imgs = pd.read_csv(os.path.join(ROOT, "results", "results_images.csv"))
    assert imgs.output_valid.all() and (imgs.status == "ok").all()
    for iid in meta.image_id:
        assert os.path.exists(os.path.join(ROOT, "outputs", "watermarked", iid + "_wm.png"))
```

Part 7: Configuration

Settings and dependencies.

7.1 config.json

Experiment settings. 37 lines.

```
{
  "alpha": 0.1,
  "image_size": 512,
  "logo_size": 64,
  "logo_text": "CYBERION",
  "other_logos": [
    "OTHERLOGO",
    "OWNER-B"
  ],
  "protocols": [
    "uint8",
    "float"
  ],
  "primary_protocol": "uint8",
  "base_seed": 20260713,
  "alphas": [
    0.02,
    0.05,
    0.08,
    0.1,
    0.15,
    0.2,
    0.3,
    0.4
  ],
  "sweep_attacks": [
    "JPEG Q=50",
    "Gaussian Noise (var=0.02)",
    "Gaussian Blur (sigma=1.5)",
    "Rotation 5 deg"
  ],
  "n_controls_per_image": 5,
  "nc_threshold": 0.75,
  "data_dir": "../../data",
  "results_dir": "../../results",
  "outputs_dir": "../../outputs"
}
```

7.2 requirements.txt

Python packages used. 13 lines.

```
numpy==2.5.3
scipy==1.18.1
scikit-image==0.26.0
PyWavelets>=1.8
Pillow==12.3.0
pandas==3.0.5
matplotlib==3.11.2
opencv-python-headless==5.0.0.93
blind-watermark==0.4.4
imagehash==4.3.2
python-docx==1.2.0
python-pptx==1.0.2
pytest>=8
```